

What Algorithms Can Transformers Learn In-Context?

Online selection of online algorithms

Senem ışık
Stanford University

Nico Christianson
Stanford University

Ellen Vitercik
Stanford University

Amin Saberi
Stanford University

June 10, 2026

Contents

1	Introduction: In-Context Learning and Online Algorithms	2
1.1	Related Work	3
1.2	Research Question	3
2	Setup: Online Decision Making Problem	4
2.1	Generative model	4
2.2	Training distributions	4
2.3	Test regimes	5
2.4	Oracle: an Approximate Dynamic Program (ADP)	6
2.5	Labeling	6
3	Transformer	7
3.1	Architecture	7
3.2	Objective	7
4	Algorithm Baselines	8
5	Mechanistic Hypothesis: Proving Expressivity	9
6	Experimental Results	12
6.1	Static Variance	12
6.2	Random Variance	14
6.3	OOD Generalization	18
6.4	Temporal Shift	20
7	Mechanistic Analysis: Hypothesis Testing and More	27
7.1	Probe Protocol	27
7.2	Base Models and Controls	27
7.3	Probing Setup	28
7.4	Probe Types	28

7.5	Probe Loss	29
7.6	Probe Targets	29
7.7	Paired Ensemble	29
7.8	Statistical Tests	30
7.8.1	Effect size	31
7.8.2	Sign test	31
7.9	Probing Experiment Results	31
A	Probing Controls: Literature Review	36
B	Probe Targets	37
C	Secondary Probing Targets	38
C.1	Horizon Offset η_t	38
C.2	Exponentiated Log- σ Estimate	40
D	Baseline Details	43
E	Mechanistic Hypothesis Proofs	44
E.1	Coordinates	44
E.2	Main Proofs	44
F	Training	50
F.1	Base-Model Hyperparameters	50
F.2	Probe Hyperparameters	53
G	Approximate Dynamic Program Oracle	53
G.1	Static Variance	54
G.2	Random Variance	55
G.3	Convergence Gate	58
G.4	Static-ADP Convergence Check	59
G.5	Random-ADP Convergence Check	59
H	Agreement and Payoff	60
I	Alternative Supervision	60
J	Temporal Shift	62

1 Introduction: In-Context Learning and Online Algorithms

In-context learning is the phenomenon by which a sequence model, at test time, executes a procedure inferred from the data it sees in context — potentially only noisy input-output pairs — without any weight updates. Prior work has demonstrated transformers’ ability to learn simple function classes, such as linear and Boolean functions, in context. The analogous question for online algorithms, however, has received less attention. This is a natural direction: online algorithms make decisions based on the sequence observed so far, while transformers are causal sequence models whose predictions are computed from the preceding context. Can transformers learn this kind

of instance adaptability — recovering and running the right algorithmic strategy for each evolving sequence — or do they collapse onto a single fixed procedure tuned to the average training distribution?

1.1 Related Work

A growing literature studies in-context learning for familiar *learning* rules, including least squares, ridge regression, Lasso, generalized linear models, and gradient descent [? ? ?]. In particular, learning to do linear regression — that is, learning to learn real-valued linear functions in context — has attracted substantial attention, and recent work has begun to characterize, either mechanistically or by explicit construction, *how* transformers are able to perform such computations [? ?]. Recent work has also studied the in-context learning ability of transformers and LLMs through the problem of learning to learn certain Boolean function families [?]. See [?] for a survey of the capabilities and challenges of in-context learning. This line of work gives an appealing meta-learning perspective: each prompt instantiates a learning task, and the model is trained to implement a learning rule in its forward pass. Much less is known, however, about whether transformers can learn to learn and execute *classical online algorithms* specifically. The behavior of such algorithms can in principle be represented as Boolean functions, given a sufficiently rich encoding of the input history, but prior work on Boolean function families [?] has not considered such online algorithms themselves as the target in-context learning task.

1.2 Research Question

Optimal stopping is a canonical problem in operations research and computer science, and one of the simplest models for online decision making. We use it as a deliberately simple toy model whose learned computations can be inspected mechanistically in depth. A decision maker observes samples X_1 to X_n sequentially and must irrevocably accept the current item or continue. In Bayesian stopping, the optimal rule compares X_t to the *continuation value* C_t — the threshold the policy should use at step t — is the Bayesian value of declining X_t and continuing optimally, i.e. the expected best future reward under the posterior predictive given the prefix,

$$C_t = \mathbb{E}[\max\{X_{t+1}, C_{t+1}\} \mid X_{1:t}],$$

so the Bayes-optimal policy accepts iff $X_t \geq C_t$. Conditioning on $X_{1:t}$ updates the posterior over future samples, so computing C_t is itself the in-context inference problem.

Motivated by this connection, we introduce a model of in-context learning tailored to online algorithms, using a Bayesian stopping problem with a Normal–Normal conjugate prior as our testbed. We call this setting *online selection of online algorithms*: at test time, the transformer sees a sequence with unknown noise level and must recover, from the prefix alone, the corresponding stopping rule. This creates a useful tension between offline learning and online adaptation: the model parameters are learned offline across many training sequences, but $X_1, \dots, X_t$ are online samples from the current instance, arriving before the decision at time t .

Concretely, at position t , the causal decoder receives only the raw prefix $X_1, \dots, X_t$. Its supervised output is one of

$$\hat{C}_t(X_1, \dots, X_t; \theta) \in \mathbb{R}, \quad \hat{a}_t(X_1, \dots, X_t; \theta) \in (0, 1).$$

From now on, we write $X_{1:t}$ for $X_1, \dots, X_t$. Training is still ordinary supervised prediction:

$$\sum_{t < n} \frac{(\hat{C}_t - y_t^{\text{cv}})^2}{\sigma_i^2} \quad \text{or} \quad \sum_{t < n} \text{BCE}(y_t^{\text{act}}, \hat{a}_t),$$

with exact averaged losses in Section 3.2. The difference from much of the in-context-learning literature is that the context contains no labeled examples (x_j, y_j) : oracle values or actions appear only as training targets, never as input tokens. Thus test-time adaptation must be inferred from the observation prefix alone.

By varying whether the noise level is fixed or random across training distributions, we engineer a setting that discriminates between two hypotheses about the trained model:

- **Learning-algorithm hypothesis.** The transformer infers the noise level in context and applies the corresponding threshold rule, adapting per instance and generalizing across noise levels.
- **Shortcut hypothesis.** The transformer applies a single training-distribution-tuned policy that ignores the noise level, matching the oracle in distribution but degrading sharply elsewhere.

This paper presents empirical evidence on which hypothesis holds under each training distribution, using competitive-ratio, threshold-trajectory, and action-agreement diagnostics across in-distribution, OOD (out-of-distribution), and per-sequence regime-shift tests. Section 5 states our mechanistic hypothesis and some expressivity results as theoretical justification. Section 7 then adds a statistically grounded mechanistic protocol: across matched ensembles of frozen decoders, we probe hidden states for the prefix statistics, noise-level estimates, and continuation values that the algorithmic hypothesis uses, comparing true-label models against upstream controls on the same downstream tasks. Appendix A situates this choice relative to existing probing controls.

2 Setup: Online Decision Making Problem

2.1 Generative model

For each sample $i \in \{1, \dots, N\}$, draw a within-sequence noise scale σ_i from a prior, draw a latent mean $\mu_i \sim \mathcal{N}(\mu_0, \tau_0^2)$, then draw a sequence

$$X_1, X_2, \dots, X_n \mid \mu_i, \sigma_i \stackrel{\text{i.i.d.}}{\sim} \mathcal{N}(\mu_i, \sigma_i^2).$$

We fix $\mu_0 = 0$, $\tau_0 = 10$, horizon $n = 256$ throughout. The agent observes X_t at each $t \in \{1, \dots, n\}$ and chooses to *stop* (collect X_t) or *continue*; at $t = n$ it must stop. Both μ_i and (in the unknown- σ setting) σ_i are latent.

2.2 Training distributions

We have five training distributions, three with σ fixed (*static-variance*) and two with σ random (*random-variance*):

- $\mathcal{D}_1$: static, $\sigma = 1$ (data-dominant).
- $\mathcal{D}_2$: static, $\sigma = 10$ (balanced).
- $\mathcal{D}_3$: static, $\sigma = 100$ (prior-dominant).
- $\mathcal{D}_{\text{disc}}$: random, $\sigma \sim \text{Uniform}\{1, 10, 100\}$, each with probability $1/3$.
- $\mathcal{D}_{\text{logu}}$: random, $\log_{10} \sigma \sim \text{Uniform}(0, 2)$, equivalently $p(\sigma) = 1/(2\sigma \ln 10)$ on $[1, 100]$.

The discrete random-variance prior gives equal mass to the three representative scales 1, 10, 100. The continuous prior is uniform in $\log_{10} \sigma$, so equal-width intervals on the log scale, such as $[1, 10]$ and $[10, 100]$, have equal probability.

Data- vs. prior-dominant regimes. The labels (data-dominant, balanced, prior-dominant) refer to how much the conjugate posterior mean weights the sample mean $\bar{X}_t$ versus the prior mean μ_0 : at $t = 1$ the data weight is

$$\frac{\tau_0^2}{(\tau_0^2 + \sigma^2)} = \frac{1}{1 + \rho} \text{ with } \rho := \frac{\sigma^2}{\tau_0^2}.$$

With $\tau_0 = 10$ this gives data weight 0.99 at $\sigma = 1$ (data-dominant), 0.50 at $\sigma = 10$ (balanced), and 0.01 at $\sigma = 100$ (prior-dominant). Section 2.3 tabulates the per-regime values; Appendix G.1 contains the derivation.

Supervision schemes (continuation-value / action). Each training distribution is paired with one of two supervision schemes, abbreviated as the suffixes `_cv` and `_act` on model names:

- *continuation-value* (`_cv`): MSE regression against the numerical approximate dynamic program (ADP) continuation value $\hat{C}_t^*$ at every decision time (real-valued target).
- *action* (`_act`): binary cross-entropy against the oracle action $\mathbf{1}[X_t \geq \hat{C}_t^*]$ (binary target).

Five distributions $\times$ two supervisions gives ten trained models. Full loss formulas are in Section 3.2.

The static-variance distributions $\mathcal{D}_1, \mathcal{D}_2, \mathcal{D}_3$ play a double role: training distributions for the single-regime models, and test regimes for all evaluation (Section 2.3). We name single-regime models by their training data (D1_cv, D1_act, etc.) when we need to disambiguate.

2.3 Test regimes

Evaluation uses seven static- σ test regimes: Three coincide with the static-variance training distributions $\mathcal{D}_1, \mathcal{D}_2, \mathcal{D}_3$ at $\sigma \in \{1, 10, 100\}$ (in-prior for both random-variance training distributions), and four additional regimes at $\sigma \in \{0.3, 3, 30, 300\}$ are out-of-distribution for every training prior we use — $\sigma = 3$ and $\sigma = 30$ interpolate between the discrete training values $\{1, 10, 100\}$ (in-prior support of $\mathcal{D}_{\log u}$ but not $\mathcal{D}_{\text{disc}}$); $\sigma = 0.3$ and $\sigma = 300$ extrapolate beyond the support of both random-variance priors. Each regime corresponds to a noise-to-prior-precision ratio $\rho := \sigma^2 / \tau_0^2$ with the data weight at $t = 1$ given by $1 / (1 + \rho)$:

test regime	σ	σ^2	ρ	data weight at $t = 1$	character
$\mathcal{D}_{\text{ood},0.3}$ (extrapolation, low)	0.3	0.09	0.0009	0.999	strongly data-dominant
$\mathcal{D}_1$ (data-dominant)	1	1	0.01	0.99	plug-in is asymptotically Bayes
$\mathcal{D}_{\text{ood},3}$ (interpolation)	3	9	0.09	0.92	between $\mathcal{D}_{\text{disc}}$ training values
$\mathcal{D}_2$ (balanced)	10	100	1	0.50	no closed-form is asymptotic
$\mathcal{D}_{\text{ood},30}$ (interpolation)	30	900	9	0.10	between $\mathcal{D}_{\text{disc}}$ training values
$\mathcal{D}_3$ (prior-dominant)	100	10000	100	0.01	prior-only is asymptotically Bayes
$\mathcal{D}_{\text{ood},300}$ (extrapolation, high)	300	90000	900	0.0011	strongly prior-dominant

Datasets. Training sequences are sampled fresh from the training distribution at each step (no fixed training set); the meaningful budget is gradient steps $\times$ batch size, specified in Appendix F. Validation and test sets are pre-generated and held fixed at $N_{\text{val}} = N_{\text{test}} = 10^4$ sequences each (validation drawn from the training distribution, used for checkpoint selection only; test drawn per regime, used for all evaluation). Validation and test sets use disjoint random seeds. The OOD test caches use disjoint seeds and apply the same generative model with $\mu_i \sim \mathcal{N}(\mu_0, \tau_0^2)$ throughout.

2.4 Oracle: an Approximate Dynamic Program (ADP)

Each training distribution induces a Bayes-optimal stopping policy that we use as the labeling target during training. The continuation value C_t has no closed form, so we approximate it by backward dynamic programming on a discretized belief state and store $\hat{C}_t$ as a per-stage threshold table. In the empirical sections, $\hat{C}_t^*$ denotes this numerical ADP target after the convergence checks in Appendix G.3. Static- σ oracle tables also define the per-regime Bayes-optimal payoff R^* used to scale evaluation results. Algorithms, grid sizing, convergence checks, and the marginal-likelihood derivation that the random-variance case relies on are deferred to Appendix G.

Static-variance setting. When σ is fixed and known, only μ is latent; the Normal–Normal posterior depends on $X_{1:t}$ only through the running sum S_t , equivalently the sample mean $\bar{X}_t = S_t/t$, and is summarized by a one-dimensional belief — the conjugate posterior mean μ_t . We discretize μ_t on a per-stage uniform grid scaled to its marginal standard deviation across sequences and run backward induction with Gauss–Hermite quadrature for the predictive expectation (Appendix G.1).

Random-variance setting. When both μ and σ are latent, the joint sufficient statistic for the posterior $p(\mu, \sigma \mid X_{1:t})$ is the pair (S_t, Q_t) with $S_t := \sum_{s \leq t} X_s$ and $Q_t := \sum_{s \leq t} X_s^2$. The marginal log-likelihood $\log p(X_{1:t} \mid \sigma)$ has a closed form in (S_t, Q_t) (equation (1), derived in Appendix G.2). The numerical ADP table is indexed by transformed coordinates $(\bar{X}_t, \lambda_t)$, where λ_t is a projected log-scale version of the running standard-deviation estimate. This log coordinate is a table-lookup convenience; the plug-in hypothesis instead focuses on the directly interpretable prefix scale estimate $\sqrt{\hat{\sigma}_t^2}$ and the horizon multiplier η_t . We integrate the mixture predictive over σ (a 3-point sum for $\mathcal{D}_{\text{disc}}$, a Gauss–Legendre quadrature on $\log \sigma$ for $\mathcal{D}_{\text{logu}}$) before applying Gauss–Hermite for the per-component expectation.

$$\log p(X_{1:t} \mid \sigma) = \text{const}(t) - \frac{t}{2} \log \sigma^2 - \frac{1}{2} \log \left(1 + \frac{t\tau_0^2}{\sigma^2} \right) - \frac{1}{2} \left[\frac{Q_t}{\sigma^2} - \frac{\tau_0^2 S_t^2 / \sigma^4}{1 + t\tau_0^2 / \sigma^2} \right]. \quad (1)$$

2.5 Labeling

For each training distribution we precompute the corresponding ADP threshold table $\{\hat{C}_t\}$ once (Appendix G.1 for $\mathcal{D}_1, \mathcal{D}_2, \mathcal{D}_3$; Appendix G.2 for $\mathcal{D}_{\text{disc}}, \mathcal{D}_{\text{logu}}$). Both tables expose the same query: given the belief state at time t , return $\hat{C}_t$.

For each sequence $X_{1:n}$ from training distribution $\mathcal{D}$, at each $t \in \{1, \dots, n-1\}$ we update the belief state and query the table for $\hat{C}_t$, then emit

$$y_t^{\text{cv}} := \hat{C}_t, \quad y_t^{\text{act}} := \mathbf{1}[X_t \geq \hat{C}_t].$$

At $t = n$, $y_n^{\text{act}} = 1$ deterministically. The continuation-value target at $t = n$ is set to 0 but is masked out of the training loss.

Offline-optimum labels. As an alternative to the ADP-oracle labels, we also consider *offline-optimum* labels — $y_t^{\text{cv}} := \max_{s > t} X_s$ and $y_t^{\text{act}} := \mathbf{1}[X_t \geq \max_{s > t} X_s]$ — which inspect the realised suffix rather than its expectation under optimal online play. They are computed in $O(n)$ per sequence by a single reverse-cummax pass, with no ADP table. We use these labels only as a sensitivity check in Appendix I: they are close to the ADP-label results for the log-uniform prior, but differ on the discrete prior’s $\sigma = 3$ interpolation regime.

3 Transformer

3.1 Architecture

The trained model, GPTSTOPPER, is a scale-blind GPT-2-style causal decoder. It receives raw scalar observations plus learned absolute position embeddings, has no σ -dependent input or output scaling, and uses separate scalar heads for continuation value and action logits. The exact hyperparameters are listed in Appendix F, Table 2.

3.2 Objective

We train ten models: the five training distributions $\mathcal{D}_1$, $\mathcal{D}_2$, $\mathcal{D}_3$, $\mathcal{D}_{\text{disc}}$, and $\mathcal{D}_{\text{logu}}$, each under two supervisions (continuation-value and action). The model emits one prediction per timestep t via the causal mask. We supervise at every $t \in \{1, \dots, n-1\}$ (the terminal $t = n$ has no decision, since the agent must accept X_n): each timestep contributes its own oracle label and its own gradient signal, giving $n-1 = 255$ training signals per sequence. The per-sequence losses, with sequence-specific σ_i used only for continuation-value loss normalization (the model never sees σ_i), are

$$\ell_{\text{cv}}(\theta; X_{1:n}, \sigma_i) = \frac{1}{(n-1)\sigma_i^2} \sum_{t=1}^{n-1} \left(\hat{C}_t(X_{1:t}; \theta) - y_t^{\text{cv}} \right)^2,$$

$$\ell_{\text{act}}(\theta; X_{1:n}) = -\frac{1}{n-1} \sum_{t=1}^{n-1} \left[y_t^{\text{act}} \log \hat{a}_t + (1 - y_t^{\text{act}}) \log(1 - \hat{a}_t) \right].$$

Each is the within-sequence mean over per-step terms. The mini-batch loss is the further mean over $B = 64$ sequences sampled fresh from the training distribution.

The $1/\sigma_i^2$ factor normalizes the continuation-value loss so high- σ and low- σ sequences contribute equally: continuation-value errors scale with σ , and the divisor puts every regime on a common loss scale — matching the regime-equal weighting of the headline payoff metric R/R^* . Figure 1 shows this for $\mathcal{D}_{\text{logu-cv}}$: absolute continuation-value error scales with σ (left panel); the normalized error is uniform across the prior (right panel).

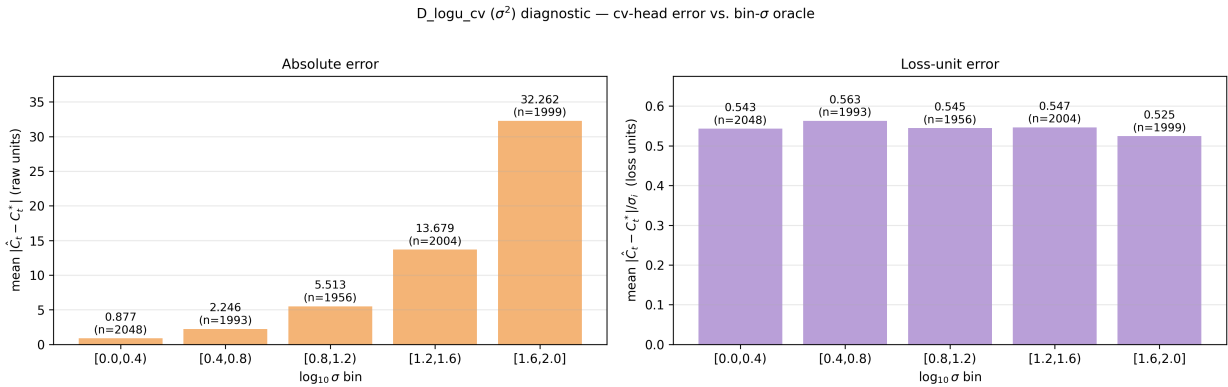


Figure 1: Continuation-value error diagnostic for $\mathcal{D}_{\text{logu-cv}}$. Left: mean $|\hat{C}_t - \hat{C}_t^*|$ per $\log_{10} \sigma$ bin (raw units); the absolute error scales with σ . Right: the same error in loss units ($|\hat{C}_t - \hat{C}_t^*|/\sigma_i$); after the $1/\sigma_i^2$ normalization the error is balanced across all five bins.

Optimization details, training budgets, and training curves are reported in Appendix F.

4 Algorithm Baselines

We compare the trained models against five families of reference policies. Several use η_t , the standardized continuation offset for the known- μ , known- σ Gaussian stopping problem: if $c_t = \mu + \sigma\eta_t$, then

$$\eta_{n-1} = 0, \quad \eta_t = \psi(\eta_{t+1}), \quad \psi(z) = z\Phi(z) + \varphi(z),$$

where Φ and φ are the standard-normal CDF and PDF. Appendix D derives this recursion.

Known- σ threshold substitutions. The plug-in, prior-only, and myopic baselines all know the test-regime scale σ and use a threshold of the form

$$\theta_t = m_t + \delta \sigma \eta_t, \quad \pi_t(X_{1:t}) = \mathbf{1}[X_t \geq \theta_t].$$

They differ only in the location estimate m_t and whether they include the continuation offset:

rule	m_t	δ
plug-in	$\bar{X}_t$	1
prior-only	μ_0	1
myopic	μ_t	0.

Thus plug-in trusts the sample mean, prior-only ignores the prefix except for the current offer, and myopic uses the conjugate posterior mean but ignores future option value.

Unknown- σ plug-ins. For $t \geq 2$, define the running MLE-scale estimate used in the plug-in baseline by

$$\hat{\sigma}_t^{\text{MLE}} := \sqrt{\hat{\sigma}_t^2}, \quad \hat{\sigma}_t^2 := \frac{1}{t-1} \sum_{s=1}^t (X_s - \bar{X}_t)^2.$$

At $t = 1$, before the sample variance is defined, we use a fixed initialization $\hat{\sigma}_1^{\text{MLE}} = \sigma_{\text{init}}$. The divisor $t - 1$ is the Bessel-corrected version of the Gaussian divisor- t MLE; the difference vanishes as t grows, and this is the running scale used in the experiments below. Let $p_{\text{train}}(\sigma)$ denote the training prior mass or density over σ , with support $\mathcal{S}_{\text{train}}$. The MAP-scale estimate is

$$\hat{\sigma}_t^{\text{MAP}} \in \arg \max_{\sigma \in \mathcal{S}_{\text{train}}} \{ \log p(X_{1:t} \mid \sigma) + \log p_{\text{train}}(\sigma) \}.$$

Here $\mathcal{S}_{\text{train}} = \{1, 10, 100\}$ for $\mathcal{D}_{\text{disc}}$ and $[1, 100]$ for $\mathcal{D}_{\text{logu}}$. The implementability result below uses a finite grid and softmax relaxation of this argmax. The MLE- σ and MAP- σ baselines keep the same plug-in threshold $\bar{X}_t + \hat{\sigma}_t \eta_t$, but differ in how they estimate σ :

- *Prior-free* Maximum Likelihood Estimation (MLE) : Use $\hat{\sigma}_t^{\text{MLE}}$. This depends only on the prefix and not on the training prior.
- *Prior-dependent* Maximum A Posteriori (MAP): Use $\hat{\sigma}_t^{\text{MAP}}$, with the closed-form marginal likelihood (1) supplying $\log p(X_{1:t} \mid \sigma)$.

Dynamic-programming oracles. The per-regime oracle $\pi_{\mathcal{D}_i}^*$ knows the test-regime σ_i and is Bayes-optimal under $(\mu_0, \tau_0^2, \sigma_i)$; its expected payoff defines R^* . The random-variance ADP oracles instead use the full training prior over σ , with one oracle for $\mathcal{D}_{\text{disc}}$ and one for $\mathcal{D}_{\text{logu}}$.

Scale-free order baseline. The secretary rule skips the first $\lfloor n/e \rfloor$ observations and then accepts the first value above their maximum.

Offline optimum. The offline optimum reports $\mathbb{E}[\max_t X_t]$. It is not implementable online, but upper-bounds every online policy’s payoff. Appendix D derives the η_t recursion and the offline-optimum integral.

5 Mechanistic Hypothesis: Proving Expressivity

Our mechanistic hypothesis is that the trained decoder may implement a simpler online rule than the ADP oracle used for labeling: a plug-in threshold computed from prefix statistics and a scale estimate. This section is therefore an expressivity check for that hypothesis. We say that a causal decoder can implement a threshold rule if there is a fixed parameter setting whose forward pass maps every prefix $X_{1:t}$ to the rule’s threshold, or equivalently to the action margin $X_t - \theta_t$, at position t . This is an expressivity result, not a claim that gradient descent finds these weights.

The construction shows that the transformer interface can represent thresholds of the form

$$\theta_t(\hat{\sigma}_t) = \bar{X}_t + \hat{\sigma}_t \eta_t, \quad \pi_t(X_{1:t}) = \mathbf{1}[X_t \geq \theta_t(\hat{\sigma}_t)].$$

Here $\hat{\sigma}_t$ is either the prior-free running scale $\sqrt{\hat{\sigma}_t^2}$ or a prior-dependent MAP scale. For a fixed scale grid $\Sigma_K = \{\sigma_1, \dots, \sigma_K\}$, let $w = (w_1, \dots, w_K)$ be the prior mass or quadrature weight assigned to the grid points, and define

$$\ell_{t,k} := \log w_k + \log p(X_{1:t} \mid \sigma_k).$$

Define the softmax weights and finite- β soft grid-MAP scale by

$$\alpha_{t,k}^{(\beta)} := \frac{\exp(\beta \ell_{t,k})}{\sum_{r=1}^K \exp(\beta \ell_{t,r})}, \quad \hat{\sigma}_{t,\beta,K}^{\text{MAP}} := \sum_{k=1}^K \sigma_k \alpha_{t,k}^{(\beta)}.$$

For readability, the construction is written for a pre-norm causal decoder with ReLU feed-forward blocks. This keeps the algebra exact; the trained GPTSTOPPER checkpoints use GELU, and Lemma 4 gives the standard rescaling bridge from ReLU rows to GELU rows. At block ℓ , an attention sublayer and a ReLU feed-forward sublayer update the residual stream by

$$\begin{aligned} \tilde{h}_t^{(\ell)} &= h_t^{(\ell)} + \sum_a W_O^{(a)} \sum_{s \leq t} \alpha_{ts}^{(a)} W_V^{(a)} \text{LN}(h_s^{(\ell)}), \\ \alpha_{ts}^{(a)} &= \text{softmax}_{s \leq t} \left(\frac{(W_Q^{(a)} \text{LN}(h_t^{(\ell)}))^{\top} (W_K^{(a)} \text{LN}(h_s^{(\ell)}))}{\sqrt{d_{\text{head}}}} \right), \\ h_t^{(\ell+1)} &= \tilde{h}_t^{(\ell)} + W_2 \left[W_1 \text{LN}(\tilde{h}_t^{(\ell)}) + b_1 \right]_+ + b_2. \end{aligned}$$

In Figure 2, $r_c^{\top}$ denotes the readout row for coordinate c , so $r_c^{\top} \text{LN}(h_t)$ reads the quantity stored in slot c . The displayed matrices specify one possible weight setting for the hypothesized plug-in computation.

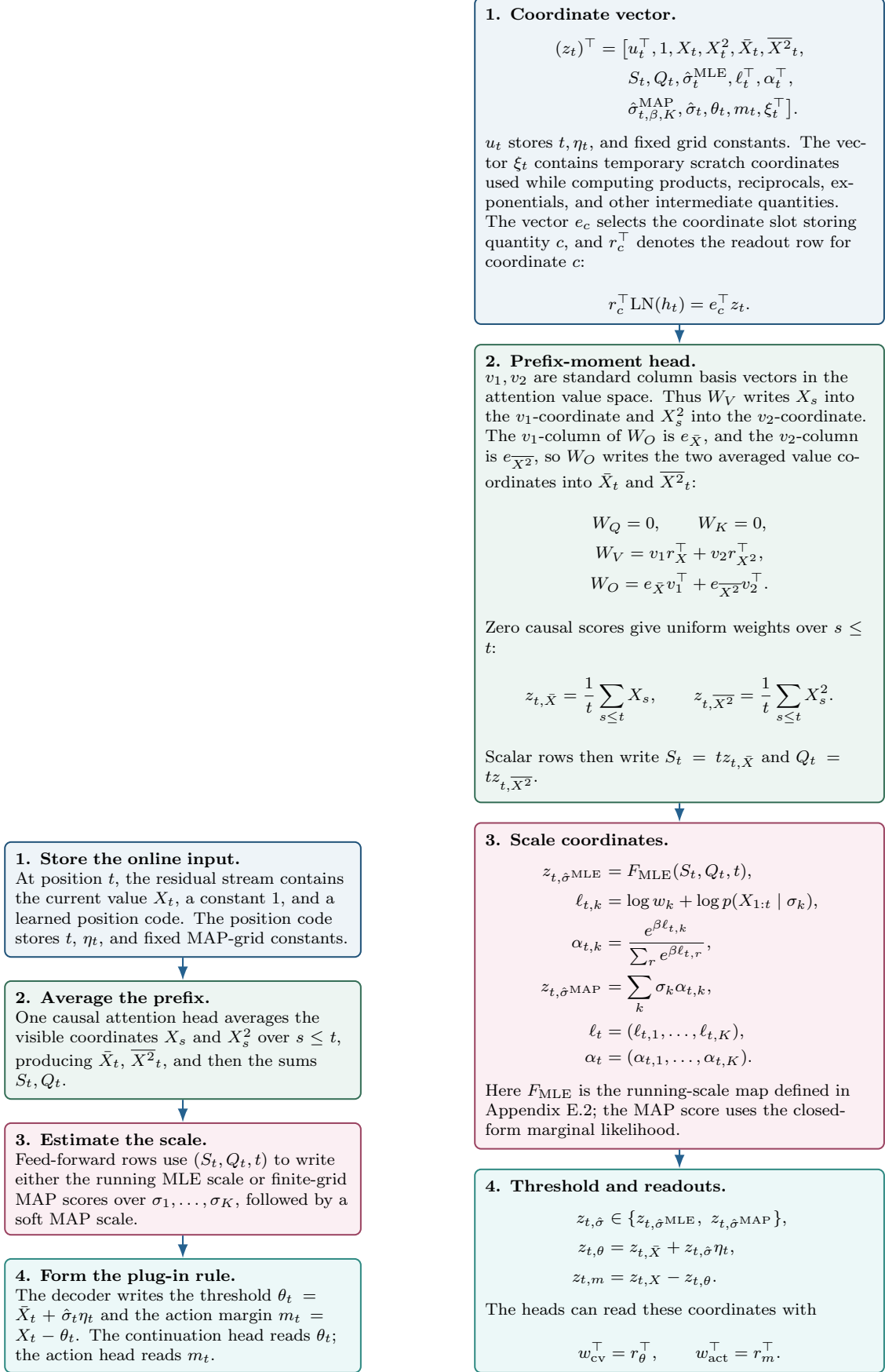


Figure 2: Hypothesized plug-in mechanism in two views

Lemma 1 (Finite-grid MAP criterion). *On the fixed scale grid Σ_K , the posterior-maximizing scale is*

$$\hat{\sigma}_{t,K}^{\text{MAP}} := \sigma_{k_t^*}, \quad k_t^* \in \arg \max_{1 \leq k \leq K} \ell_{t,k}.$$

Lemma 2 (Soft grid-MAP convergence). *If the hard grid-MAP maximizer $k_t^* \in \arg \max_k \ell_{t,k}$ is unique, then*

$$\alpha_{t,k_t^*}^{(\beta)} \rightarrow 1, \quad \alpha_{t,r}^{(\beta)} \rightarrow 0 \quad (r \neq k_t^*), \quad \hat{\sigma}_{t,\beta,K}^{\text{MAP}} \rightarrow \hat{\sigma}_{t,K}^{\text{MAP}} := \sigma_{k_t^*}$$

as $\beta \rightarrow \infty$. Moreover, if

$$\gamma_t := \ell_{t,k_t^*} - \max_{r \neq k_t^*} \ell_{t,r} > 0,$$

then

$$|\hat{\sigma}_{t,\beta,K}^{\text{MAP}} - \hat{\sigma}_{t,K}^{\text{MAP}}| \leq (\sigma_{\max} - \sigma_{\min})(K-1)e^{-\beta\gamma_t}.$$

Appendix E.2 proves these two finite-grid MAP facts and the implementability theorem below.

Theorem 1 (Causal implementation of plug-in thresholds). *Fix the horizon n , a clipped coordinate domain, and any fixed Σ_K, w, β . For every $\epsilon > 0$, there is a sufficiently wide pre-norm causal decoder with ReLU feed-forward blocks whose online threshold is uniformly within ϵ of each plug-in threshold*

$$\theta_t = \bar{X}_t + \hat{\sigma}_t \eta_t$$

with $\hat{\sigma}_t$ equal to the MLE scale $\hat{\sigma}_t^{\text{MLE}}$ or the smoothed finite-grid MAP scale $\hat{\sigma}_{t,\beta,K}^{\text{MAP}}$. The action matches the corresponding rule whenever $|X_t - \theta_t| > \epsilon$.

For the displayed parameterization, u_t has $2K + 4$ coordinates. If $d_\xi := \dim(\xi_t)$ scratch coordinates are retained, the persistent logical state has

$$d_z = 4K + 16 + d_\xi$$

coordinates. The balanced LayerNorm code in Lemma 3 then uses physical residual width

$$d \geq 2d_z + 2.$$

The plug-in computation is shallow in principle. Once attention has written the prefix moments, such as $\bar{X}_t$ and $\bar{X}_t^2$, a sufficiently wide feed-forward layer can approximate the remaining scalar map from these moments to the threshold. The MAP version is more expensive because it must score each candidate scale $\sigma_1, \dots, \sigma_K$, form softmax weights over these scores, and average the candidate scales. The width scales with both the interpolation resolution J and the number of scale candidates K . A one-dimensional scalar approximation with J intervals uses $J + 1$ hidden units. Applying such approximations across K candidate scales costs on the order of $K(J + 1)$ hidden units. Therefore, a finer MAP grid gives a more accurate scale estimate but requires a wider feed-forward block.

Approximation enters through finite scalar interpolation and the MAP relaxation from a continuous search to a finite grid and then to the temperature- β soft scale. For the trained GELU architecture, Lemma 4 adds an arbitrarily small transfer error. Thus the theorem is compatible with the trained model’s eight-block depth as a serial computation pattern, but it is not a parameter-count claim for the finite GPTSTOPPER checkpoints with $d_{\text{emb}} = 128$ and MLP width 512.

6 Experimental Results

All ten trained models are evaluated on each test regime. We report three metrics per (model, regime) cell: expected payoff $R(\pi; \mathcal{D}_i) := \mathbb{E}[X_{\tau_\pi}]$, competitive ratio $R/R_{\mathcal{D}_i}^*$ relative to the per-regime oracle (the regime-invariant headline), and per-step action agreement against each baseline. Threshold trajectories (mean ± 1 std over $N_{\text{test}} = 10^4$ test sequences) are reported per continuation-value-supervised model and visualize whether the trained policy adapts to the test regime within the prefix.

6.1 Static Variance

Each single-regime model matches the per-regime oracle in-distribution and degrades sharply off-diagonally. The three columns labelled $\mathcal{D}_{1\text{-cv}}$, $\mathcal{D}_{2\text{-cv}}$, $\mathcal{D}_{3\text{-cv}}$ in the heatmap of Figure 7 (shown in Section 6.2) show this directly: each has a single saturated diagonal cell and pale off-diagonals, with the worst falling to 0.016 ($\mathcal{D}_{3\text{-cv}}$ on $\mathcal{D}_1$). Action counterparts ($\mathcal{D}_{1\text{-act}}$, $\mathcal{D}_{2\text{-act}}$, $\mathcal{D}_{3\text{-act}}$) are within 0.04 of the continuation-value entries cell-for-cell.

Figures 3–5 visualize the same effect at the threshold-trajectory level: each model’s mean threshold sits flat at its training-distribution scale across all three test regimes, with no adaptation observable in the test prefix.

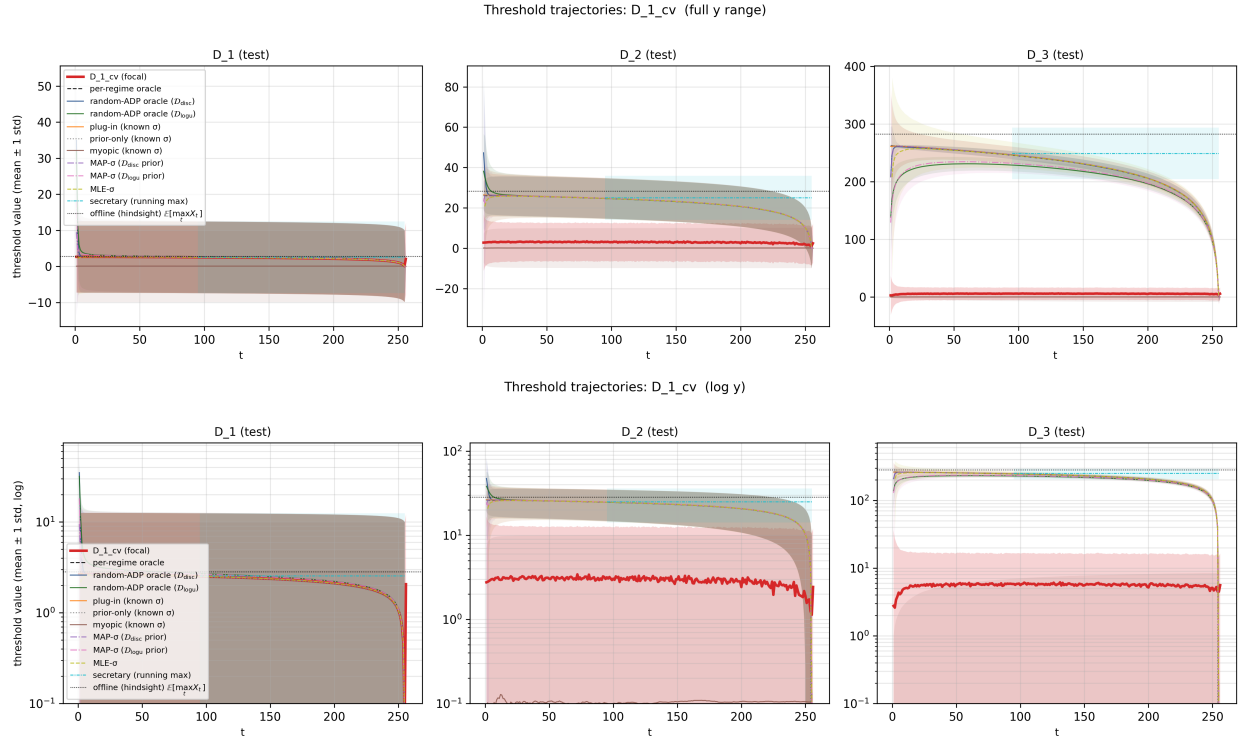


Figure 3: Threshold trajectories for $\mathcal{D}_{1\text{-cv}}$ (focal model, red, mean ± 1 std band over $N_{\text{test}} = 10^4$ sequences) on each of the three test regimes, with all baselines from Section 4 overlaid. The model’s threshold remains at the $\sigma = 1$ scale (≈ 2.5) on every regime. Top: linear y, full range (negative-side ± 1 std bands shown). Bottom: log y, negatives clipped at 0.1.

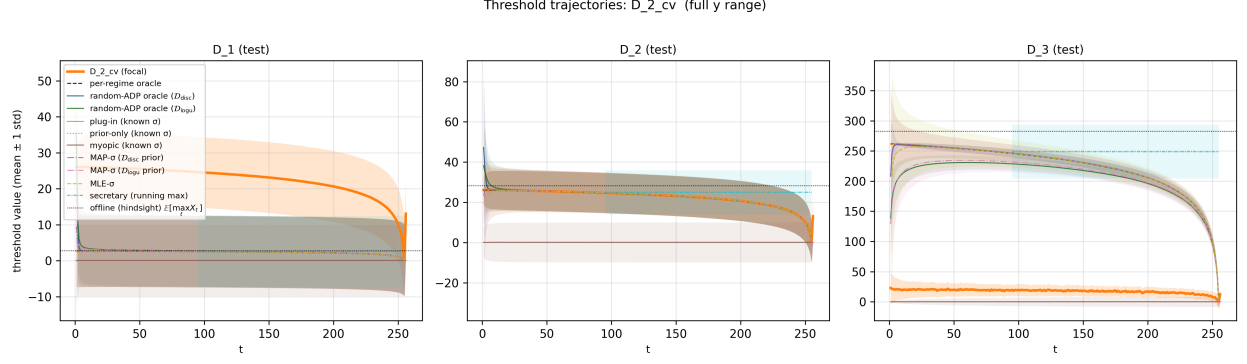


Figure 4: Threshold trajectories for $\mathcal{D}_2\text{-cv}$. The model’s threshold remains at the $\sigma = 10$ scale (≈ 26) on every regime.

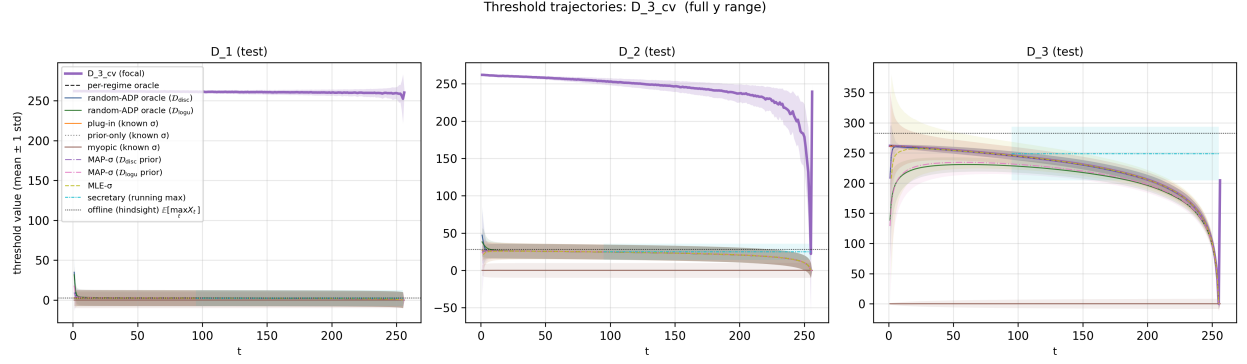


Figure 5: Threshold trajectories for $\mathcal{D}_3\text{-cv}$. The model’s threshold remains at the $\sigma = 100$ scale (≈ 260) on every regime.

Figure 6 overlays all five continuation-value-trained models on each test regime, juxtaposing the three single-regime plateaus against the multi-regime $\mathcal{D}_{\text{disc-cv}}$ adaptation.

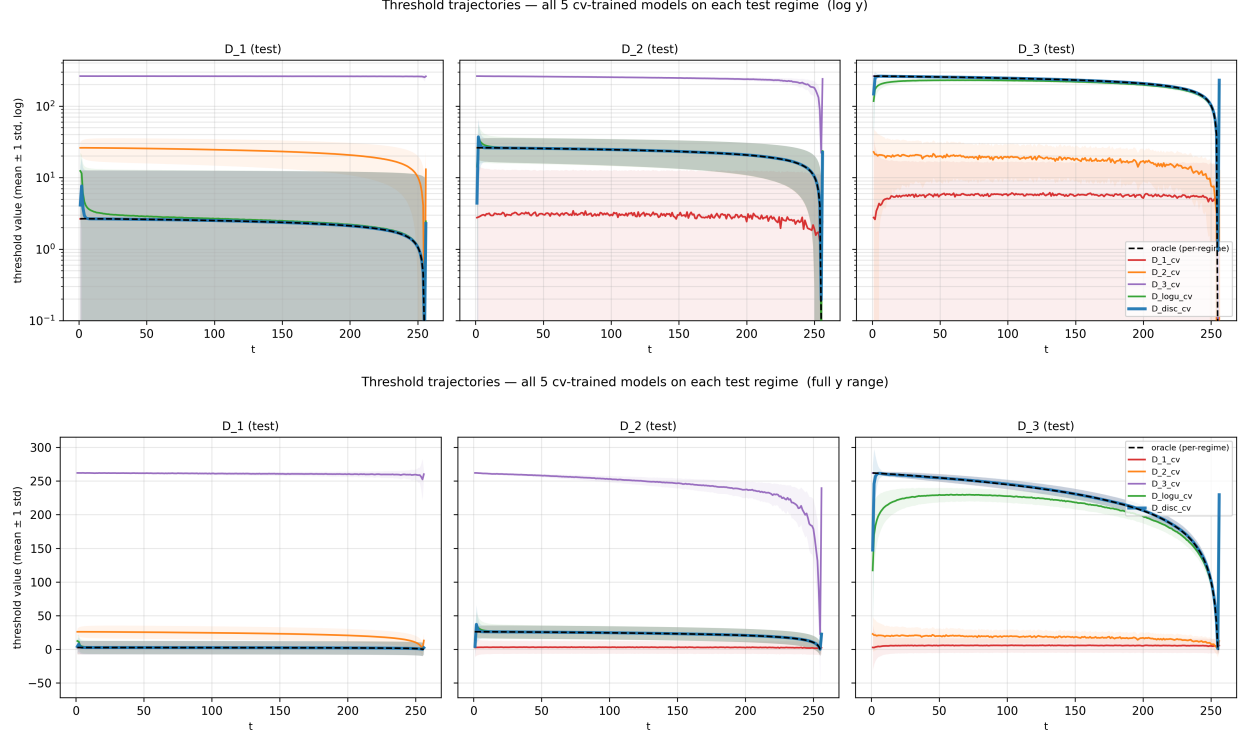


Figure 6: All five continuation-value-trained models overlaid on each test regime, with the per-regime oracle (black dashed) for reference. The three single-regime models ($\mathcal{D}_1$ -cv red, $\mathcal{D}_2$ -cv orange, $\mathcal{D}_3$ -cv purple) plateau at their training-distribution scale on every regime; the multi-regime models $\mathcal{D}_{\text{disc-cv}}$ (blue) and $\mathcal{D}_{\text{logu-cv}}$ (green) both track the per-regime oracle on each panel. Top: log y, negatives clipped. Bottom: linear y, full range (negative-side ± 1 std bands shown).

These single-regime results are consistent with the shortcut hypothesis: each model applies one training-distribution-tuned threshold and ignores the test regime’s noise level. With only one σ in the training data, the optimizer had no incentive to learn a noise-conditional rule; the experiment therefore cannot tell us whether the architecture could express one. The random-variance runs test that next.

6.2 Random Variance

Training on a distribution over σ is the natural test of the learning-algorithm hypothesis: no single threshold is Bayes-optimal across $\sigma \in \{1, 10, 100\}$, so matching the per-regime oracle on every in-prior test regime is evidence that the network is inferring σ in context and applying the corresponding rule. Both $\mathcal{D}_{\text{disc-cv}}$ and $\mathcal{D}_{\text{disc-act}}$ show this pattern (right four columns of Figure 7): $R/R^* = 0.999, 0.999, 1.000$ for continuation-value supervision and $0.999, 0.999, 0.999$ for action supervision on $\mathcal{D}_1, \mathcal{D}_2, \mathcal{D}_3$. $\mathcal{D}_{\text{logu-cv}}$ attains $R/R^* = 0.972, 0.976, 0.973$ and $\mathcal{D}_{\text{logu-act}}$ attains $0.974, 0.976, 0.975$ on the same three regimes; the gap between supervisions is below 0.005 on every regime.

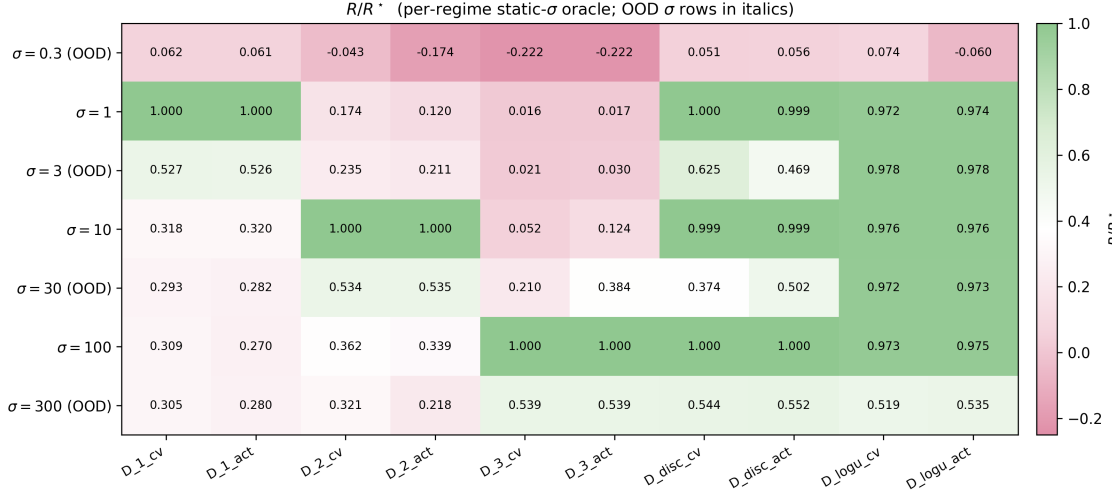


Figure 7: Competitive ratio R/R^* per (test regime, trained model): seven test regimes (rows; OOD in italics) $\times$ ten trained models (columns). Primary results figure for Section 6.

We use R/R^* as the primary metric throughout the report. Per-step action agreement with the per-regime oracle is a tempting alternative, but it is misleading on its own — $\mathcal{D}_{3_cv}$ on $\mathcal{D}_1$ agrees with the oracle on 97.8% of decisions yet attains $R/R^* = 0.016$. We report agreement only as a diagnostic; the heatmap and the full discussion are deferred to Appendix H.

The per- σ payoff breakdown confirms uniform competence across the training prior (Figure 8). For $\mathcal{D}_{disc_cv}$: $R/R^* \in \{1.001, 0.999, 0.999\}$ at $\sigma \in \{1, 10, 100\}$; $\mathcal{D}_{disc_act}$ is within 0.001 of the oracle in every bin. For $\mathcal{D}_{logu_cv}$: $R/R^* \in [0.999, 1.000]$ across all five $\log_{10} \sigma$ bins, indistinguishable from $\mathcal{D}_{logu_act}$.

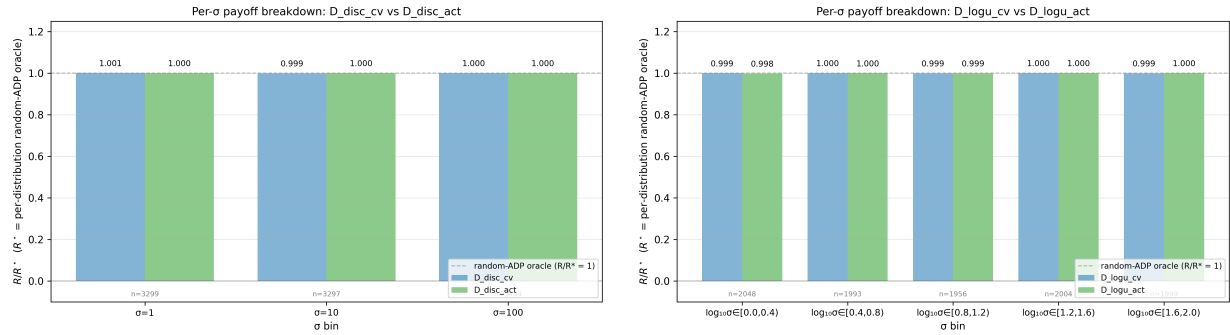


Figure 8: Per- σ payoff breakdown. Left: $\mathcal{D}_{disc_cv}$ (blue) vs. $\mathcal{D}_{disc_act}$ (green). Right: $\mathcal{D}_{logu_cv}$ vs. $\mathcal{D}_{logu_act}$. R^* = per-regime static- σ oracle in both panels; both bracket the dashed $R/R^* = 1$ reference in every σ bin.

The small gap between action and continuation-value supervision in Figure 8 is itself notable. Action supervision is a coarser target: a binary $\{0, 1\}$ label tells the model only whether to stop, not by how much, and carries no direct information about distance to the optimal threshold. The action-supervised models nonetheless match the continuation-value counterparts in payoff, suggesting that the decision boundary may be supported by a continuation-value-like internal representation. Section 7 tests this by probing whether $\hat{C}_t^*$ is linearly decodable from the action-supervised models’

hidden states, even though $\hat{C}_t^*$ never appears in their training loss.

The threshold trajectories (Figures 9–11) show the corresponding signature: on each test regime the model’s threshold moves toward the per-regime oracle within a few observations and tracks it for the remainder of the horizon. This behavior is the expected signature of in-context σ inference followed by a regime-corresponding threshold rule.

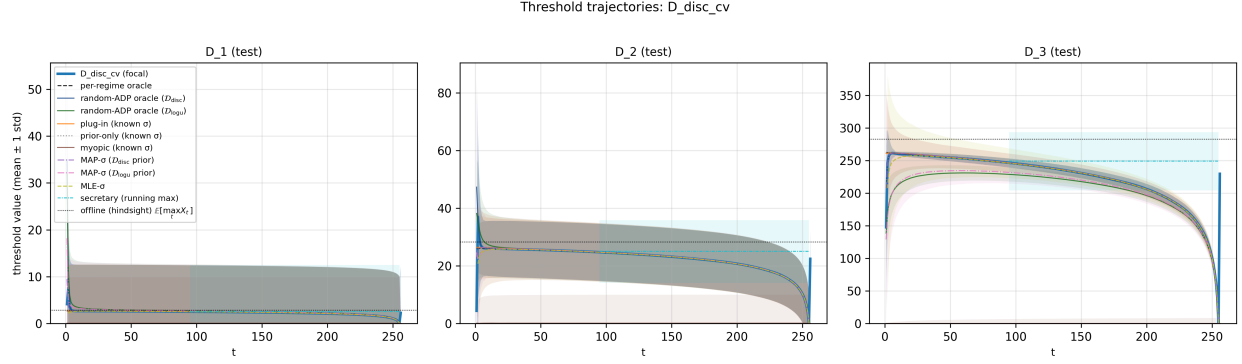


Figure 9: Threshold trajectory for $\mathcal{D}_{\text{disc-cv}}$ (focal, blue, mean ± 1 std band) on each in-distribution test regime, with all baselines from Section 4 overlaid. The model adapts to the test regime within a few observations and tracks the per-regime oracle (black dashed) and the $\mathcal{D}_{\text{disc}}$ random-ADP oracle for the remainder of the horizon on every regime.

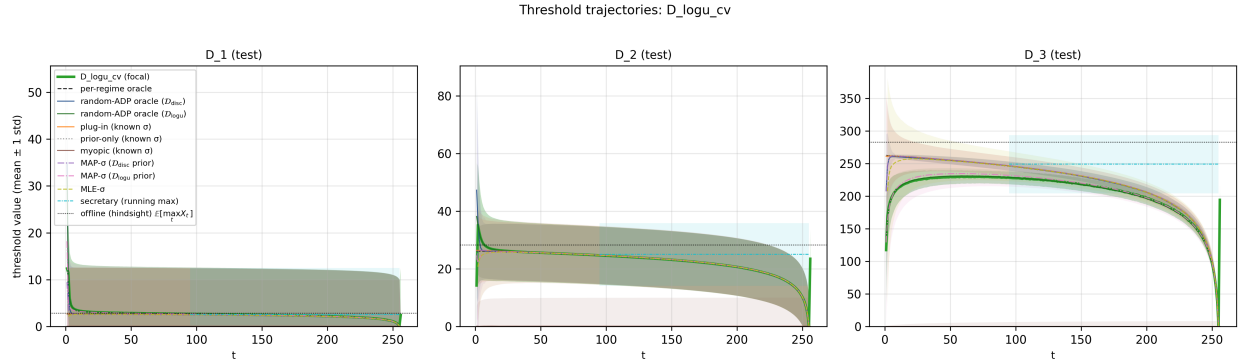


Figure 10: Threshold trajectory for $\mathcal{D}_{\text{logu-cv}}$ on each in-distribution test regime.

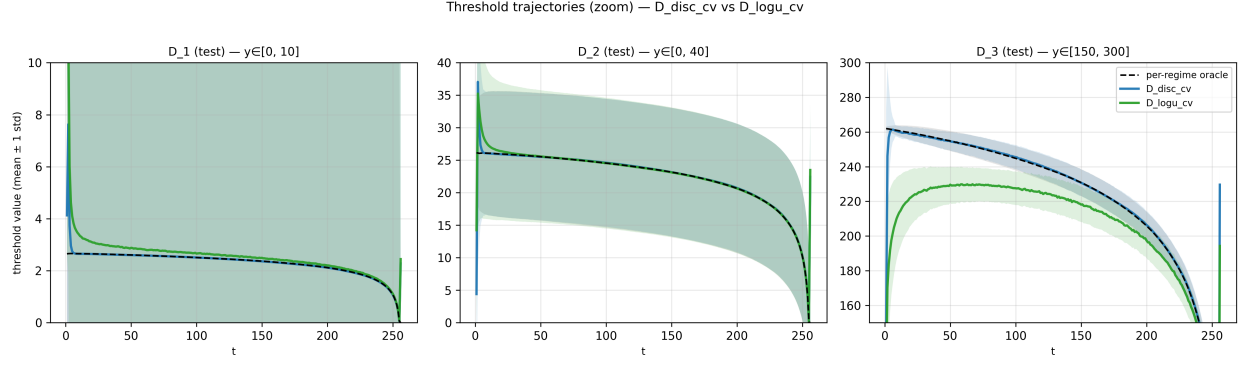


Figure 11: Zoomed comparison of $\mathcal{D}_{\text{disc_cv}}$ and $\mathcal{D}_{\text{logu_cv}}$ against the per-regime oracle (black dashed) on each test regime, with per-panel y -limits: $y \in [0, 10]$ on $\mathcal{D}_1$, $[0, 40]$ on $\mathcal{D}_2$, $[150, 300]$ on $\mathcal{D}_3$.

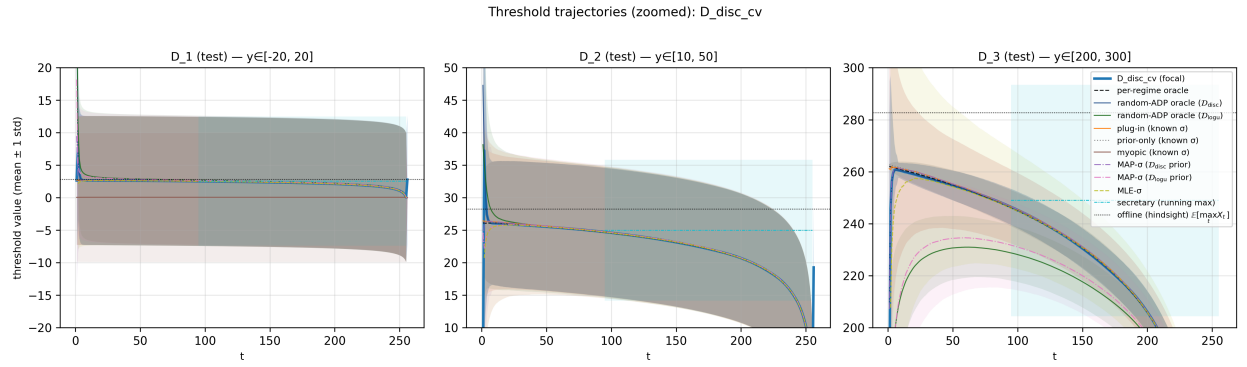


Figure 12: Per-panel zoomed threshold trajectory for $\mathcal{D}_{\text{disc_cv}}$ against every baseline. Each panel uses a regime-specific y -window: $y \in [-20, 20]$ on $\mathcal{D}_1$, $[10, 50]$ on $\mathcal{D}_2$, $[200, 300]$ on $\mathcal{D}_3$.

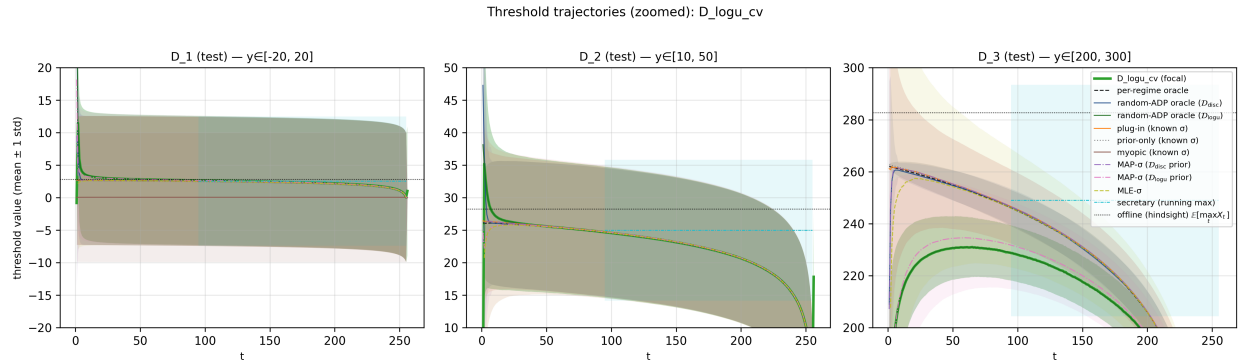


Figure 13: Per-panel zoomed threshold trajectory for $\mathcal{D}_{\text{logu_cv}}$ against every baseline, same per-regime y -windows as Figure 12.

6.3 OOD Generalization

A noise-conditional rule should produce sensible thresholds not only at the training σ values but at novel σ as well. We test this with four OOD test regimes at $\sigma \in \{0.3, 3, 30, 300\}$ — two interpolation points between $\mathcal{D}_{\text{disc}}$ ’s training values $\{1, 10, 100\}$, two extrapolation points outside the support of both random-variance priors. The OOD rows are included in the single report-level heatmap in Figure 7; this subsection unpacks those entries with per- σ curves.

On the two interpolation points, $\mathcal{D}_{\text{disc-cv}}$ attains $R/R^* = 0.625$ at $\sigma = 3$ and 0.374 at $\sigma = 30$, beating both the random-ADP oracle at the matching $\mathcal{D}_{\text{disc}}$ prior (0.244 and 0.229) and the matched MAP- σ plug-in (0.260 and 0.243) by $1.6\times$ to $2.5\times$ in R/R^* (Figure 14). The random-ADP-disc baseline is the Bayes-optimal policy under the discrete training prior, so $\mathcal{D}_{\text{disc-cv}}$ exceeding it on novel σ means the trained model is not limited to the prior’s discrete support. On $\sigma \in \{3, 30\}$, both $\mathcal{D}_{\text{logu-cv}}$ and $\mathcal{D}_{\text{logu-act}}$ match MAP- σ at the $\mathcal{D}_{\text{logu}}$ prior to four decimal places. At extrapolation $\sigma = 300$ every trained model and the matching random-ADP oracle land in the 0.50–0.55 band; at $\sigma = 0.3$ the trained random-variance models have near-zero payoff.

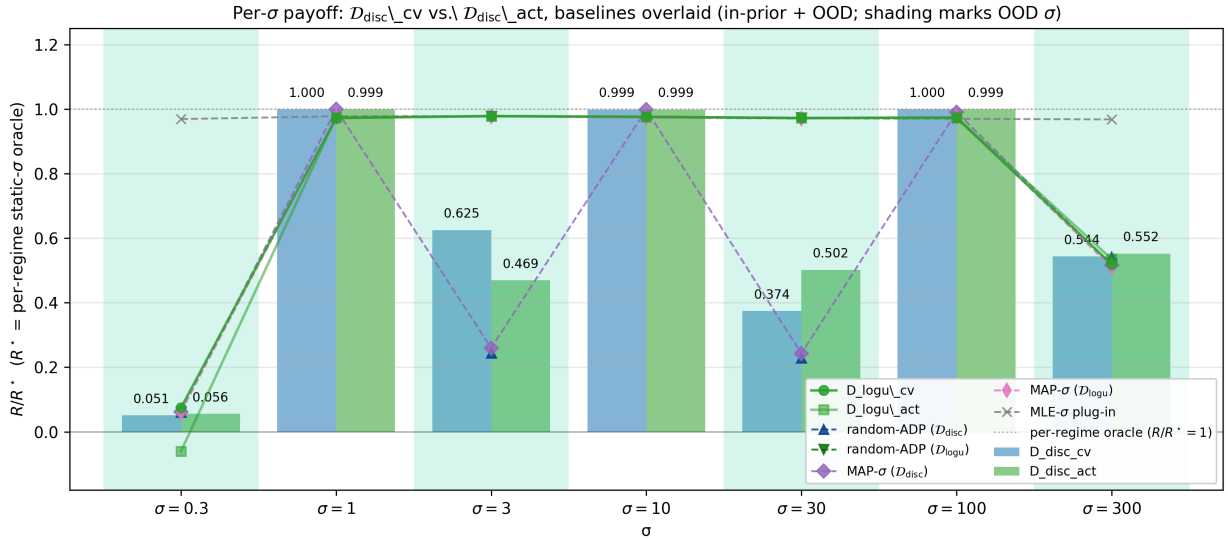


Figure 14: Per- σ payoff for $\mathcal{D}_{\text{disc-cv}}$ vs. $\mathcal{D}_{\text{disc-act}}$ across all seven test σ values, with the five data-only and prior-aware baselines overlaid as dashed lines + markers and the alternative continuous-prior models ($\mathcal{D}_{\text{logu-cv}}$, $\mathcal{D}_{\text{logu-act}}$) overlaid as solid green lines. $R^* = \text{per-regime static-}\sigma \text{ oracle}$; shading marks OOD bins.

The interpolation bins also reveal a qualitative gap between the discrete-prior oracle and the trained decoder. The oracle can only update posterior mass over $\{1, 10, 100\}$, so off-support evidence is forced toward nearby support points. By contrast, $\mathcal{D}_{\text{disc-cv}}$ behaves as if it retains a more continuous estimate of prefix scale: its thresholds interpolate between neighboring training regimes. The OOD gain therefore suggests that the model is not merely tabulating the discrete posterior, but is using likelihood information in $X_{1:t}$ that remains meaningful away from the training support.

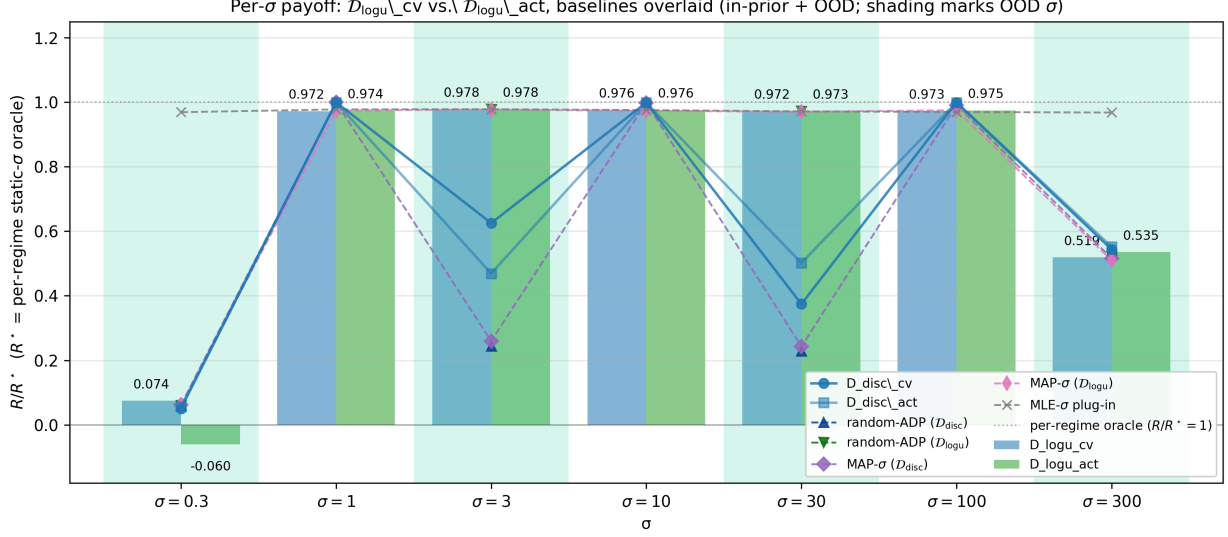


Figure 15: Per- σ payoff for $\mathcal{D}_{\text{logu_cv}}$ vs. $\mathcal{D}_{\text{logu_act}}$ across all seven test σ values, with the same five baselines and the alternative discrete-prior models ($\mathcal{D}_{\text{disc_cv}}$, $\mathcal{D}_{\text{disc_act}}$) overlaid as solid blue lines. R^* = per-regime static- σ oracle.

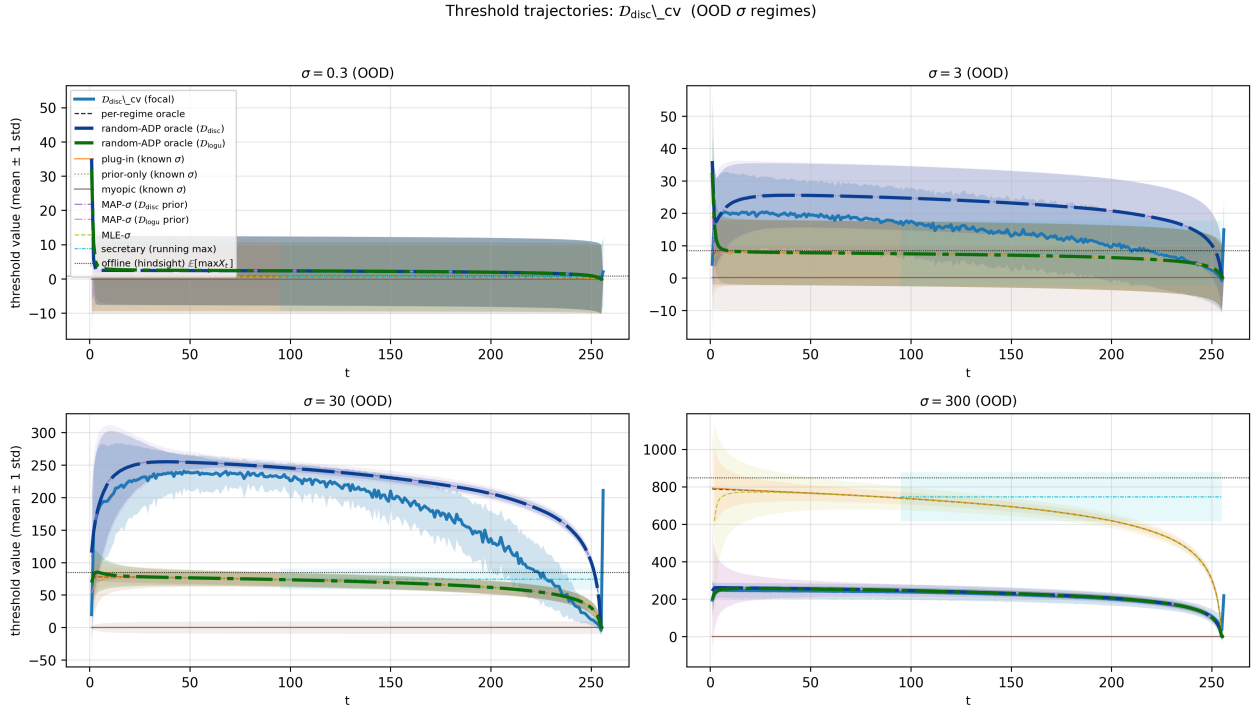


Figure 16: Threshold trajectories for $\mathcal{D}_{\text{disc_cv}}$ on the four OOD test regimes ($\sigma \in \{0.3, 3, 30, 300\}$). Each panel overlays the trained model (focal, blue) on every baseline from Section 4, with the random-ADP oracles emphasised as thick long-dash lines. In-distribution panels for $\sigma \in \{1, 10, 100\}$ are reported separately (Figure 9).

Figure 16 shows the same pattern at the trajectory level. In both the $\sigma = 3$ (top-right) and $\sigma = 30$ (bottom-left) panels, $\mathcal{D}_{\text{disc-cv}}$ ’s threshold settles at an interpolated level between its two nearest training σ values, above the random-ADP-disc oracle (which moves toward its nearest training modes) and above the matched MAP- σ plug-in. The trained model does not reach the per-regime static- σ oracle (black dashed) — that oracle knows the true σ exactly — but the gap pattern is consistent across both interpolation bins and matches the competitive-ratio gaps in Figure 14, so the bars and the trajectories tell the same story by two different routes.

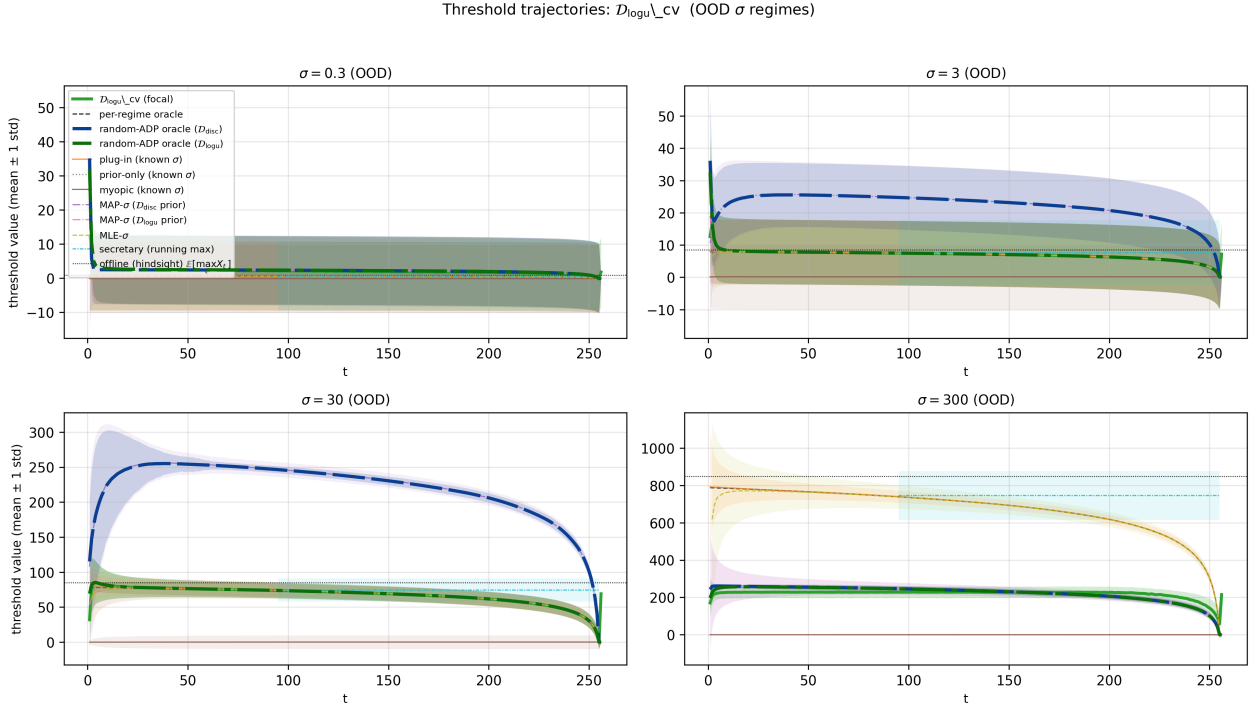


Figure 17: Threshold trajectories for $\mathcal{D}_{\text{logu-cv}}$ on the four OOD test regimes.

6.4 Temporal Shift

Matching the per-regime oracle on average (Sections 6.2–6.3) does not by itself prove per-sequence adaptation: a policy that averages over the prior could also score well on i.i.d. test regimes. To make per-sequence adaptation directly observable, we generate caches whose noise scale switches mid-sequence: for each ordered pair (σ_a, σ_b) we draw $N_{\text{test}} = 10^4$ sequences with $X_t \mid \mu_i, \sigma_a \sim \mathcal{N}(\mu_i, \sigma_a^2)$ for $t \leq 128$ and $X_t \mid \mu_i, \sigma_b \sim \mathcal{N}(\mu_i, \sigma_b^2)$ for $t \geq 129$, with the same $\mu_i \sim \mathcal{N}(\mu_0, \tau_0^2)$ throughout. Experiment B.1 uses pairs from the training-prior support $\{1, 10, 100\}$; Experiment B.2 uses pairs from the OOD support $\{3, 30, 300\}$.

Adaptation progress ρ_t . The model’s threshold $\hat{C}_t$ should sit near the static- σ_a oracle’s threshold while $t \leq 128$ and migrate toward the static- σ_b oracle’s threshold for $t > 128$. To put this on a common axis across all pairs — thresholds span orders of magnitude as σ ranges over $\{0.3, \dots, 300\}$ — we measure progress in log-units against the population-mean reference of each static- σ oracle, i.e. $\langle C_{\sigma_a, t}^* \rangle$ averaged over N_{test} iid- σ_a sequences (and analogously for σ_b). The normalized adaptation

ratio is

$$\rho_t := \frac{\log |\widehat{C}_t| - \log |\langle C_{\sigma_a, t}^* \rangle|}{\log |\langle C_{\sigma_b, t}^* \rangle| - \log |\langle C_{\sigma_a, t}^* \rangle|}.$$

By construction $\rho_t = 0$ when the model’s threshold matches the pre-shift reference and $\rho_t = 1$ when it matches the post-shift reference, with linear interpolation in log space between the two. Values outside $[0, 1]$ mean the model is undershooting the pre-shift level or overshooting the post-shift level. Operating in log-space makes the metric scale-invariant; absolute values inside the log handle any sign of $\widehat{C}_t$. We report the population-mean ρ_{200} as the “settled” adaptation level, far enough past the shift to read out steady-state behavior but before the terminal-horizon regime where the static oracles’ thresholds collapse and ρ_t becomes ill-conditioned.

Half-time $t_{1/2}$. Per sequence, $t_{1/2}$ is the smallest $\Delta t \geq 1$ such that $\rho_{128+\Delta t} \geq 0.5$ — the number of post-shift observations the model needs to move halfway from the σ_a regime to the σ_b regime in log-threshold units. A small $t_{1/2}$ indicates fast in-context adaptation; sequences whose ρ_t never reaches 0.5 within the 128-step post-shift window are counted at the ceiling $t_{1/2} = 128$ (the “never crossed” column in Table 6). Reporting the per-sequence distribution rather than the population mean exposes within-pair heterogeneity — some sequences happen to receive an early high- σ_b outlier and adapt in one step, others see a long run of in-range observations that look consistent with σ_a continuing.

The full per-shift summary statistics — mean and $p_{10}/p_{50}/p_{90}$ percentiles of $t_{1/2}$, pre/post-shift tracking errors, and ρ_{200} for each of the twenty (set, model, shift) cells — are tabulated in Appendix J (Table 6), together with per-sequence $t_{1/2}$ histograms (Figures 42, 43).

Threshold trajectories under regime shift: $\mathcal{D}_{\text{disc}} \backslash \text{cv}$

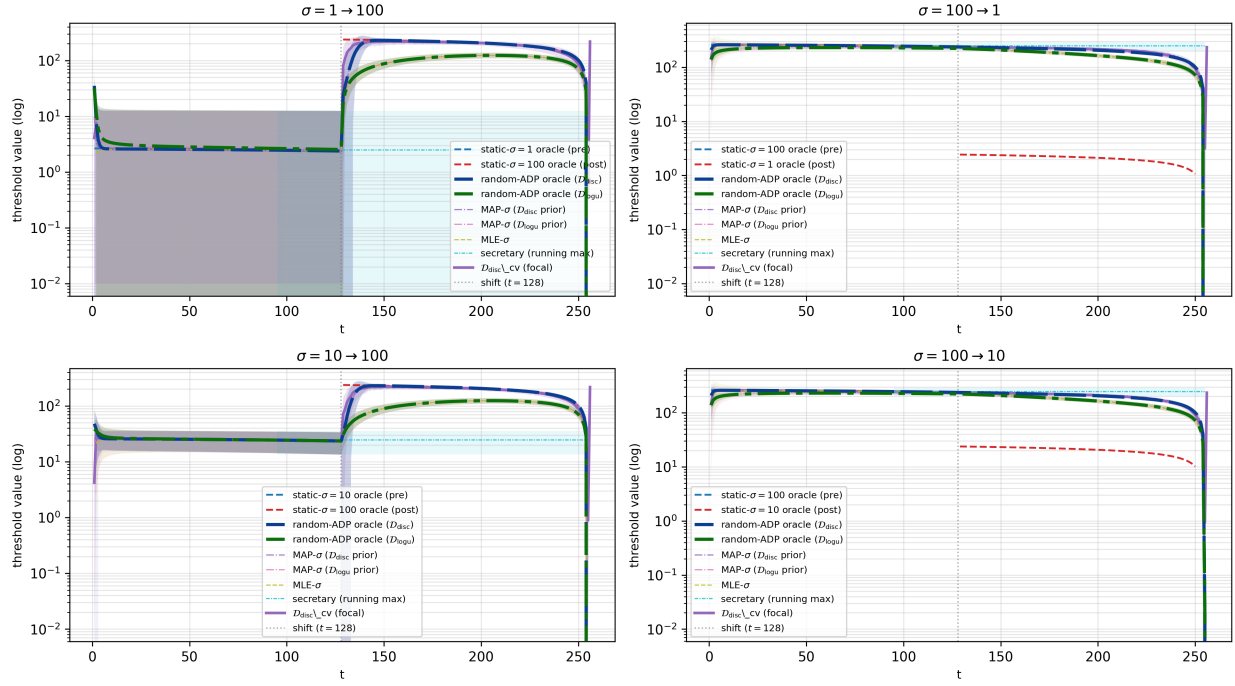


Figure 18: B.1 threshold trajectories (mean ± 1 std) for $\mathcal{D}_{\text{disc}} \backslash \text{cv}$ across the four training-prior pairs. Vertical dashed line at the regime switch $t = 128$. Random-ADP oracles at the matching priors are emphasised (thick long-dash for disc, thick long-dash-dot for logu).

Threshold trajectories under regime shift: $\mathcal{D}_{\log u_cv}$

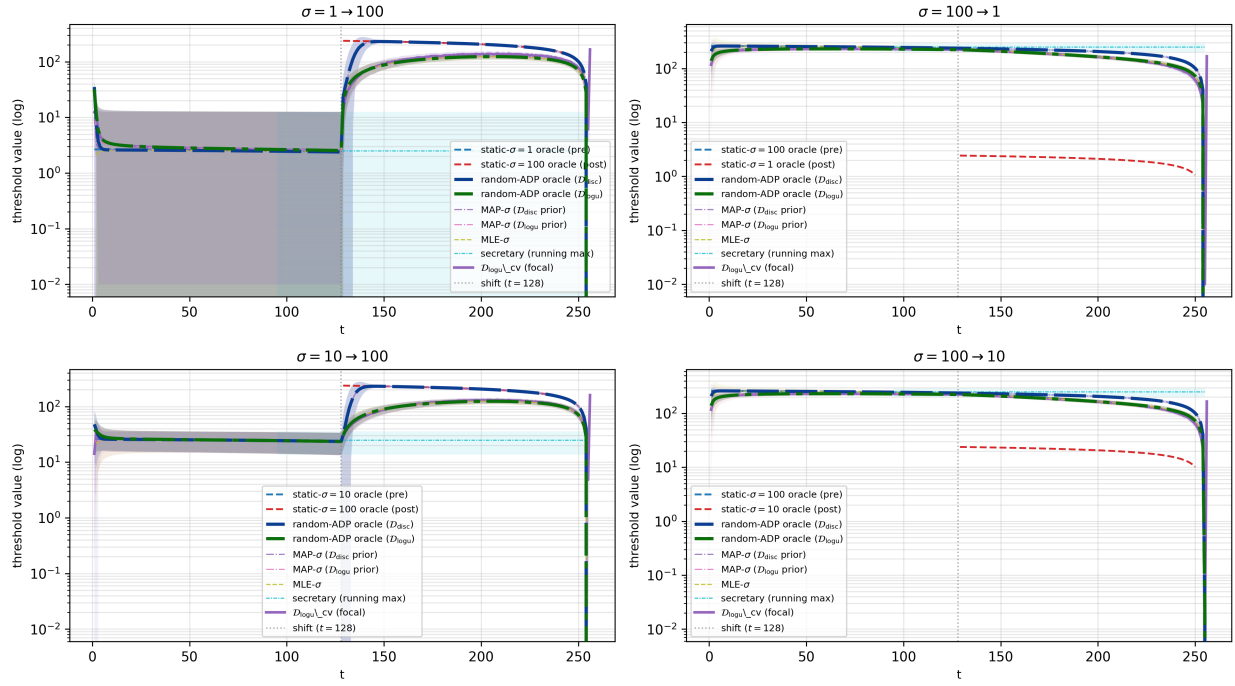


Figure 19: B.1 threshold trajectories for $\mathcal{D}_{\log u_cv}$ across the four training-prior pairs.

B.2 trajectories (OOD large-step pairs): $\mathcal{D}_{\text{disc_cv}}$

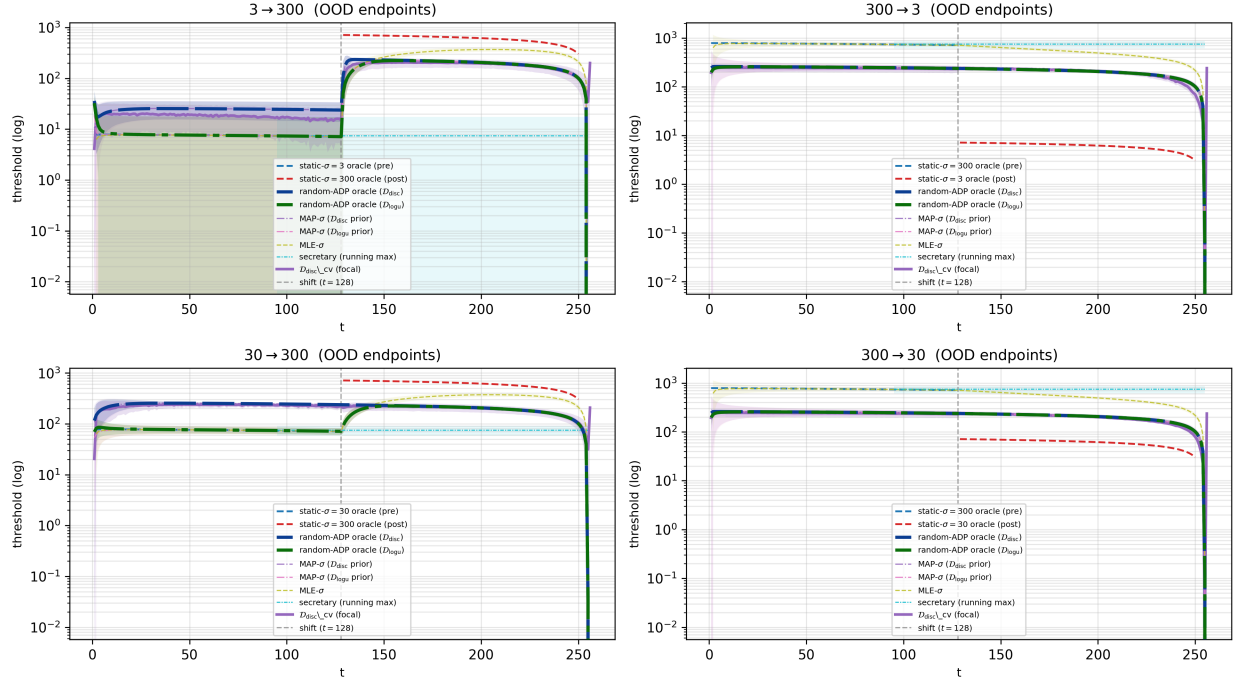


Figure 20: B.2 threshold trajectories for $\mathcal{D}_{\text{disc_cv}}$ across the four large-step OOD pairs.

B.2 trajectories (OOD large-step pairs): $\mathcal{D}_{\log u_cv}$

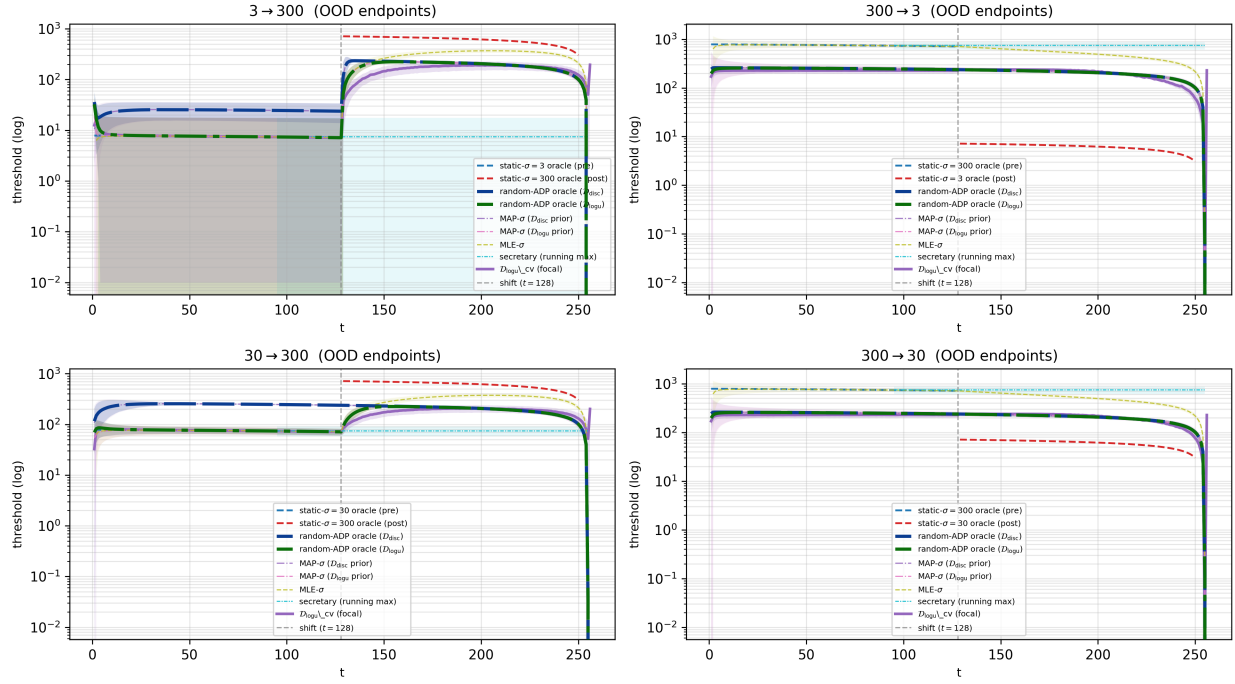


Figure 21: B.2 threshold trajectories for $\mathcal{D}_{\log u_cv}$ across the four large-step OOD pairs.

$\mathcal{D}_{\text{disc_cv}}$: memorized vs interpolated σ -conditional rule (100 $\times$ upward shift)

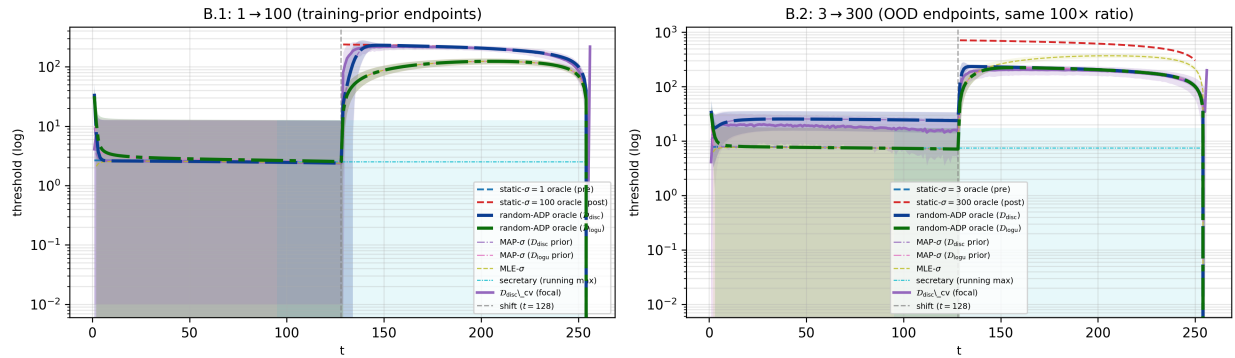


Figure 22: Side-by-side trajectory comparison for $\mathcal{D}_{\text{disc_cv}}$ on a 100 $\times$ upward shift: 1 $\rightarrow$ 100 (training-prior endpoints, B.1) vs. 3 $\rightarrow$ 300 (OOD endpoints, B.2). Same scale ratio, same direction; the OOD post-shift threshold saturates at the $\sigma = 100$ band rather than reaching the $\sigma = 300$ oracle.

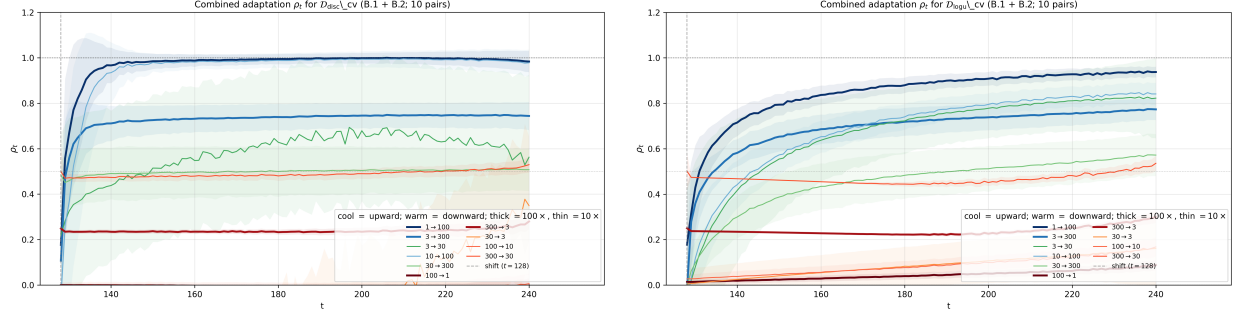


Figure 23: Combined adaptation ρ_t across all ten pairs (B.1 + B.2), $t \in [128, 240]$. Left: $\mathcal{D}_{\text{disc-cv}}$. Right: $\mathcal{D}_{\text{logu-cv}}$. Cool = upward shifts; warm = downward; thick = 100× ratio, thin = 10×.

The regime-shift results give direct evidence of in-context adaptation: after the switch, trained-model thresholds move in the direction of the post-shift static oracle, even though training sequences were i.i.d. Adaptation is asymmetric: upward shifts settle within a few observations, while downward shifts often miss $\rho = 0.5$ within the 128-step window. Two effects likely compound here. First, stopping at a maximum makes large values decision-critical, so the learned policy is naturally more responsive to the high tail than to the absence of large values. Second, Bayesian evidence for larger σ can arrive in one outlier, while evidence for smaller σ accumulates only gradually.

Equal weighting of past observations. Every baseline in Section 4 is derived under an i.i.d. assumption: $X_{1:t}$ is treated as evidence for one fixed distribution or one fixed prior mixture. On shifted sequences, this means post-shift observations are diluted by pre-shift observations rather than treated as evidence for a change point. The trained models inherit a milder version of the same bias: training data contained no temporal shifts, so the network appears to weight old and new observations roughly equally rather than explicitly down-weighting pre-shift evidence. Within this picture the $\sigma=1 \rightarrow 100$ gap between $\mathcal{D}_{\text{disc-cv}}$ ($t_{1/2} = 1.8$, post-shift error 17.9) and $\mathcal{D}_{\text{logu-cv}}$ ($t_{1/2} = 3.2$, post-shift error 45.1; Table 6, Figures 18–19) is a prior-shape effect: the disc prior has an atom at $\sigma = 100$, while the logu prior is continuous on $[1, 100]$ and leaves a visible saturation-level threshold gap. A policy that explicitly modeled change points or down-weighted observations older than a window T would answer a different problem.

7 Mechanistic Analysis: Hypothesis Testing and More

A mechanistic question raised by the random-variance results is whether the trained models internally represent quantities needed by a Bayes-style online stopping rule — sufficient statistics, uncertainty estimates, and continuation values — or whether they produce good actions through a less interpretable shortcut. We test this by training auxiliary regressors (probes) on frozen hidden states.

The scope here is narrower than the main empirical grid. Instead of revisiting all ten training runs, the controlled mechanistic experiments use only action-supervised models on $\mathcal{D}_{\text{logu}}$.

7.1 Probe Protocol

The protocol has three operations. The key design choice is that controls change the frozen base representation, while the downstream probe task, targets, and datasets stay fixed; Appendix A gives the longer methodological rationale.

1. Construct and freeze matched base representations.
2. Cache hidden states on a separate probe split and train readouts.
3. Compare paired held-out probe errors across ensemble members.

The schematic shows the base-model and probing dataflow; the paired tests operate on the held-out errors produced by the probes.

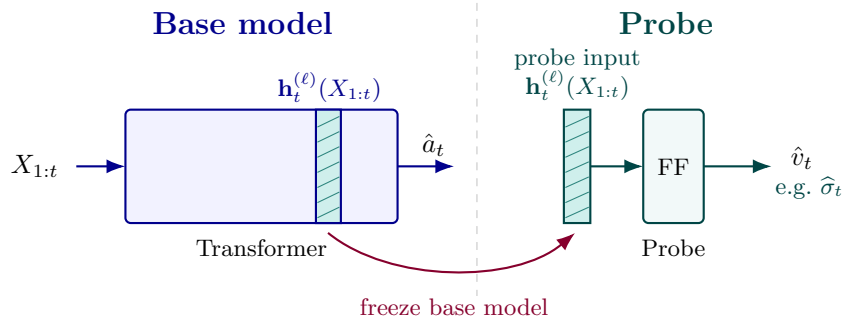


Figure 24: Schematic of the probing dataflow: freeze base models, cache $\mathbf{h}_t^{(\ell)}(X_{1:t})$, and train probes from those hidden states.

7.2 Base Models and Controls

To isolate whether probe success comes from the learned representation, each ensemble member contains three frozen base models, all matched by initialization θ_0 :

- *True trained base model*: train from θ_0 on the ordinary log-uniform action-supervision training dataset or stream, with the true oracle action labels.
- *Globally permuted-label base model*: train from θ_0 for the same budget as the true model, using the same base-training inputs but replacing only the training labels by a global shuffle over supervised sequence–time pairs. Let $\mathcal{R}_{\text{base}}$ index the base-training sequences, so a pair (r, t) means sequence $r \in \mathcal{R}_{\text{base}}$ at prefix length $t \in \{1, \dots, n-1\}$. With $\mathcal{I} = \mathcal{R}_{\text{base}} \times \{1, \dots, n-1\}$, draw a permutation $\pi : \mathcal{I} \rightarrow \mathcal{I}$; if $\pi(r, t) = (r', s)$, set $\tilde{y}_{r,t}^{\text{act, glob}} := y_{r',s}^{\text{act}}$.

- *Untrained-init base model*: the seed-deterministic initialization θ_0 itself, with no training applied.

The global shuffle preserves the overall action-label marginal while breaking both sequence and prefix-length alignment; only training labels are shuffled, with validation and test labels remaining true. The three controls answer different questions: perm-glob asks whether meaningful labels matter beyond having any training capacity; init asks whether training matters at all relative to architecture + positional priors. Together they let us separate the contribution of *having* training from the contribution of having *useful* training.

7.3 Probing Setup

For a frozen base model and prefix $X_{1:t}$, let $\mathbf{h}_t^{(\ell)}(X_{1:t}) \in \mathbb{R}^{d_{\text{emb}}}$ be the residual stream at the current position after block $\ell \in \{0, \dots, L\}$; $\ell = 0$ is the input-plus-position embedding and $\ell = L$ is the final decoder block. Causality makes this vector a function only of $X_{1:t}$. We also cache the prefix stack

$$H_{1:t}^{(\ell)}(X_{1:t}) := \begin{pmatrix} \mathbf{h}_1^{(\ell)}(X_{1:1}) & \dots & \mathbf{h}_t^{(\ell)}(X_{1:t}) \end{pmatrix}^\top \in \mathbb{R}^{t \times d_{\text{emb}}}.$$

The probes answer four representation questions: whether a quantity is decodable beyond the control representations; where it appears across layers and prefix lengths; whether the current-position state suffices or prefix pooling helps; and whether one decoder works across the horizon or a time-local decoder is needed. A positive probe result shows accessible representation, not causal use.

7.4 Probe Types

For a scalar target v_t depending on $X_{1:t}$, the suite has different probe types: two timestep-shared probes and fixed-prefix probes. Each base model, target v , layer ℓ , and probe type gets its own probe parameters. The simple variants read $\mathbf{h}_t^{(\ell)}(X_{1:t})$, while the attention-pooled variants read $H_{1:t}^{(\ell)}(X_{1:t})$. The attention-pooled readout adapts the position-attention idea used in [?] to prefix-valued stopping targets. Throughout this section, FF denotes a learned affine scalar readout, $\text{FF}(z) = w^\top z + b$. The two *timestep-shared* probes use one parameter set across all valid prefix lengths. They are not timestep-blind: $\mathbf{h}_t^{(\ell)}$ contains positional information, and the attention-pooled probe uses a prefix mask that changes with t .

Timestep-shared simple probe. The simplest probe reads only the hidden state at the current position:

$$\hat{v}_t = \text{FF}(\mathbf{h}_t^{(\ell)}),$$

This tests whether the current-position residual stream contains the target in a reusable, horizon-wide form.

Timestep-shared attention-pooled probe. The second shared probe can gather information from the whole prefix before applying the readout:

$$\boldsymbol{\alpha}_t = \text{softmax}((\mathbf{s}_v)_{1:t}) \in \Delta^{t-1}, \quad \hat{v}_t = \text{FF}\left(\sum_{i=1}^t (\boldsymbol{\alpha}_t)_i W_v \mathbf{h}_i^{(\ell)}\right),$$

where $\mathbf{s}_v \in \mathbb{R}^n$ is a learned position-score vector and $W_v \in \mathbb{R}^{d_{\text{emb}} \times d_{\text{emb}}}$ is a learned projection. This probe is strictly more general than the simple probe: it reduces to the simple probe by putting nearly all mass on position t and taking $W_v = I$, but it can also pool earlier positions.

Fixed-prefix probes. The remaining six probes are simple and attention-pooled variants trained separately at $t^* \in \{25, 125, 250\}$, covering early, middle, and late stages:

$$\begin{aligned}\hat{v}_{t^*}^{\text{simple}} &= \text{FF}_{t^*}(\mathbf{h}_{t^*}^{(\ell)}), \\ \boldsymbol{\alpha}_{t^*} &= \text{softmax}((\mathbf{s}_{v,t^*})_{1:t^*}) \in \Delta^{t^*-1}, \quad \hat{v}_{t^*}^{\text{attn}} = \text{FF}_{t^*}\left(\sum_{i=1}^{t^*} (\boldsymbol{\alpha}_{t^*})_i W_{v,t^*} \mathbf{h}_i^{(\ell)}\right).\end{aligned}$$

Each is trained and evaluated only at its selected prefix length. These readouts test whether a time-local decoder is easier than one shared across the horizon. They see about $n = 256$ times fewer prefix-target examples than timestep-shared probes, but solve an easier specialization problem.

Probe-attention localization. For attention-pooled probes we also report the learned position weights. For a fixed-prefix attention probe, each target v , prefix length t^* , and layer ℓ has weights $\boldsymbol{\alpha}_{t^*}^{v,\ell} \in \Delta^{t^*-1}$ over positions $1, \dots, t^*$. We visualize these as layer-position heatmaps, analogous to Figure 4 of ?]. These weights localize the probe readout, not the transformer’s internal self-attention heads.

7.5 Probe Loss

Probe training, validation selection, and test reporting all use the same pointwise σ -normalized MSE. For a prefix-target pair $(X_{1:t}, v_t)$ drawn from a sequence with true latent σ_i , the per-example error is

$$((\hat{v}_t - v_t)/\sigma_i)^2,$$

and the reported loss is the mean over the evaluation set $\mathcal{E}$:

$$\text{sMSE}_v(\mathcal{E}) := \frac{1}{|\mathcal{E}|} \sum_{(X_{1:t}, v_t, \sigma_i) \in \mathcal{E}} \left(\frac{\hat{v}_t(X_{1:t}) - v_t}{\sigma_i} \right)^2.$$

Scope of this choice. We use σ_i as the normalizer for the two preliminary targets, $\sqrt{\hat{\sigma}_t^2}$ and $\hat{C}_t^*$, because both scale linearly with the sequence’s latent σ_i (the former by definition, the latter by the Gaussian continuation recursion in Appendix D, equation (3)). This makes sMSE_v a fractional error on a common scale. Other targets, such as cumulants or log likelihoods, may require target-specific normalizers.

7.6 Probe Targets

We probe three tiers of quantities, each computable from $X_{1:t}$ and known oracle parameters: raw cumulants, posterior or likelihood quantities over σ , and continuation-value quantities. Raw cumulants ask whether the model stores basic prefix statistics; posterior and likelihood targets ask whether those statistics are organized into uncertainty estimates over σ ; continuation-value targets ask whether action-supervised models internally recover the Bayes value even though it is never a training label. The plug-in hypothesis gives special status to $\bar{X}_t$, $\sqrt{\hat{\sigma}_t^2}$, and η_t , since the MLE plug-in threshold is $\bar{X}_t + \sqrt{\hat{\sigma}_t^2} \eta_t$. The ADP coordinate λ_t is not a probing target. The full target list is in Appendix B, Table 1.

7.7 Paired Ensemble

Single matched comparisons give no uncertainty estimate. To quantify variability across dataset draws and initializations, we use a paired-ensemble design with N independent training datasets.

Base ensemble. For $i = 1, \dots, N$, sample an independent initialization $\theta_{0,i}$, true-label base-training data $\mathcal{D}_i^{\text{base,train}}$, and unpermuted validation/test caches. Applying the controls above gives frozen representations

$$b \in \{\text{true}, \text{perm-glob}, \text{init}\}.$$

The true and permuted-label models use the same base-model budget as the original $\mathcal{D}_{\log u}$ action run: fresh sampled sequences, batch size 64, and 1.5×10^5 gradient steps. They share inputs and differ only in training labels; checkpointing and action-performance reporting use true labels. The third representation, *init*, is regenerated from the seed used to draw $\theta_{0,i}$ and is shared across the pair (the true and permuted-label arms start from the same $\theta_{0,i}$). Across N members this yields $2N$ trained models in matched pairs plus N regenerated untrained models, i.e. three frozen representations per member.

Probe split. For each pair i , sample a separate probe train/validation/test split of length- $n = 256$ sequences,

$$\mathcal{D}_i^{\text{probe,train}}, \quad \mathcal{D}_i^{\text{probe,val}}, \quad \mathcal{D}_i^{\text{probe,test}},$$

disjoint from base training and validation data. The preliminary run uses $M_{\text{train}} = 1024$, $M_{\text{val}} = 512$, and $M_{\text{test}} = 10^4$. Let $\mathcal{P}$ be the eight probe configurations. For each base representation b , configuration q , target v , and layer ℓ , we train a separate probe with the same split, targets, optimizer, and budget; only the cached hidden states differ. Let $\mathcal{T}_{i,\text{test}}^{q,v}$ be the held-out prefix-target set induced by q (all valid prefixes for timestep-shared probes, only $t^* \in \{25, 125, 250\}$ for fixed-prefix probes). The held-out error is

$$\widetilde{M}_{i,q,v,\ell}^b := \text{sMSE}_v(\mathcal{T}_{i,\text{test}}^{q,v}).$$

We report the all errors separately rather than averaging across probe configurations.

7.8 Statistical Tests

We compare the true trained representation against *two* controls — the globally permuted-label model *perm-glob* (capacity-matched, random supervision) and the seed-deterministic untrained model *init* (no supervision at all). The primary inferential unit is a target-layer-probe contrast, not a separate hypothesis for every displayed prefix length. A statistical cell is a tuple

$$(v, \ell, q), \quad q \in \mathcal{P},$$

where v is the probe target, ℓ is the layer, and q is the probe configuration. For each cell and ensemble member i we form two paired improvements, one per control:

$$\Delta_{i,v,\ell,q}^{\text{perm-glob}} := \widetilde{M}_{i,q,v,\ell}^{\text{perm-glob}} - \widetilde{M}_{i,q,v,\ell}^{\text{true}}, \quad \Delta_{i,v,\ell,q}^{\text{init}} := \widetilde{M}_{i,q,v,\ell}^{\text{init}} - \widetilde{M}_{i,q,v,\ell}^{\text{true}}.$$

Positive Δ means the trained representation gives lower held-out relative error than the control for that target, layer, and probe configuration. The two contrasts answer different questions: $\Delta^{\text{perm-glob}}$ measures what *useful* supervision adds over random-label training of equal capacity; Δ^{init} measures what training adds at all over the seed-matched untrained model. Bonferroni multiplicity is applied per contrast, so each table is corrected over its own k cells.

7.8.1 Effect size

For each cell, the effect size is the mean paired improvement

$$\bar{\Delta}_{v,\ell,q} = \frac{1}{N} \sum_{i=1}^N \Delta_{i,v,\ell,q},$$

reported in the same σ -normalized-MSE units as the probe error. For example, $\bar{\Delta} = 0.05$ means that, on average across ensemble members, the control representation has 0.05 higher held-out σ -normalized MSE than the true trained representation for that target, layer, and probe. We also report the central-95% interval of the per-member differences — the 2.5th and 97.5th percentiles of $\Delta_1, \dots, \Delta_N$ — which describes the empirical spread across members and makes no distributional assumption (it is not a confidence interval for $\bar{\Delta}$, which would be narrower by a factor of order $\sqrt{N}$).

7.8.2 Sign test

We also report a one-sided paired sign test. Let

$$W_{v,\ell,q} := \sum_{i=1}^N \mathbb{I}[\Delta_{i,v,\ell,q} > 0].$$

Under the null that the true and control representations are exchangeable for this probe target, layer, and probe configuration, $W_{v,\ell,q} \sim \text{Binomial}(N, 1/2)$, and the cellwise p -value is

$$p_{v,\ell,q} = \Pr_{Z \sim \text{Binomial}(N, 1/2)}[Z \geq W_{v,\ell,q}].$$

The sign test answers how consistently the trained representation beats the control across independent datasets; the paired interval answers by how much.

7.9 Probing Experiment Results

We probe $N = 100$ matched true/perm-glob ensemble pairs at every depth $\ell \in \{0, \dots, 8\}$ of the 8-layer decoder, on two targets — the running standard-deviation estimate $\sqrt{\hat{\sigma}_t^2}$ and the numerical ADP continuation value $\hat{C}_t^*$ — and with both timestep-shared probe families: the *simple* readout of the current hidden state $\mathbf{h}_t^{(\ell)}$ and the *attention-pooled* readout of the causal prefix stack $H_{1:t}^{(\ell)}$. We use *two* controls. The first, perm-glob, has identical training capacity but globally permuted action labels: it isolates whether the trained representation reflects meaningful supervision versus capacity-matched fitting. The second, init, is the seed-deterministic *untrained* model (the step-0 state, regenerated from the same initialization seed as each member): it isolates what the architecture + positional embeddings yield before any training, so the init-vs-true gap measures what training itself adds. Each contrast is scored independently with the paired effect size, central-95% member interval, and sign test of Section 7.8; Bonferroni multiplicity is applied per contrast ($k = 18$ cells each).

We keep the probe-training split small by design. ?] define probe selectivity as the gap between a real probing task and a control task, and show smaller probe sets raise selectivity by limiting control-task memorization. Our analogue is the true-vs-perm-glob held-out error gap, so limited probe supervision is part of the design, not only a compute shortcut.

Plot bands. Shaded bands are the central-95% member interval of Section 7.8 (2.5–97.5 percentile of the per-member values); paired-effect and budget panels additionally show ± 1 standard deviation across members.

Significance reference. The table has $k = 2 \text{ targets} \times 9 \text{ layers} = 18$ cells per probe family, each its own hypothesis test. Testing 18 cells at the nominal 0.05 inflates the chance of at least one false positive; Bonferroni controls the family-wise error rate at 0.05 by testing each cell at $\alpha_{\text{cell}} = 0.05/18 \approx 2.78 \times 10^{-3}$ (the union bound: $18 \times \alpha_{\text{cell}} = 0.05$). For the $N = 100$ one-sided sign test this is a win-count threshold $W \geq 65/100$: $\Pr[W \geq 65] \approx 1.8 \times 10^{-3} < \alpha_{\text{cell}}$, while $W = 64$ gives 3.3×10^{-3} and just fails.

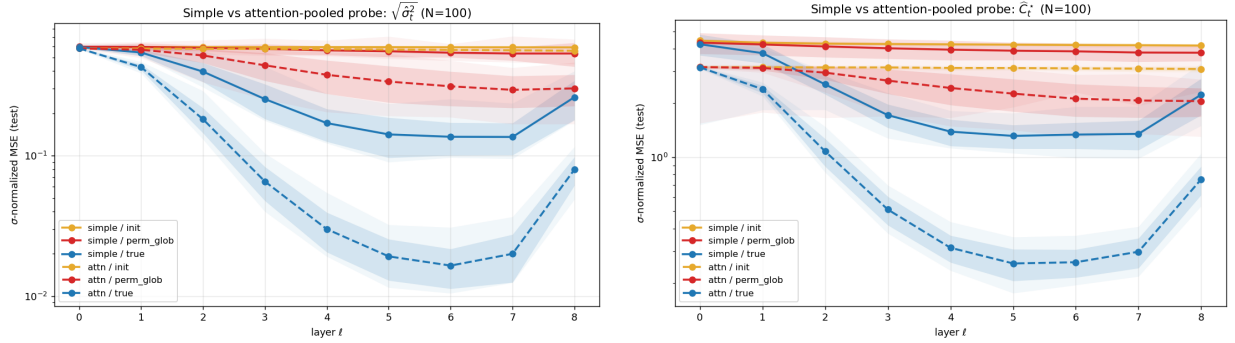


Figure 25: Simple vs. attention-pooled probe test error by layer, $\sqrt{\hat{\sigma}_t^2}$ (left) and $\hat{C}_t^*$ (right), $N = 100$. Solid = simple, dashed = attention-pooled; blue = true, red = perm-glob, orange-amber = init. Bands are the central 95% of members (2.5–97.5 percentile).

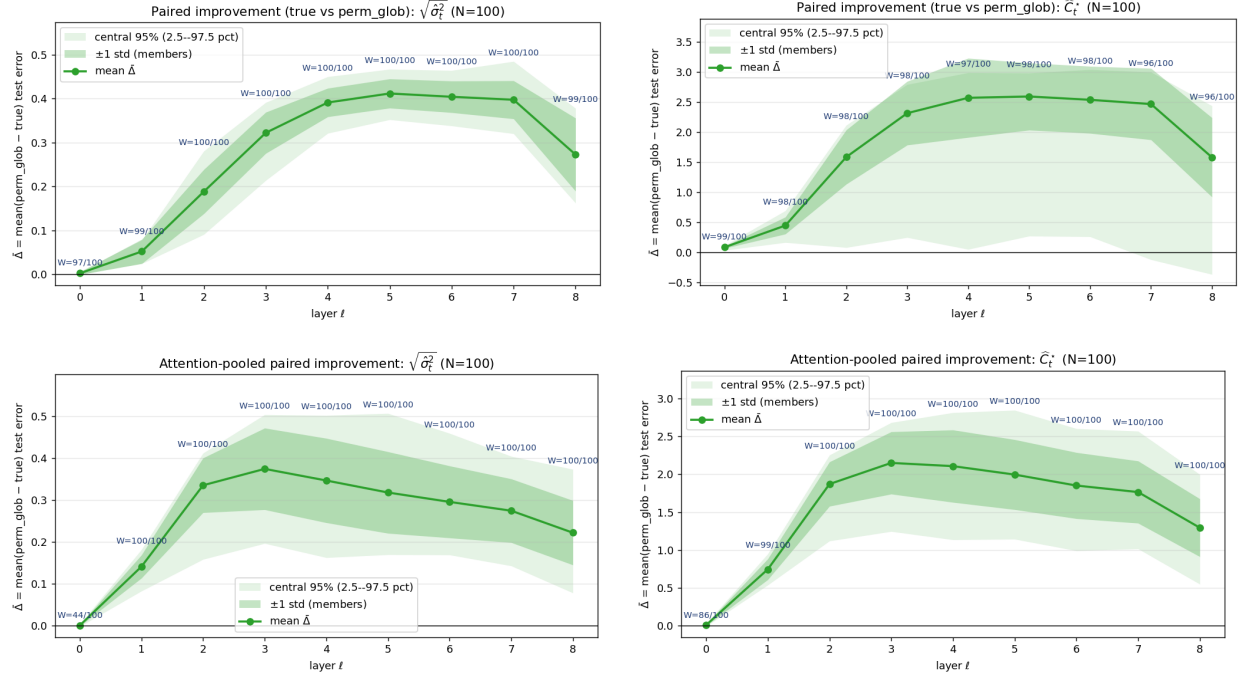


Figure 26: Per-layer paired effect size $\bar{\Delta} = \text{mean}(\text{perm-glob} - \text{true})$ test error (σ -normalized MSE), $\sqrt{\hat{\sigma}_t^2}$ (left) and $\hat{C}_t^*$ (right), for the *simple* probe (top row) and the *attention-pooled* probe (bottom row), $N = 100$. Dark green band: ± 1 std across members; light band: central 95% (2.5–97.5 percentile); dark-blue labels: sign-test win counts W/N . Positive $\bar{\Delta}$ = true representation has lower error.

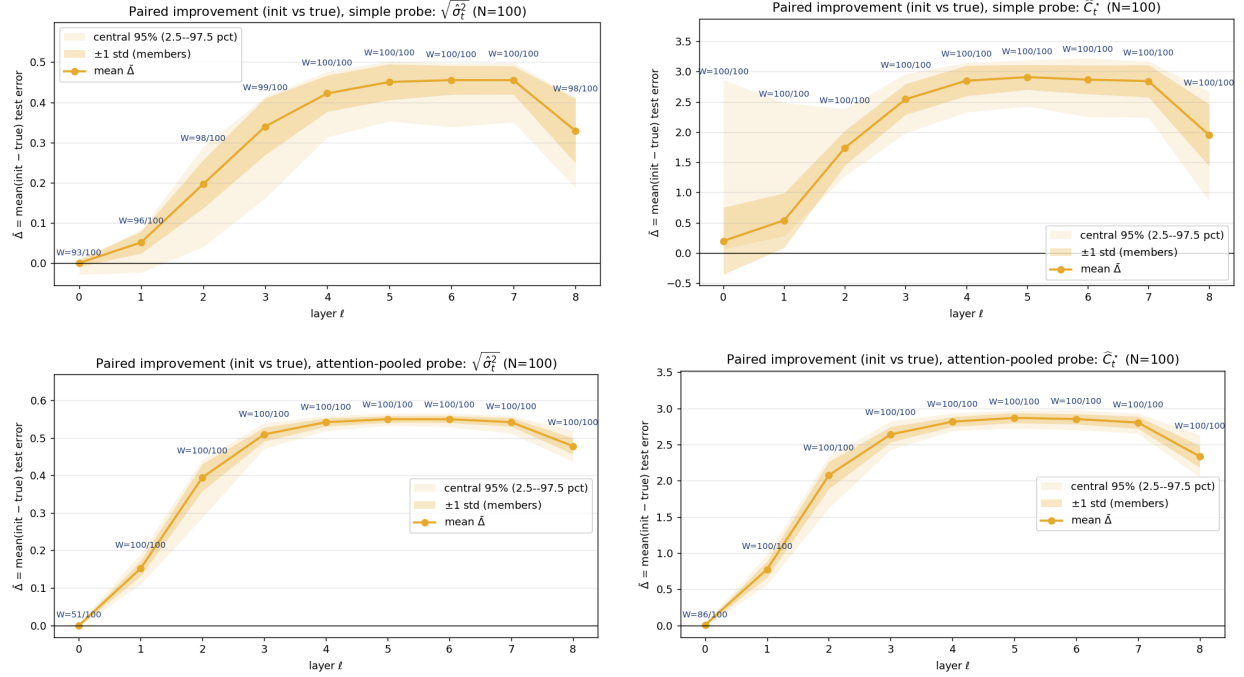


Figure 27: Per-layer paired effect size $\bar{\Delta} = \text{mean}(\text{init} - \text{true})$ test error (σ -normalized MSE), $\sqrt{\hat{\sigma}_t^2}$ (left) and $\hat{C}_t^*$ (right), for the *simple* probe (top row) and the *attention-pooled* probe (bottom row), $N = 100$. Same band conventions and Bonferroni reference as Figure 26; positive $\bar{\Delta}$ = the trained representation has lower error than the seed-matched untrained model, i.e. the gap is what training adds.

Label budget sweep. To quantify how much probe supervision the readout needs, we train every probe cell on a nested sequence-budget grid $M_{\text{train}} \in \{64, 128, 256, 512, 1024\}$ with validation and test splits held fixed. Because the probes are timestep-shared, one training sequence contributes one example per valid prefix, so M_{train} corresponds to $B_v = M_{\text{train}} |\mathcal{T}_v|$ labels ($|\mathcal{T}_v| = 255$ for $\hat{C}_t^*$ over $t = 1, \dots, 255$ and 254 for $\sqrt{\hat{\sigma}_t^2}$ over $t = 2, \dots, 256$). The curves (Figures 28–29) show the true representation reaches a given validation error with far fewer labels than perm-glob at every mid-stack layer, and the true-vs-control gap widens with budget rather than closing — the gap is a property of the representation, not of probe undertraining.

Budget sweep, simple vs attention-pooled: $\sqrt{\hat{\sigma}_t^2}$ (mean $\pm$ std across members; shared y-axis)

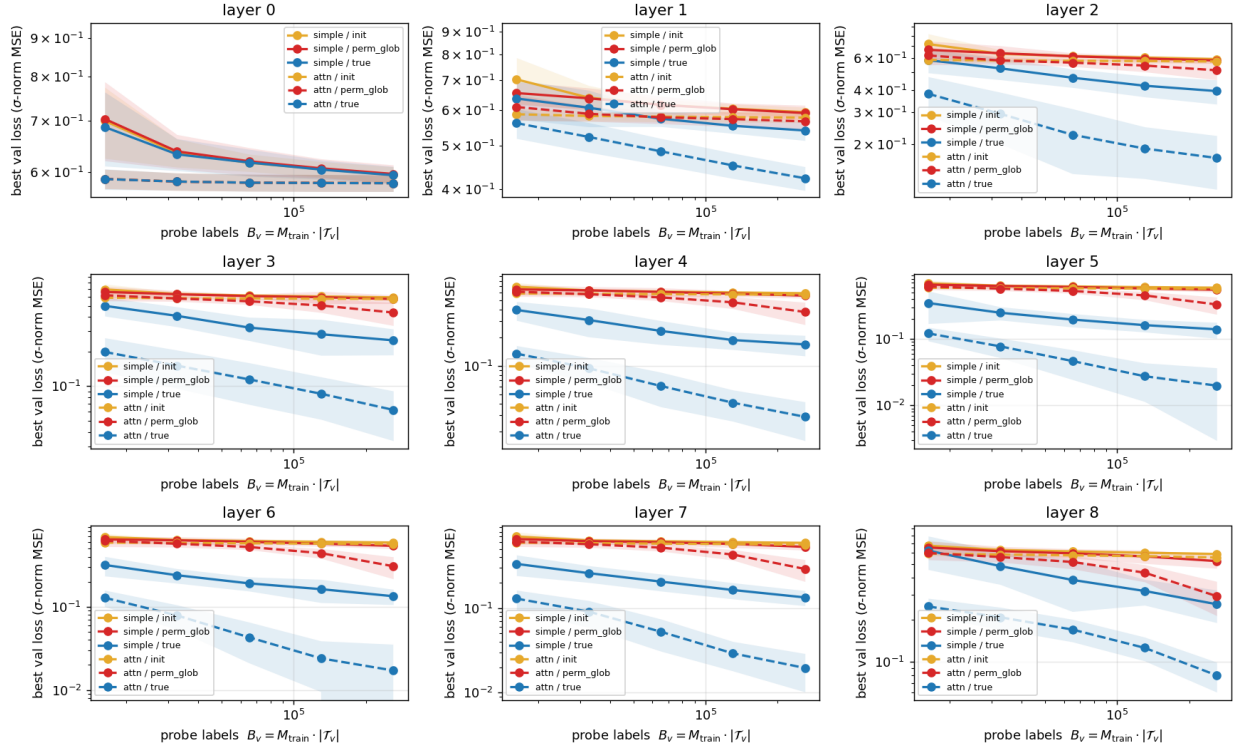


Figure 28: Probe-label budget sweep for $\sqrt{\hat{\sigma}_t^2}$, $N = 100$, both probe families overlaid. Each panel is one layer; x -axis is total probe labels B_v (log), y -axis best validation error (log, shared across panels). Solid = simple probe, dashed = attention-pooled; blue = true, red = perm-glob, orange-amber = init; bands are ± 1 std across members.

Budget sweep, simple vs attention-pooled: $\hat{C}_t^*$ (mean $\pm$ std across members; shared y-axis)

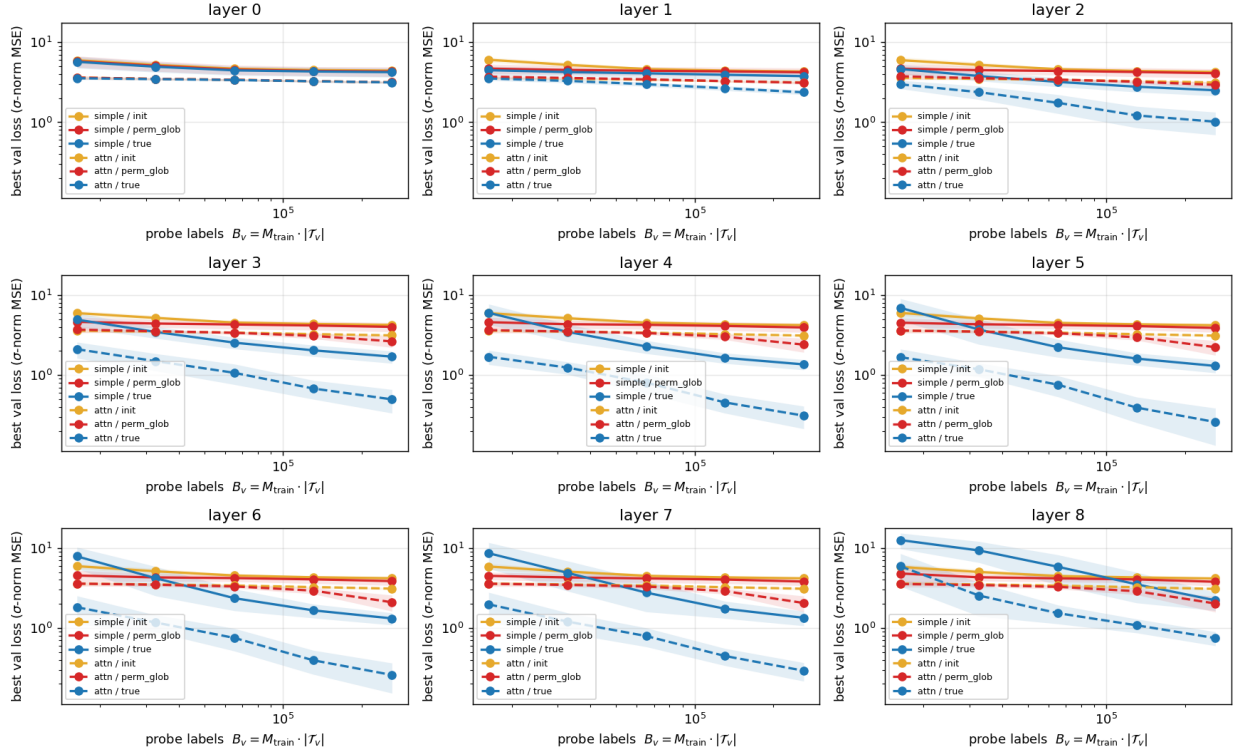


Figure 29: Probe-label budget sweep for $\hat{C}_t^*$, $N = 100$, both probe families overlaid. Layout, axes, line styles (solid = simple, dashed = attention-pooled), colours, and ± 1 std bands as in Figure 28.

References

A Probing Controls: Literature Review

Probe accuracy is not, by itself, evidence that a model uses the probed quantity. A powerful probe can sometimes exploit high-dimensional hidden states to predict a label even if the base model did not organize its computation around that label [?]. This concern motivates control tasks: [?], for example, map word types to random labels and compare true-task probe performance with this memorization baseline. The useful idea for us is the label permutation: because it preserves marginal label statistics while breaking input–label alignment, it asks how much of the measured probe success could arise from supervised fitting capacity to marginally similar randomness rather than from meaningful representation.

The difficulty is that many such controls are downstream controls: the base model is held fixed, but the probe target is changed. If one probe predicts the true label and another predicts shuffled labels, the two probes no longer solve the same statistical problem. The comparison can mix representational evidence with target difficulty, label entropy, and probe memorization. Degenerate label controls, such as fixed or nearly constant labels, have a related problem: they can collapse the probe into an intercept-like predictor, so failure against a varying label is not a meaningful baseline. This is why sanity checks of the kind used in probing and in-context-learning analyses [? ?] could be useful preliminary diagnostics but are not statistically grounded controls.

One reason standard probing often holds the base model fixed is pragmatic: retraining a BERT-scale or other multi-layer transformer for every control is expensive. Our protocol, however, retrains the base model using matched true-label and permuted-label training runs, and only then freeze the resulting models for identical downstream probes. This asks a targeted question in a statistically grounded way: does true action supervision reproducibly make stopping-relevant quantities, such as $\sqrt{\hat{\sigma}_t^2}$ and $\hat{C}_t^*$, more decodable from the frozen representation? The true and control probes have the same label distribution, loss, architecture, optimizer, and evaluation split; only the representation supplied by the frozen base model changes. This still does not by itself prove causal use, but it moves the evidence beyond a single-checkpoint audit.

The probe-label budget sweep in Section 7.9 also freezes the base model: it asks how many supervised probe labels are needed to approximate a target from an existing representation. This is close in spirit to MDL probing, which treats label efficiency as evidence about how readily a property can be extracted [?].

Finally, we use matched ensembles rather than a single checkpoint because learned representations can vary across random seeds [?]. These tests support claims about improved decodability; for causal conclusion, we plan to use separate intervention experiments, such as activation patching or interchange interventions [?].

B Probe Targets

Table 1 lists the targets used by the mechanistic probing protocol in Section 7.6. Each target is determined from the observed prefix $X_{1:t}$ and known oracle parameters; probes are trained to recover it from hidden states.

notation	definition	what it represents
<i>Tier 1 — Sufficient statistics</i>		
S_t	$\sum_{s \leq t} X_s$	cumulative sum of observations through t
Q_t	$\sum_{s \leq t} X_s^2$	cumulative sum of squared observations
$\bar{X}_t$	S_t/t	running sample mean (estimator of μ_i)
$\hat{\sigma}_t^2$	$\frac{1}{t-1} \sum_{s \leq t} (X_s - \bar{X}_t)^2 = \frac{Q_t - t \bar{X}_t^2}{t-1}$	Bessel-corrected running sample variance of $X_{1:t}$ (unbiased estimator of σ_i^2 under fixed μ_i); defined for $t \geq 2$
<i>Tier 2 — Posterior probabilities and point estimates</i>		
$\hat{\sigma}_t^{\text{MAP}}$	$\arg \max_{\sigma} [\log p(X_{1:t} \sigma) + \log p(\sigma)]$	MAP- σ under the model’s training prior $p(\sigma)$ (3-pt discrete on $\mathcal{D}_{\text{disc}}$, log-uniform on $\mathcal{D}_{\text{logu}}$); marginal likelihood given by Eq. (1)
$\sqrt{\hat{\sigma}_t^2}$	$\sqrt{\frac{Q_t - t \bar{X}_t^2}{t-1}}$	prior-free running standard deviation used by the MLE- σ plug-in baseline; the Bessel correction differs from the divisor- t Gaussian MLE by a factor $1 + O(1/t)$
$\log p(X_{1:t} \sigma)$	Eq. (1) at $\sigma \in \{1, 10, 100\}$	log marginal likelihood at three fixed σ values (one target each)
$\log p(X_{1:t} \sigma_i)$	Eq. (1) at $\sigma = \sigma_i$	log marginal likelihood at the <i>true</i> per-sequence latent (model never sees σ_i)
<i>Tier 3 — Continuation value</i>		
η_t	recursion in Eq. (3)	deterministic standardized continuation offset; a position/horizon quantity used by plug-in thresholds
$\hat{C}_t^*$	ADP table for $\mathcal{D}_{\text{train}}$	numerical ADP continuation value under the model’s training prior
$\hat{C}_t^{\text{plug-in, MAP}}$	$\bar{X}_t + \hat{\sigma}_t^{\text{MAP}} \eta_t$	data-only plug-in threshold using MAP- σ
$\hat{C}_t^{\text{plug-in, MLE}}$	$\bar{X}_t + \sqrt{\hat{\sigma}_t^2} \eta_t$	data-only plug-in threshold (no σ -prior); conventionally called the “MLE- σ plug-in”

Table 1: Probe targets. These quantities are determined from $X_{1:t}$ and known oracle parameters; probes are trained to recover each from the model’s hidden state at every layer.

C Secondary Probing Targets

We repeat the probing protocol of Section 7.9 on two further targets that sharpen the interpretation of the main results. Both use the same frozen base models, cached hidden states, splits, probe families, and percentile/sign-test machinery; only the target and its loss differ.

C.1 Horizon Offset η_t

η_t is the standardized known- μ continuation offset of Eq. (3): it depends only on the timestep t , not on $X_{1:t}$ or σ_i , and we probe it with a raw-MSE readout. Because it is constant across sequences at each t , it could be stored in the weights rather than inferred in-context; it carries no in-context inferential content, so a smaller ensemble suffices and we report it at $N = 25$. It is a position/horizon diagnostic, not evidence of in-context inference (Figures 30–32).

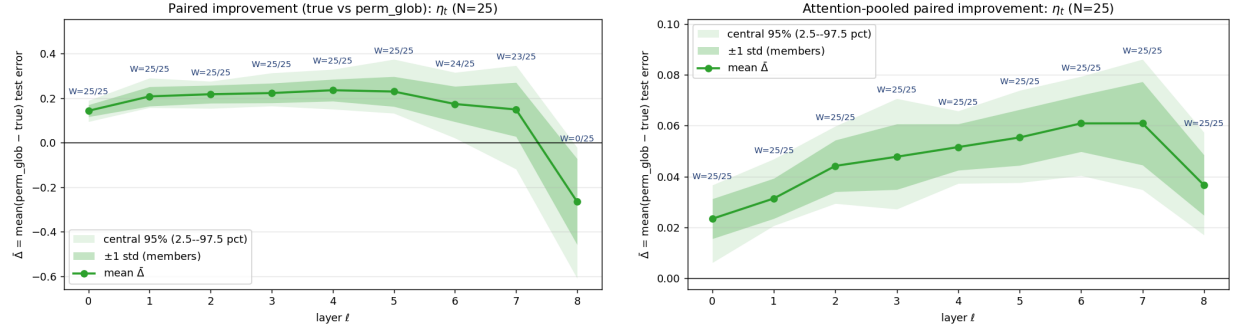


Figure 30: η_t probe ($N = 25$): per-layer paired effect size for the simple probe (left) and the attention-pooled probe (right). Raw η_t MSE.

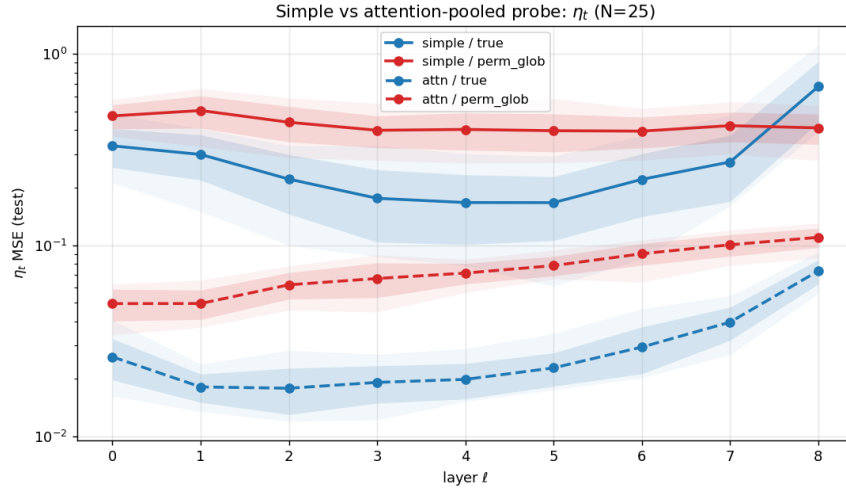


Figure 31: η_t probe ($N = 25$): test error by layer, simple vs. attention-pooled probe (solid = simple, dashed = attention-pooled).

Budget sweep, simple vs attention-pooled: η_t (mean $\pm$ std across members; shared y-axis)

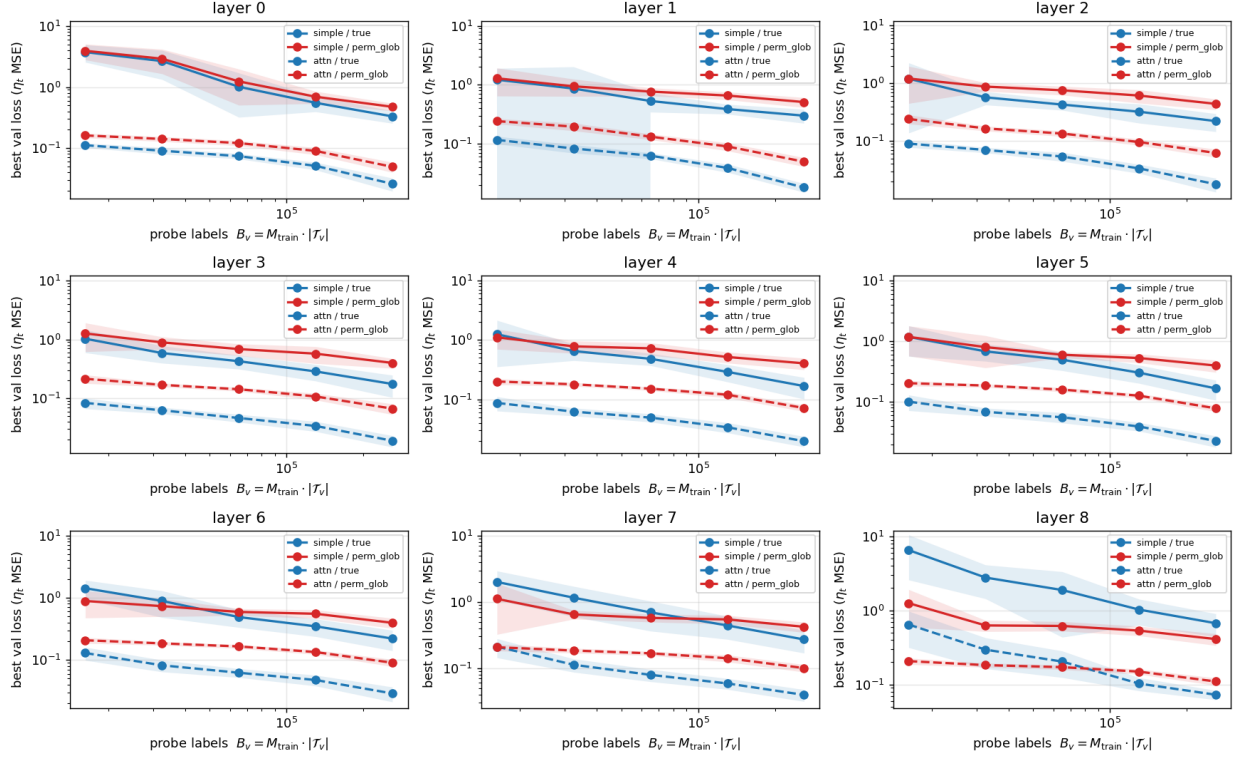


Figure 32: η_t probe ($N = 25$): probe-label budget sweep with both probe families overlaid (solid = simple, dashed = attention-pooled; blue = true, red = perm-glob).

C.2 Exponentiated Log- σ Estimate

The second target is the online prefix standard deviation $\hat{\sigma}_t$, probed through a log-output parameterization ($\hat{\sigma}_{\text{pred}} = \exp a$) trained with relative σ -space MSE. The log parameterization is the one the ADP oracle itself uses: the random-variance solver indexes its continuation table by $\lambda_t = \log \hat{\sigma}_t^2$ on a uniform grid and (for $\mathcal{D}_{\log u}$) marginalizes σ on a log-spaced grid (Appendix G), because σ spans orders of magnitude and the log scale gives a far better grid approximation than a linear one. Probing a log-output $\hat{\sigma}_t$ therefore reads out the same log-variance coordinate the oracle’s table is built on. The true representation decodes it markedly better than the control across mid-stack layers, with the same depth profile as the main $\sqrt{\hat{\sigma}_t^2}$ target (Figures 33–35). That this simple running-SD estimate is more cleanly and cheaply linearly decodable than the full ADP continuation value $\hat{C}_t^*$ is further evidence the transformer organizes around simple data statistics rather than reproducing the exact ADP recursion internally.

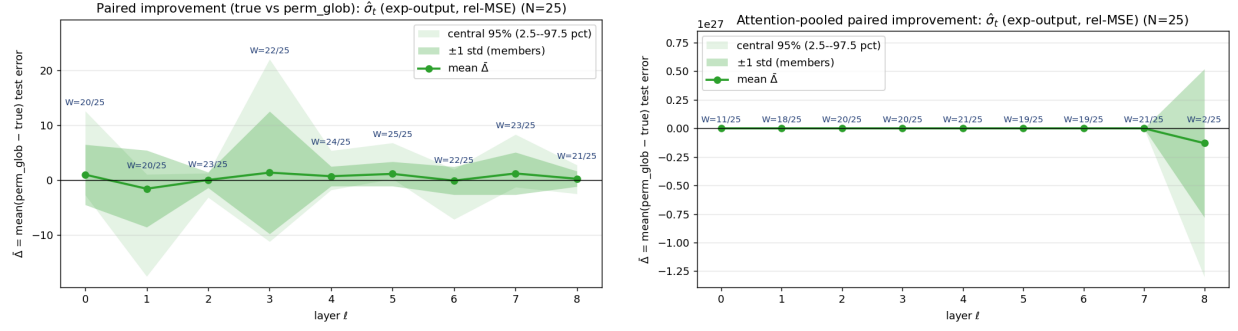


Figure 33: Exponentiated log- σ probe ($N = 25$): per-layer paired effect size for the simple probe (left) and the attention-pooled probe (right). Relative σ -space MSE.

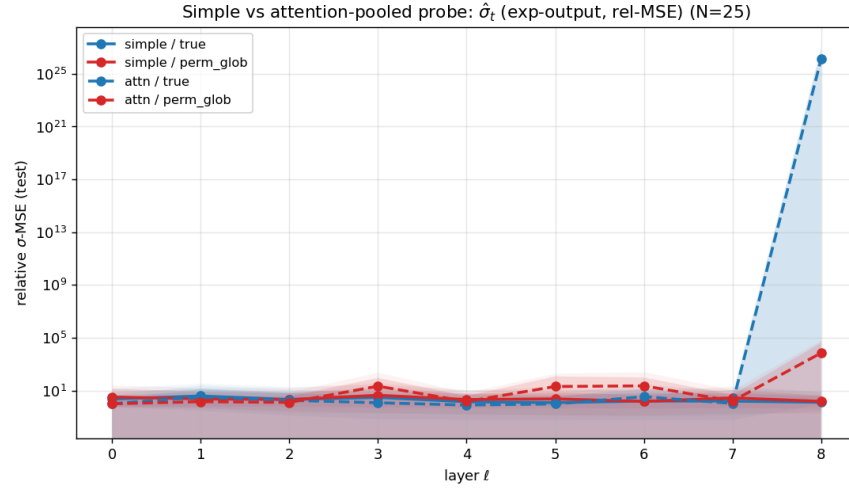


Figure 34: Exponentiated log- σ probe ($N = 25$): test error by layer, simple vs. attention-pooled probe (solid = simple, dashed = attention-pooled).

Budget sweep, simple vs attention-pooled: $\hat{\sigma}_t$ (exp-output, rel-MSE) (mean $\pm$ std across members; shared y-axis)

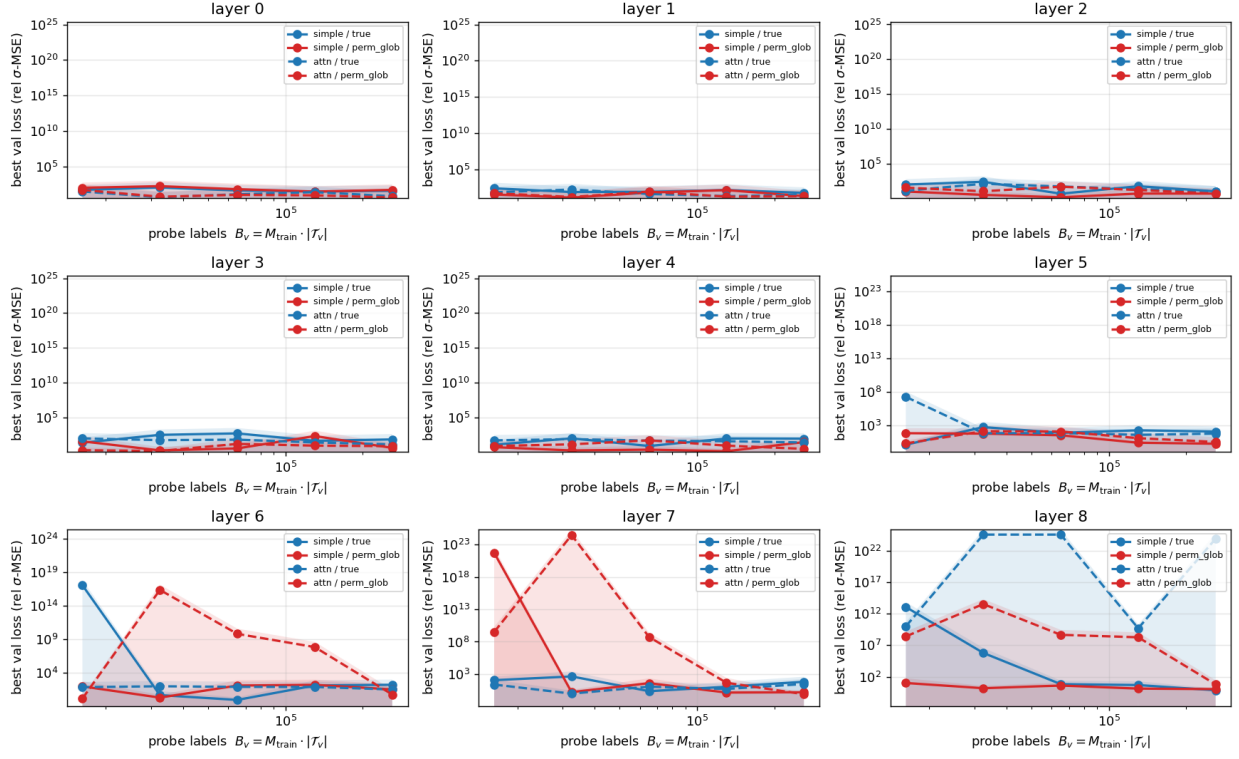


Figure 35: Exponentiated log- σ probe ($N = 25$): probe-label budget sweep with both probe families overlaid (solid = simple, dashed = attention-pooled; blue = true, red = perm-glob).

D Baseline Details

The baseline policies are defined in Section 4. This appendix gives only the two derivations used there: the standardized continuation offset η_t and the offline-optimum expectation.

Standardized continuation-value recursion. Throughout this appendix, Φ and φ denote the standard-normal CDF and PDF, respectively. Suppose first that both μ and σ are known, so the offers $X_1, \dots, X_n$ are i.i.d. $\mathcal{N}(\mu, \sigma^2)$ and no belief update is needed. The optimal policy at step t accepts X_t iff X_t exceeds the value of continuing,

$$c_t := \mathbb{E}[\max\{X_{t+1}, c_{t+1}\}], \quad t = 1, \dots, n-1,$$

with terminal condition $c_{n-1} = \mathbb{E}[X_n] = \mu$ (at $t = n$ the agent must accept X_n). Each c_t is a deterministic real number: the expectation is over $X_{t+1} \sim \mathcal{N}(\mu, \sigma^2)$ and c_{t+1} has no remaining randomness. To put (3) in closed form we use the truncated-normal identity

$$\mathbb{E}[\max\{X, a\}] = \mu + \sigma \psi\left(\frac{a-\mu}{\sigma}\right), \quad \psi(z) := z\Phi(z) + \varphi(z), \quad (2)$$

valid for any $a \in \mathbb{R}$ and $X \sim \mathcal{N}(\mu, \sigma^2)$. To see it, split the expectation at a , change variables $u = (x - \mu)/\sigma$, and use $\int_z^\infty u \varphi(u) du = \varphi(z)$:

$$\mathbb{E}[\max\{X, a\}] = a\Phi\left(\frac{a-\mu}{\sigma}\right) + \mu(1 - \Phi\left(\frac{a-\mu}{\sigma}\right)) + \sigma\varphi\left(\frac{a-\mu}{\sigma}\right),$$

which collapses to (2). Defining the *standardized* continuation value $\eta_t := (c_t - \mu)/\sigma$ and applying (2) with $X = X_{t+1}$, $a = c_{t+1}$ gives $c_t = \mu + \sigma\psi(\eta_{t+1})$, so the recursion is $\eta_t = \psi(\eta_{t+1})$ and the dependence on μ, σ drops out:

$$\eta_{n-1} = 0, \quad \eta_t = \psi(\eta_{t+1}), \quad t = n-2, \dots, 1. \quad (3)$$

The known- μ , known- σ optimal rule is therefore $\mathbf{1}[X_t \geq \mu + \sigma\eta_t]$.

Offline optimum. Under regime $\mathcal{D}_i$, the offers are $X_t = \mu + \sigma_i \varepsilon_t$ with $\varepsilon_t \sim \mathcal{N}(0, 1)$ i.i.d. and $\mu \sim \mathcal{N}(\mu_0, \tau_0^2)$ independent of $\{\varepsilon_t\}$, so

$$\max_t X_t = \mu + \sigma_i Z_n, \quad Z_n := \max_{t \leq n} \varepsilon_t.$$

Taking expectations and using $\mathbb{E}[\mu] = \mu_0 = 0$ gives

$$\mathbb{E}[\max_t X_t; \mathcal{D}_i] = \mathbb{E}[\mu] + \sigma_i \mathbb{E}[Z_n] = \sigma_i \mathbb{E}[Z_n].$$

The maximum of n i.i.d. standard normals has CDF $\Pr[Z_n \leq z] = \Phi(z)^n$, so its density is $f_{Z_n}(z) = n\Phi(z)^{n-1}\varphi(z)$ (Φ, φ defined as in the previous paragraph), giving

$$\mathbb{E}[Z_n] = \int_{-\infty}^{\infty} z \cdot n\Phi(z)^{n-1}\varphi(z) dz.$$

The integral has no elementary closed form; we evaluate it to machine precision by 1D quadrature.

E Mechanistic Hypothesis Proofs

E.1 Coordinates

The proof stores a logical coordinate vector $z_t^{(\ell)}$ at each position t and block ℓ . Physically, this vector is encoded as $h_t^{(\ell)} = E(z_t^{(\ell)})$; Lemma 3 defines E and shows that LayerNorm preserves linear access to every coordinate. The coordinates used for the plug-in construction are

$$(z_t^{(\ell)})^\top = [u_t^\top, 1, X_t, X_t^2, \bar{X}_t, \overline{X^2}_t, S_t, Q_t, \hat{\sigma}_t^{\text{MLE}}, \\ \ell_{t,1}, \dots, \ell_{t,K}, \alpha_{t,1}^{(\beta)}, \dots, \alpha_{t,K}^{(\beta)}, \\ \hat{\sigma}_{t,\beta,K}^{\text{MAP}}, \hat{\sigma}_t, \theta_t, m_t, \xi_t^\top]. \quad (4)$$

Here $\overline{X^2}_t = t^{-1} \sum_{s \leq t} X_s^2$, $m_t = X_t - \theta_t$, and u_t contains the position-dependent constants. The vector $\xi_t \in \mathbb{R}^{d_\xi}$ contains temporary scratch coordinates used only during the finite arithmetic construction, for example to hold products, reciprocals, exponentials, or other intermediate quantities. All other coordinates have the meanings fixed in Section 5.

Let e_c denote the logical basis vector selecting the coordinate slot in z_t that stores quantity c , and let $r_c^\top$ denote the readout row for coordinate c . Lemma 3 gives

$$r_c^\top \text{LN}(E(z_t)) = e_c^\top z_t.$$

The matrices in the proof below are written in these logical coordinates. A physical residual block implements a logical update $z \mapsto z + \Delta(z)$ by outputting $E(z + \Delta(z)) - E(z)$. The only operations needed are a causal attention head that averages prefix coordinates and ReLU feed-forward blocks that implement scalar piecewise-linear tables; both calculations are carried out where they are used in the proof.

E.2 Main Proofs

Proof of Lemma 1. By Bayes' rule on the finite grid Σ_K ,

$$p(\sigma_k \mid X_{1:t}) = \frac{w_k p(X_{1:t} \mid \sigma_k)}{\sum_{r=1}^K w_r p(X_{1:t} \mid \sigma_r)}.$$

The denominator is independent of k . Therefore the posterior-maximizing grid point is

$$k_t^* \in \arg \max_{1 \leq k \leq K} w_k p(X_{1:t} \mid \sigma_k) = \arg \max_{1 \leq k \leq K} \ell_{t,k}, \quad \hat{\sigma}_{t,K}^{\text{MAP}} := \sigma_{k_t^*}.$$

□

Proof of Lemma 2. For $r \neq k_t^*$, write $\Delta_r := \ell_{t,k_t^*} - \ell_{t,r} > 0$. Dividing numerator and denominator of the softmax by $\exp(\beta \ell_{t,k_t^*})$ gives

$$\alpha_{t,k_t^*}^{(\beta)} = \frac{1}{1 + \sum_{r \neq k_t^*} e^{-\beta \Delta_r}}, \quad \alpha_{t,r}^{(\beta)} = \frac{e^{-\beta \Delta_r}}{1 + \sum_{j \neq k_t^*} e^{-\beta \Delta_j}}.$$

Since every $\Delta_r > 0$, $e^{-\beta \Delta_r} \rightarrow 0$ as $\beta \rightarrow \infty$. Thus the weight on k_t^* converges to 1 and all other weights converge to 0, which proves $\hat{\sigma}_{t,\beta,K}^{\text{MAP}} \rightarrow \sigma_{k_t^*}$.

For the quantitative bound, $\Delta_r \geq \gamma_t$ for all $r \neq k_t^*$, so

$$\sum_{r \neq k_t^*} \alpha_{t,r}^{(\beta)} \leq \sum_{r \neq k_t^*} e^{-\beta \Delta_r} \leq (K-1)e^{-\beta \gamma_t}.$$

Finally,

$$|\hat{\sigma}_{t,\beta,K}^{\text{MAP}} - \sigma_{k_t^*}| = \left| \sum_{r \neq k_t^*} \alpha_{t,r}^{(\beta)} (\sigma_r - \sigma_{k_t^*}) \right| \leq (\sigma_{\max} - \sigma_{\min}) \sum_{r \neq k_t^*} \alpha_{t,r}^{(\beta)},$$

which gives the stated bound. $\square$

Lemma 3 (Balanced coordinate code for LayerNorm). *Let $z \in \mathcal{K} \subset \mathbb{R}^{d_z}$ be a working coordinate vector on a compact domain. Choose a residual width $d \geq 2d_z + 2$ and a number R with $R^2 > 2 \sup_{z \in \mathcal{K}} \|z\|_2^2$. Define*

$$a(z) = \sqrt{R^2/2 - \|z\|_2^2}, \quad E(z) = [z^\top, -z^\top, a(z), -a(z), 0, \dots, 0]^\top \in \mathbb{R}^d.$$

Let $\epsilon_{\text{LN}} > 0$ be the fixed LayerNorm stabilizer. Then $E(z)$ has zero mean and fixed squared norm R^2 . With unit LayerNorm gain and zero bias,

$$\text{LN}(E(z)) = \frac{E(z)}{\sqrt{R^2/d + \epsilon_{\text{LN}}}}.$$

Every coordinate of z is therefore a fixed linear readout of $\text{LN}(E(z))$.

Proof. The entries of $E(z)$ sum to zero because every coordinate is paired with its negative:

$$\sum_i E(z)_i = \sum_j z_j - \sum_j z_j + a(z) - a(z) = 0.$$

Its squared norm is constant:

$$\|E(z)\|_2^2 = \|z\|_2^2 + \|-z\|_2^2 + a(z)^2 + (-a(z))^2 = 2\|z\|_2^2 + 2(R^2/2 - \|z\|_2^2) = R^2.$$

The logical coordinate z_j , $j \in \{1, \dots, d_z\}$, is recovered by the fixed linear row

$$z_j = \frac{1}{2\sqrt{R^2/d + \epsilon_{\text{LN}}}} (\text{LN}(E(z))_j - \text{LN}(E(z))_{d_z+j}).$$

Thus each logical update $z \mapsto z'$ can be implemented by the residual addition $E(z') - E(z)$, since

$$E(z) + (E(z') - E(z)) = E(z').$$

Hence the residual stream remains in the encoded form after every block. The only nonlinear part is the scalar $a(\cdot)$, which is a square root of a bounded quantity; Lemma 5 supplies this approximation. $\square$

Lemma 4 (ReLU rows transfer to GELU rows). *Take $z \in \mathcal{K}$. Let*

$$F(z) = A[Bz + b]_+ + c$$

be one ReLU feed-forward block on a compact set $\mathcal{K}$. Define the GELU block

$$F_\kappa(z) = \frac{A}{\kappa} \text{GELU}(\kappa(Bz + b)) + c.$$

Here $\text{GELU}(u) = u\Phi(u)$ is applied coordinatewise, and $\Phi(u) = \mathbb{P}(G \leq u)$ for $G \sim \mathcal{N}(0, 1)$ is the standard normal cumulative distribution function. Then $F_\kappa \rightarrow F$ as $\kappa \rightarrow \infty$ uniformly on $\mathcal{K}$. More concretely, for each output coordinate r ,

$$\sup_{z \in \mathcal{K}} |(F_\kappa(z) - F(z))_r| \leq \frac{C_G}{\kappa} \sum_j |A_{rj}|, \quad C_G := \sup_{y \geq 0} y(1 - \Phi(y)) < \infty.$$

Thus any finite ReLU construction in this section has a GELU parameterization with arbitrarily small uniform error, obtained by multiplying the first-layer row and bias by κ and dividing the corresponding output column by κ .

Proof. Set

$$\phi_\kappa(u) := \kappa^{-1} \text{GELU}(\kappa u) = u\Phi(\kappa u).$$

For any $u \in \mathbb{R}$, using $\Phi(-y) = 1 - \Phi(y)$,

$$|\phi_\kappa(u) - [u]_+| = |u|(1 - \Phi(\kappa|u|)) = \frac{1}{\kappa}(\kappa|u|)(1 - \Phi(\kappa|u|)) \leq \frac{C_G}{\kappa}.$$

Applying this to $u_j(z) = (Bz + b)_j$, for output coordinate r ,

$$|(F_\kappa(z) - F(z))_r| \leq \sum_j |A_{rj}| |\phi_\kappa(u_j(z)) - [u_j(z)]_+| \leq \frac{C_G}{\kappa} \sum_j |A_{rj}|.$$

The bound is independent of z , so taking $\sup_{z \in \mathcal{K}}$ gives the uniform bound, which vanishes as $\kappa \rightarrow \infty$. $\square$

Lemma 5 (Finite scalar arithmetic). *On clipped compact domains, ReLU feed-forward blocks uniformly approximate the scalar operations used in the construction: square, product, reciprocal, division, square root, logarithm, exponential, clipping, soft thresholds, and finite softmax normalization. If $f \in C^2([a, b])$ and*

$$\Delta := \max_k (u_{k+1} - u_k)$$

is the largest spacing of the interpolation grid $a = u_0 < \dots < u_K = b$, then the piecewise-linear interpolant satisfies

$$\|f - I_\Delta f\|_\infty \leq \frac{\Delta^2}{8} \|f''\|_\infty.$$

Proof. ReLU blocks exactly implement affine maps and positive parts $[u]_+$. Hence they implement piecewise-linear primitives such as

$$|u| = [u]_+ + [-u]_+, \quad \max\{a, b\} = b + [a - b]_+,$$

and therefore clipping by writing

$$\text{clip}_{[L, U]}(u) = L + [u - L]_+ - [u - U]_+.$$

Clipping is used because the approximation is uniform only on compact sets. It also keeps singular operations well-defined, for example by ensuring denominators stay bounded away from zero before applying $1/u$, $\log u$, or $\sqrt{u}$. For a scalar C^2 function f on $[a, b]$, define slopes

$$m_k = \frac{f(u_{k+1}) - f(u_k)}{u_{k+1} - u_k}.$$

Its piecewise-linear interpolant has the ReLU representation

$$I_\Delta f(u) = f(u_0) + m_0(u - u_0) + \sum_{k=1}^{K-1} (m_k - m_{k-1})[u - u_k]_+.$$

Thus one sufficiently wide ReLU block implements $I_\Delta f$ exactly, and the interpolation error obeys

$$\|f - I_\Delta f\|_\infty \leq \frac{\Delta^2}{8} \|f''\|_\infty.$$

For example, if $f(u) = e^u$ on $[a, b]$, then $f''(u) = e^u$, so

$$\|e^u - I_\Delta e^u\|_\infty \leq \frac{\Delta^2}{8} e^b.$$

Similarly, for $f(u) = e^{\beta u}$ on $[-M, M]$,

$$\|e^{\beta u} - I_\Delta e^{\beta u}\|_\infty \leq \frac{\Delta^2}{8} \beta^2 e^{\beta M}.$$

This covers u^2 , $1/u$, $\sqrt{u}$, $\log u$, and e^u on their clipped nonsingular domains. Products follow from

$$xy = \frac{(x+y)^2 - x^2 - y^2}{2},$$

and division follows from $x/y = x \cdot (1/y)$. A finite softmax is obtained by approximating each $e^{\beta \ell_k}$, summing them, approximating the reciprocal of the denominator, and multiplying. Since the number of operations is finite and all variables are clipped to compact domains, the local errors can be chosen small enough to give any desired final uniform error. $\square$

Proof of Theorem 1. Write $z_{t,c}$ for coordinate c of z_t . By Lemma 3, it is enough to specify the logical-coordinate assignments; after each assignment the physical block re-encodes the state as $E(z)$. The input and position embeddings initialize

$$z_{t,u} \leftarrow u_t, \quad z_{t,1} \leftarrow 1, \quad z_{t,X} \leftarrow X_t, \quad z_{t,c} \leftarrow 0 \quad \text{for the remaining construction coordinates.}$$

Here u_t stores the position-dependent constants, including t , η_t , and the fixed grid constants. The scalar-table construction from Lemma 5, applied with input row $r_X^\top$, output coordinate X^2 , and $f(u) = u^2$, gives

$$z_{t,X^2} \leftarrow z_{t,X}^2.$$

Next use one zero-query/zero-key causal attention head. Choose two standard column basis vectors v_1, v_2 in the head's value space. These are not logical residual coordinates; they are temporary basis directions inside the attention head. The value matrix writes X_s into the v_1 -coordinate and X_s^2

into the v_2 -coordinate. The v_1 -column of W_O is $e_{\bar{X}}$, and the v_2 -column is $e_{\bar{X}^2}$, so W_O sends the averaged v_1 -coordinate to $\bar{X}_t$ and the averaged v_2 -coordinate to $\bar{X}^2_t$:

$$W_Q = 0, \quad W_K = 0, \quad W_V = v_1 r_X^\top + v_2 r_{X^2}^\top, \quad W_O = e_{\bar{X}} v_1^\top + e_{\bar{X}^2} v_2^\top.$$

For every source position $s \leq t$,

$$q_t = W_Q \text{LN}(h_t) = 0, \quad k_s = W_K \text{LN}(h_s) = 0,$$

so every causal attention score is zero and

$$\alpha_{ts} = \frac{\exp(0)}{\sum_{r \leq t} \exp(0)} = \frac{1}{t}.$$

The attention value vector at s is

$$\begin{aligned} \nu_s &= W_V \text{LN}(h_s) \\ &= v_1 r_X^\top \text{LN}(h_s) + v_2 r_{X^2}^\top \text{LN}(h_s) \\ &= v_1 z_{s,X} + v_2 z_{s,X^2}. \end{aligned}$$

Thus the attention average is

$$\sum_{s \leq t} \alpha_{ts} \nu_s = v_1 \frac{1}{t} \sum_{s \leq t} z_{s,X} + v_2 \frac{1}{t} \sum_{s \leq t} z_{s,X^2}.$$

Finally, since $v_i^\top v_j = \mathbf{1}\{i = j\}$, the output projection gives

$$W_O \sum_{s \leq t} \alpha_{ts} \nu_s = e_{\bar{X}} \frac{1}{t} \sum_{s \leq t} z_{s,X} + e_{\bar{X}^2} \frac{1}{t} \sum_{s \leq t} z_{s,X^2}.$$

Therefore the head writes

$$z_{t,\bar{X}} \leftarrow \frac{1}{t} \sum_{s \leq t} z_{s,X}, \quad z_{t,\bar{X}^2} \leftarrow \frac{1}{t} \sum_{s \leq t} z_{s,X^2}.$$

All remaining statistical coordinates are scalar functions of already-written coordinates:

$$\begin{aligned} z_{t,S} &\leftarrow t z_{t,\bar{X}}, \\ z_{t,Q} &\leftarrow t z_{t,\bar{X}^2}, \\ z_{t,\hat{\sigma}^{\text{MLE}}} &\leftarrow F_{\text{MLE}}(z_{t,S}, z_{t,Q}, t), \\ z_{t,\ell_k} &\leftarrow \log w_k + \log p(X_{1:t} \mid \sigma_k), \quad k = 1, \dots, K, \\ z_{t,\alpha_k} &\leftarrow \frac{\exp(\beta z_{t,\ell_k})}{\sum_{r=1}^K \exp(\beta z_{t,\ell_r})}, \quad k = 1, \dots, K, \\ z_{t,\hat{\sigma}^{\text{MAP}}} &\leftarrow \sum_{k=1}^K \sigma_k z_{t,\alpha_k}. \end{aligned}$$

Here the running-scale map is

$$F_{\text{MLE}}(S, Q, t) := \begin{cases} \sigma_{\text{init}}, & t = 1, \\ \sqrt{\frac{Q - S^2/t}{t-1}}, & t \geq 2, \end{cases}$$

since $Q - S^2/t = \sum_{s \leq t} (X_s - \bar{X}_t)^2$. In the ℓ_k update, $\log p(X_{1:t} \mid \sigma_k)$ means equation (1) evaluated with $S_t = z_{t,S}$, $Q_t = z_{t,Q}$, and $\sigma = \sigma_k$. The additive $\text{const}(t)$ may be dropped because it is the same for every k and therefore cancels in the softmax over k . Each line is a finite scalar composition on the clipped domain, so Lemma 5 implements it coordinate by coordinate to any prescribed tolerance.

It remains to form the plug-in threshold and margin. Add the coordinate updates

$$\begin{aligned} z_{t,\hat{\sigma}} &\leftarrow \begin{cases} z_{t,\hat{\sigma}^{\text{MLE}}}, & \text{MLE plug-in,} \\ z_{t,\hat{\sigma}^{\text{MAP}}}, & \text{MAP plug-in,} \end{cases} \\ z_{t,\theta} &\leftarrow z_{t,\bar{X}} + z_{t,\hat{\sigma}} z_{t,\eta}, \\ z_{t,m} &\leftarrow z_{t,X} - z_{t,\theta}. \end{aligned}$$

The scalar product in the threshold line is again supplied by Lemma 5. The output heads are literal linear rows,

$$w_{\text{cv}}^\top = r_\theta^\top, \quad w_{\text{act}}^\top = r_m^\top \quad (\text{equivalently } r_X^\top - r_\theta^\top \text{ if } m \text{ is not stored}).$$

Lemma 3 makes the construction compatible with pre-norm LayerNorm, and Lemma 5 supplies the ReLU feed-forward arithmetic. Because only finitely many scalar operations are composed on a compact domain, choose their local approximation tolerances so that the final threshold error is at most ϵ . The threshold decision is unchanged whenever the true margin has magnitude larger than ϵ . $\square$

F Training

F.1 Base-Model Hyperparameters

Hyperparameter	Value
Component	GPTSTOPPER, scale-blind GPT-2-style causal decoder
Horizon / context length	$n = 256$ observations
Input representation	scalar X_t passed through Linear(1, 128) plus learned absolute position embedding
Trainable module	full causal decoder plus continuation-value and action heads
Layers / snapshots	$L = 8$ pre-norm blocks; $L + 1 = 9$ hidden snapshots are cached for probing
Hidden dimension	$d_{\text{emb}} = 128$
Attention	$M = 4$ heads per layer; head dimension 32; joint $qkv : \mathbb{R}^{128} \rightarrow \mathbb{R}^{384}$, output $\mathbb{R}^{128} \rightarrow \mathbb{R}^{128}$
Feed-forward / readout	block MLP Linear(128, 512) $\rightarrow$ GELU $\rightarrow$ Linear(512, 128); scalar output heads
Output / target	$\hat{C}_t$ in raw oracle units or action logit for accept/continue
Loss	continuation-value: σ_i^{-2} -normalized MSE; action: BCE; supervised for $t = 1, \dots, n - 1$
Optimizer	Adam, no weight decay
Learning rate	peak 10^{-4} , cosine schedule with 20% linear warmup
Batch size	64 sequences
Training budget	static: 2×10^5 CV steps, 1×10^5 action steps; random variance: 3×10^5 CV steps, 1.5×10^5 action steps
Validation cadence	validation evaluated on the fixed validation cache during training
Checkpoint selection	minimum validation loss
Train split / budget	fresh sequences sampled every step; no fixed base-training set
Validation / test split	$N_{\text{val}} = N_{\text{test}} = 10^4$ fixed sequences, with disjoint seeds
Seed	independent random initialization per trained model or ensemble member
Parameter count	1,619,714 parameters ($\approx 1.62\text{M}$)

Table 2: Base-model hyperparameters used in all GPTSTOPPER runs.

Figure 36 shows train and validation loss for all ten runs over their training horizon, with the final checkpoint selected by minimum validation loss marked.

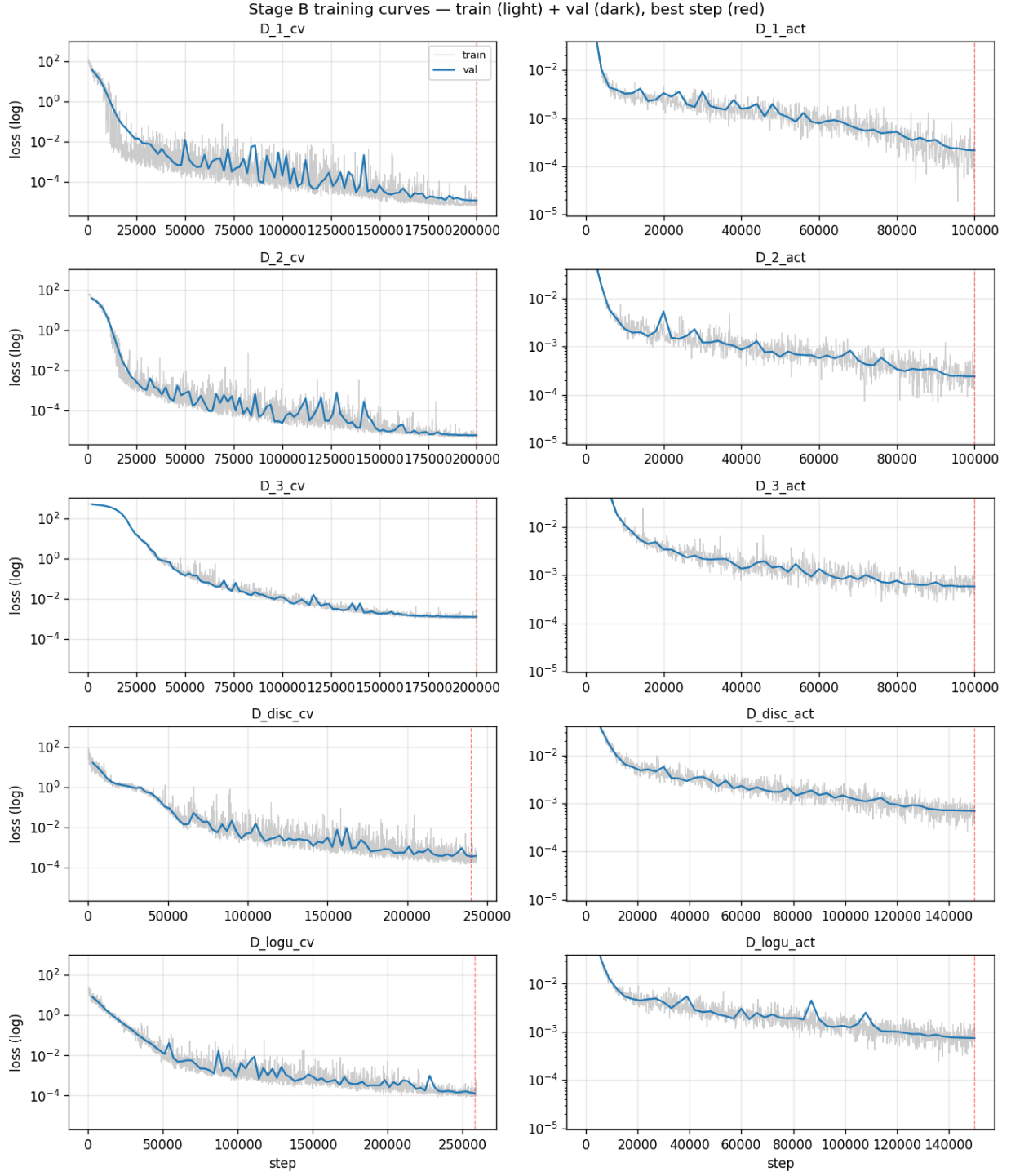


Figure 36: Train (light) and validation (dark) loss for all ten trained runs, with the best-validation-loss checkpoint step marked (red dashed).

For the four random-variance runs we additionally report validation loss separated by σ -bin (Figure 37). On the σ_i^2 -normalized continuation-value runs ($\mathcal{D}_{\text{disc-cv}}$ and $\mathcal{D}_{\text{logu-cv}}$), the per- σ

losses are within an order of magnitude of each other for most of training and within ~ 0.05 of each other by the end of the 3×10^5 -step budget, matching the scale balancing targeted by the normalization choice (Section 3.2). The same balance is reported for the action runs.

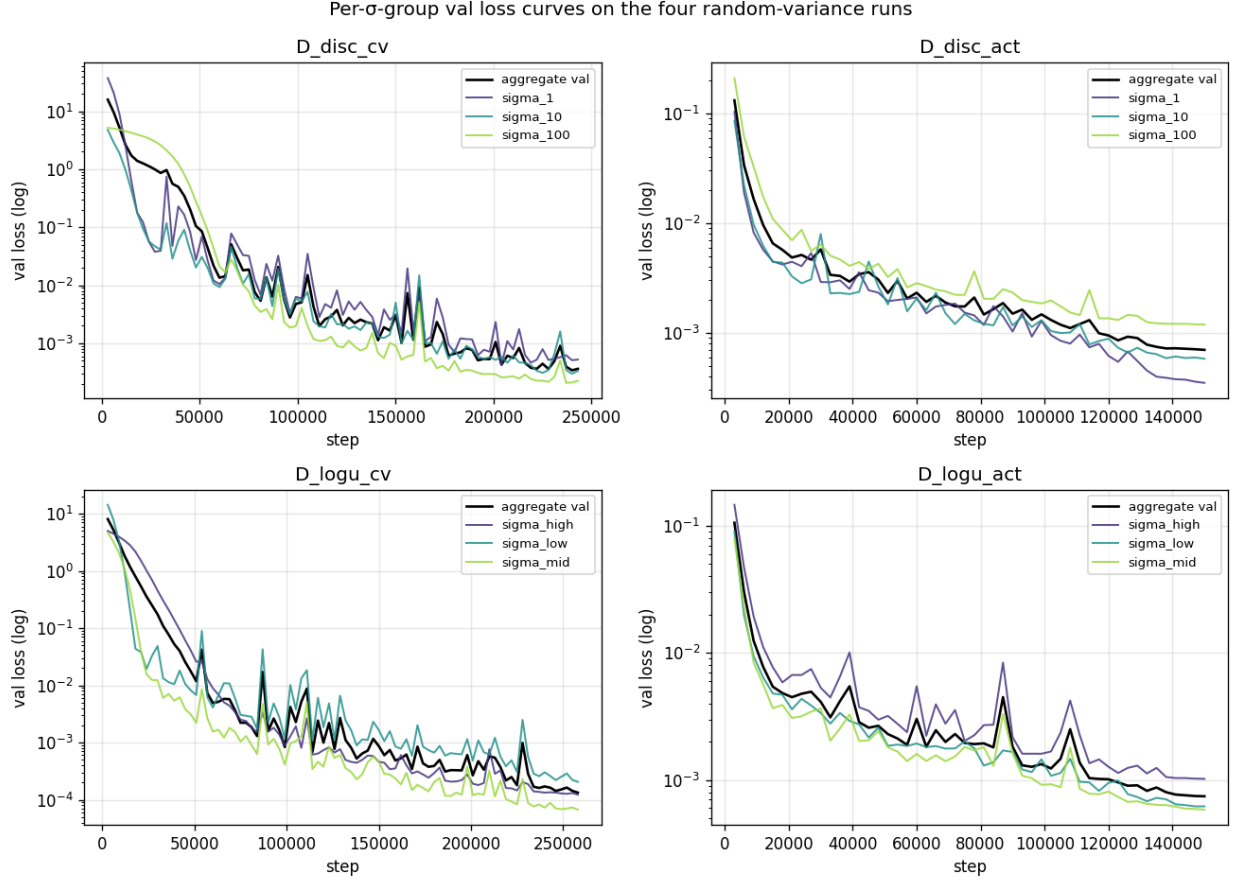


Figure 37: Per- σ -bin validation loss for the four random-variance training runs ($\mathcal{D}_{\text{disc_cv}}$, $\mathcal{D}_{\text{disc_act}}$, $\mathcal{D}_{\text{logu_cv}}$, $\mathcal{D}_{\text{logu_act}}$). Black: aggregate; coloured: per- σ slice.

F.2 Probe Hyperparameters

Hyperparameter	Value
Component	timestep-shared simple linear probe and timestep-shared attention-pooled probe
Horizon / context length	$n = 256$ cached prefix positions
Input representation	cached hidden states: $h_t^{(\ell)}$ for simple probes or $H_{1:t}^{(\ell)}$ for attention-pooled probes
Trainable module	separate probe for each ensemble member, base representation, target, layer, sequence budget, and probe family
Layers / snapshots	9 cached snapshots, $\ell = 0, \dots, 8$; $\ell = 0$ is post-embedding
Hidden dimension	$d = 128$ hidden-state coordinates
Attention	simple probe: none; attention-pooled probe: causal softmax over prefix positions with learned position scores
Feed-forward / readout	simple: $w^\top h_t^{(\ell)} + b$; attention-pooled: linear hidden projection plus linear scalar readout
Output / target	scalar probe prediction for $\sqrt{\hat{\sigma}_t^2}$ or $\hat{C}_t^*$ in the preliminary run
Loss	σ_i -normalized MSE, sMSE, on valid prefix–target pairs
Optimizer	Adam
Learning rate	10^{-3}
Batch size	256 prefix–target pairs
Training budget	1 epoch, deliberately under-trained
Validation cadence	8 evenly spaced validation passes per epoch
Checkpoint selection	minimum validation sMSE
Train split / budget	$M_{\text{train}} \in \{64, 128, 256, 512, 1024\}$ sequences; full preliminary statistics use $M_{\text{train}} = 1024$
Validation / test split	$M_{\text{val}} = 512$, $M_{\text{test}} = 10,000$ sequences
Seed	ensemble index +1000 $\ell + M_{\text{train}}$, recomputed inside each cell
Parameter count	simple: 129; attention-pooled: 16,769 for $d = 128, n = 256$

Table 3: Probe-training hyperparameters. A separate probe is trained for each ensemble member, base representation, target, layer, sequence budget, and probe family.

G Approximate Dynamic Program Oracle

This appendix defines the numerical oracle used for labels and for the per-regime Bayes-optimal payoff R^* . The idea is standard dynamic programming. At any prefix, the past observations are compressed into a belief state. The oracle asks: if we reject the current observation, what is the expected value of acting optimally from the next step onward? That number is the continuation threshold $\hat{C}_t$. We compute it by backward induction, using Bayesian predictive distributions, quadrature for expectations, and interpolation for off-grid states.

There are two cases. With known σ , the belief is one-dimensional: the posterior mean of μ . With unknown σ , the belief is two-dimensional: the running sum S_t and running square-sum Q_t , represented on a mean–scale grid.

G.1 Static Variance

State. In the static-variance case, σ is known and only μ is latent. The prefix matters only through the running sum $S_t := \sum_{s=1}^t X_s$, or equivalently the sample mean $\bar{X}_t = S_t/t$. The posterior is

$$\mu \mid X_{1:t} \sim \mathcal{N}(\mu_t, \tau_t^2), \quad \tau_t^2 = \left(\frac{1}{\tau_0^2} + \frac{t}{\sigma^2} \right)^{-1}, \quad \mu_t = \tau_t^2 \left(\frac{\mu_0}{\tau_0^2} + \frac{S_t}{\sigma^2} \right). \quad (5)$$

Thus the ADP state can be the single scalar $b_t = \mu_t$. The posterior variance τ_t^2 is deterministic once t is known. The posterior mean can also be written as

$$\mu_t = \frac{t\tau_0^2}{\sigma^2 + t\tau_0^2} \bar{X}_t + \frac{\sigma^2}{\sigma^2 + t\tau_0^2} \mu_0.$$

So μ_t is a weighted average of the data mean and the prior mean. With $\rho := \sigma^2/\tau_0^2$, the data weight is

$$w_t = \frac{t\tau_0^2}{\sigma^2 + t\tau_0^2} = \frac{1}{1 + \rho/t},$$

and at $t = 1$ this becomes the $1/(1 + \rho)$ value used in Section 2.

Equation (5) follows by multiplying the Gaussian prior and Gaussian likelihood:

$$p(\mu \mid X_{1:t}) \propto \exp \left[-\frac{(\mu - \mu_0)^2}{2\tau_0^2} - \frac{1}{2\sigma^2} \sum_{s \leq t} (X_s - \mu)^2 \right].$$

The exponent is quadratic in μ . Completing the square gives variance $(\tau_0^{-2} + t\sigma^{-2})^{-1}$ and mean $\tau_t^2(\mu_0/\tau_0^2 + S_t/\sigma^2)$.

Backward recursion. At decision time $t < n$, stopping gives X_t . Continuing gives the expected value of the next decision. Therefore

$$V_t(\mu_t, X_t) = \max\{X_t, C_t(\mu_t)\},$$

where $C_t(\mu_t)$ is the value of rejecting X_t . If the agent continues, the next observation has predictive distribution

$$X_{t+1} \mid X_{1:t} \sim \mathcal{N}(\mu_t, \tau_t^2 + \sigma^2).$$

Hence

$$C_t(\mu_t) = \mathbb{E}_{X \sim \mathcal{N}(\mu_t, \tau_t^2 + \sigma^2)} \left[\max\{X, C_{t+1}(\text{update}(\mu_t, X))\} \right], \quad (6)$$

with terminal condition $C_{n-1}(\mu) = \mu$, since the next step is forced stopping. The Bayes-optimal action is

$$\pi_{\mathcal{D}_{i,t}}^*(\mu_t, X_t) = \mathbf{1}[X_t \geq C_t(\mu_t)].$$

After drawing a possible next observation X , the posterior mean updates by the same Normal–Normal formula:

$$\text{update}(\mu_t, X) = \alpha_t \mu_t + (1 - \alpha_t) X, \quad \alpha_t := \frac{\sigma^2}{\tau_t^2 + \sigma^2}.$$

Numerical table. The recursion has no closed form, so we store it on a grid. At each t , we discretize μ_t using K uniform points over

$$[\mu_0 - 5\varsigma_t, \mu_0 + 5\varsigma_t], \quad \varsigma_t := \sqrt{\tau_0^2 - \tau_t^2}.$$

Here ς_t is the marginal standard deviation of the posterior mean across sequences. The expectation in (6) is one-dimensional Gaussian, so we approximate it with J -point Gauss–Hermite quadrature:

$$\int_{-\infty}^{\infty} f(z) e^{-z^2} dz \approx \sum_{j=1}^J w_j f(z_j)$$

with precomputed nodes and weights. For $X \sim \mathcal{N}(\mu, v_t)$, $v_t := \tau_t^2 + \sigma^2$, this gives

$$\mathbb{E}_{X \sim \mathcal{N}(\mu, v_t)}[f(X)] \approx \frac{1}{\sqrt{\pi}} \sum_{j=1}^J w_j f(\mu + \sqrt{2v_t} z_j). \quad (7)$$

The $1/\sqrt{\pi}$ factor comes from the change of variables $z = (X - \mu)/\sqrt{2v_t}$. When the updated state falls between grid points, we use linear interpolation, clipped at the grid boundary. Algorithm 1 uses $K = 2048$ and $J = 128$.

Algorithm 1 Numerical DP for the static-variance Bayes-optimal threshold (per regime).

```

1: Precompute  $\tau_t^2$ ,  $v_t := \tau_t^2 + \sigma^2$ , and  $\alpha_t$  for each  $t$ 
2: Build the stage- $t$  grid  $\{m_t^{(k)}\}_{k=1}^K$  and Gauss–Hermite nodes  $\{(z_j, w_j)\}_{j=1}^J$ 
3:  $\widehat{C}_{n-1}(m_{n-1}^{(k)}) \leftarrow m_{n-1}^{(k)}$  for  $k = 1, \dots, K$ 
4: for  $t = n - 2, n - 3, \dots, 1$  do
5:   for  $k = 1, \dots, K$  do
6:      $A \leftarrow 0$ 
7:     for  $j = 1, \dots, J$  do
8:        $x_j \leftarrow m_t^{(k)} + \sqrt{2v_t} z_j$ 
9:        $\mu'_j \leftarrow \alpha_t m_t^{(k)} + (1 - \alpha_t) x_j$ 
10:       $A \leftarrow A + w_j \max\{x_j, \widehat{C}_{t+1}^{\text{lin}}(\mu'_j)\}$ 
11:    end for
12:     $\widehat{C}_t(m_t^{(k)}) \leftarrow A/\sqrt{\pi}$ 
13:  end for
14: end for
15: return  $\{\widehat{C}_t\}$ 

```

G.2 Random Variance

Lemma 6 (Sufficient statistics for the random-variance posterior). *For the joint posterior on (μ, σ) , the prefix $X_{1:t}$ enters the likelihood only through*

$$S_t := \sum_{s=1}^t X_s, \quad Q_t := \sum_{s=1}^t X_s^2. \quad (8)$$

Together with t , the pair (S_t, Q_t) determines $p(\mu, \sigma \mid X_{1:t})$. It also gives $\bar{X}_t = S_t/t$ and the within-sequence sample variance

$$\hat{\sigma}_t^2 := \frac{1}{t-1}(Q_t - t\bar{X}_t^2).$$

Both statistics update online: $S_t = S_{t-1} + X_t$ and $Q_t = Q_{t-1} + X_t^2$.

Proof of Lemma 6. The Gaussian log-likelihood expands as

$$\log p(X_{1:t} \mid \mu, \sigma) = -\frac{t}{2} \log(2\pi\sigma^2) - \frac{1}{2\sigma^2} \sum_{s \leq t} (X_s - \mu)^2 = -\frac{t}{2} \log(2\pi\sigma^2) - \frac{Q_t - 2\mu S_t + t\mu^2}{2\sigma^2},$$

so the likelihood depends on the prefix only through S_t and Q_t . The sample-variance identity follows by setting $\mu = \bar{X}_t$:

$$\sum_{s \leq t} (X_s - \bar{X}_t)^2 = Q_t - t\bar{X}_t^2.$$

Dividing by $t - 1$ gives $\hat{\sigma}_t^2$. □

Posterior over μ and σ . The random-variance oracle must keep uncertainty about both μ and σ . Bayes' rule gives

$$p(\mu, \sigma \mid X_{1:t}) \propto p(\sigma) p(\mu) p(X_{1:t} \mid \mu, \sigma),$$

or equivalently

$$p(\mu, \sigma \mid X_{1:t}) = p(\sigma \mid X_{1:t}) p(\mu \mid X_{1:t}, \sigma).$$

For a fixed candidate σ , the conditional posterior of μ is exactly the Normal–Normal posterior in (5). The only new object is therefore $p(\sigma \mid X_{1:t})$. It is proportional to the prior $p(\sigma)$ times the marginal likelihood $p(X_{1:t} \mid \sigma)$. For $\mathcal{D}_{\text{disc}}$, this posterior is a distribution on $\{1, 10, 100\}$. For $\mathcal{D}_{\text{logu}}$, it is a density on $[1, 100]$, evaluated by quadrature.

Derivation of equation (1). Fix σ . Since μ is Gaussian and $X_s \mid \mu$ is Gaussian, the marginal vector $X_{1:t} \mid \sigma$ is Gaussian. Its mean is $\mu_0 \mathbf{1} = \mathbf{0}$. Its covariance is

$$\text{Cov}(X_r, X_s) = \mathbb{E}[\text{Cov}(X_r, X_s \mid \mu)] + \text{Cov}(\mathbb{E}[X_r \mid \mu], \mathbb{E}[X_s \mid \mu]) = \sigma^2 \delta_{rs} + \tau_0^2,$$

so

$$X_{1:t} \mid \sigma \sim \mathcal{N}(\mathbf{0}, \Sigma_t^{(\sigma)}), \quad \Sigma_t^{(\sigma)} = \sigma^2 I_t + \tau_0^2 \mathbf{1}\mathbf{1}^\top.$$

The Gaussian log-density is

$$\log p(X_{1:t} \mid \sigma) = -\frac{t}{2} \log(2\pi) - \frac{1}{2} \log \det \Sigma_t^{(\sigma)} - \frac{1}{2} X_{1:t}^\top (\Sigma_t^{(\sigma)})^{-1} X_{1:t}.$$

Because $\Sigma_t^{(\sigma)}$ is a rank-one update of $\sigma^2 I_t$,

$$\log \det \Sigma_t^{(\sigma)} = t \log \sigma^2 + \log \left(1 + \frac{t\tau_0^2}{\sigma^2} \right), \quad (\Sigma_t^{(\sigma)})^{-1} = \frac{1}{\sigma^2} I_t - \frac{\tau_0^2/\sigma^4}{1 + t\tau_0^2/\sigma^2} \mathbf{1}\mathbf{1}^\top.$$

Using $X_{1:t}^\top X_{1:t} = Q_t$ and $(\mathbf{1}^\top X_{1:t})^2 = S_t^2$, the quadratic term is

$$X_{1:t}^\top (\Sigma_t^{(\sigma)})^{-1} X_{1:t} = \frac{Q_t}{\sigma^2} - \frac{\tau_0^2 S_t^2 / \sigma^4}{1 + t\tau_0^2 / \sigma^2}.$$

Substituting these pieces gives (1), with the additive $-\frac{t}{2} \log(2\pi)$ absorbed into $\text{const}(t)$. The data enter only through S_t and Q_t . □

Predictive distribution and recursion. For each candidate σ , the next observation has a Gaussian predictive distribution with mean and variance from the conditional posterior of μ . Since σ itself is uncertain, the full predictive is a mixture:

$$p(X_{t+1} | X_{1:t}) = \int p(\sigma | X_{1:t}) \cdot \mathcal{N}(X_{t+1}; \mu_t^{(\sigma)}, v_t^{(\sigma)}) d\sigma, \quad v_t^{(\sigma)} := \tau_t^{2,(\sigma)} + \sigma^2, \quad (9)$$

with the integral replaced by a three-term sum for $\mathcal{D}_{\text{disc}}$. The belief state is $b_t = (S_t, Q_t)$. Backward induction gives

$$C_t(b_t) = \mathbb{E}_{X_{t+1} \sim p(\cdot | b_t)}[\max\{X_{t+1}, C_{t+1}(b_{t+1})\}], \quad (10)$$

where $b_{t+1} = (S_t + X_{t+1}, Q_t + X_{t+1}^2)$. The terminal value is the predictive mean,

$$C_{n-1}(b_{n-1}) = \int p(\sigma | X_{1:n-1}) \mu_{n-1}^{(\sigma)} d\sigma.$$

The optimal action is again thresholding:

$$\pi_{\mathcal{D},t}^*(b_t, X_t) = \mathbf{1}[X_t \geq C_t(b_t)].$$

Table coordinates. The true belief state is $b_t = (S_t, Q_t)$. For table lookup, we store the same information in more convenient coordinates: a mean coordinate and a scale coordinate,

$$q_t = (\tilde{X}_t, \lambda_t).$$

The tilde means the value is projected onto the table range; Π_I denotes projection onto an interval I . The two axes are:

- Mean axis:

$$\tilde{X}_t = \Pi_{I_t^{\bar{x}}}(\bar{X}_t), \quad I_t^{\bar{x}} = [-5s_t^{\bar{x}}, +5s_t^{\bar{x}}], \quad s_t^{\bar{x}} := \sqrt{\tau_0^2 + \sigma_{\max}^2/t}.$$

The interval is wider at early times because $\bar{X}_t$ is noisy when t is small. With $\sigma_{\max} = 100$, the $t = 1$ grid spans roughly $[-503, 503]$; for large t , it approaches $[-50, 50]$.

- Scale axis:

$$\lambda_t = \Pi_{I_\lambda}(\lambda_t^{\text{raw}}), \quad I_\lambda := \left[\frac{1}{2} \log 0.1, \frac{1}{2} \log 10^5 \right].$$

For $t \geq 2$, use $\lambda_t^{\text{raw}} = \frac{1}{2} \log \hat{\sigma}_{t,\text{floored}}^2$, where $\hat{\sigma}_{t,\text{floored}}^2 = \max\{\hat{\sigma}_t^2, e^{2\lambda_{\min}}\}$. At $t = 1$, set $\lambda_1 = \lambda_{\text{init}}$. This first-step scale coordinate is only a table convention, since $Q_1 = S_1^2$.

We use $K_1 = K_2 = 256$ uniform grid points, so each stage has 65,536 table states. At a grid point $(\bar{x}, \lambda)$, recover the sufficient statistics by

$$S_t = t\bar{x}, \quad Q_t = (t-1)e^{2\lambda} + t\bar{x}^2.$$

For an off-grid belief, compute q_t and use bilinear interpolation.

Mixture quadrature. The expectation in (10) has two layers: uncertainty over σ , then Gaussian uncertainty over X_{t+1} conditional on σ . We approximate the σ -integral by M components:

$$M = 3 \quad \text{for } \mathcal{D}_{\text{disc}}, \quad M = J_\sigma = 64 \quad \text{for } \mathcal{D}_{\text{logu}}.$$

For $\mathcal{D}_{\text{logu}}$, the components are Gauss–Legendre nodes on $\log \sigma \in [0, \log 100]$. For each component σ_k , we apply $J = 64$ -point Gauss–Hermite quadrature for X_{t+1} . The component weight is

$$\tilde{w}_k = \frac{p(\sigma_k | X_{1:t}) \omega_k}{Z_t},$$

where ω_k is the quadrature weight and Z_t normalizes the weights to sum to one. In the discrete case, $\omega_k = 1$.

Algorithm 2 Numerical DP for the random-variance Bayes-optimal threshold.

```

1: Build per-stage  $\bar{x}$ -grids  $\{\bar{x}_t^{(k_1)}\}$  on  $I_t^{\bar{x}} = [-5s_t^{\bar{x}}, +5s_t^{\bar{x}}]$ 
2: Build one stage-invariant  $\lambda$  grid  $\{\lambda^{(k_2)}\}$  on  $I_\lambda$ 
3: for each terminal grid point  $(k_1, k_2)$  at  $t = n - 1$  do
4:   Recover  $(S_{n-1}, Q_{n-1})$  from  $(\bar{x}_{n-1}^{(k_1)}, \lambda^{(k_2)})$ 
5:   Compute  $p(\sigma | X_{1:n-1})$  using (1)
6:    $\hat{C}_{n-1}(\bar{x}_{n-1}^{(k_1)}, \lambda^{(k_2)}) \leftarrow \int p(\sigma | X_{1:n-1}) \mu_{n-1}^{(\sigma)} d\sigma$ 
7: end for
8: for  $t = n - 2, n - 3, \dots, 1$  do
9:   for each grid point  $(k_1, k_2)$  on stage  $t$  do
10:    Recover  $(S_t, Q_t)$  from  $(t, \bar{X}_t^{(k_1)}, \lambda^{(k_2)})$ 
11:    Compute mixture weights  $\{\tilde{w}_k\}_{k=1}^M$  from  $p(\sigma | X_{1:t})$ 
12:    Compute component means/variances  $\{(\mu_t^{(\sigma_k)}, v_t^{(\sigma_k)})\}_{k=1}^M$  via (5)
13:     $A \leftarrow 0$ 
14:    for  $k = 1, \dots, M, j = 1, \dots, J$  do
15:       $x_{kj} \leftarrow \mu_t^{(\sigma_k)} + \sqrt{2v_t^{(\sigma_k)}} z_j$ 
16:       $S_{t+1} \leftarrow S_t + x_{kj}; \quad Q_{t+1} \leftarrow Q_t + x_{kj}^2$ 
17:      Convert  $(S_{t+1}, Q_{t+1})$  to  $(\bar{X}_{t+1}, \lambda_{t+1})$ 
18:      Look up  $\hat{C}_{t+1}^{\text{bilin}}$  by bilinear interpolation, clipped at the grid boundary
19:       $A \leftarrow A + \tilde{w}_k \cdot (w_j / \sqrt{\pi}) \cdot \max\{x_{kj}, \hat{C}_{t+1}^{\text{bilin}}\}$ 
20:    end for
21:     $\hat{C}_t(\bar{X}_t^{(k_1)}, \lambda^{(k_2)}) \leftarrow A$ 
22:   end for
23: end for
24: return  $\{\hat{C}_t\}$ 

```

G.3 Convergence Gate

The ADP tables are numerical, so we check that the production resolution is stable. For each oracle we solve the table twice:

$$\text{production resolution} \quad \text{and} \quad \text{reference resolution} = 2 \times \text{production resolution}.$$

We then compare the two induced labelers on $N_{\text{check}} = 10^4$ fresh test sequences per regime. We report two quantities:

- *Max threshold difference:*

$$\max_{t,m} \left| \widehat{C}_t^{\text{prod}}(m) - \widehat{C}_t^{\text{ref}}(m) \right|.$$

This is a magnitude diagnostic, not the pass/fail criterion.

- *Action-label disagreement:* the fraction of decision steps (seq, t), $t < n$, where the production and reference tables choose different actions. This is the pass/fail criterion; the required rate is $< 10^{-3}$.

Action-label disagreement is the right operational metric because the training labels depend only on which side of the threshold X_t lies. A small threshold shift does not matter if it does not change the action. Absolute threshold differences are still useful diagnostics, but they naturally grow with σ because the thresholds themselves have scale $\sim \sigma$.

G.4 Static-ADP Convergence Check

For the static oracle, production uses $(K, J) = (2048, 128)$ and reference uses $(4096, 256)$. Table 4 checks all seven evaluation σ values. Every row passes comfortably: action disagreement is about 3×10^{-5} , far below the 10^{-3} gate.

σ	regime	$\max \Delta \widehat{C} $	action-label disagree	gate
0.3	$\mathcal{D}_{\text{ood},0.3}$ (OOD)	2.5×10^{-2}	3.1×10^{-5}	PASS
1	$\mathcal{D}_1$	3.3×10^{-2}	3.8×10^{-5}	PASS
3	$\mathcal{D}_{\text{ood},3}$ (OOD)	3.5×10^{-2}	3.6×10^{-5}	PASS
10	$\mathcal{D}_2$	3.9×10^{-2}	3.6×10^{-5}	PASS
30	$\mathcal{D}_{\text{ood},30}$ (OOD)	8.6×10^{-2}	3.1×10^{-5}	PASS
100	$\mathcal{D}_3$	2.4×10^{-1}	2.9×10^{-5}	PASS
300	$\mathcal{D}_{\text{ood},300}$ (OOD)	6.3×10^{-1}	3.0×10^{-5}	PASS

Table 4: Static-ADP convergence diagnostic across the seven evaluation σ values. Production $(K, J) = (2048, 128)$ vs. reference $(4096, 256)$. Gate: action-label disagreement $< 10^{-3}$.

G.5 Random-ADP Convergence Check

For the random-variance oracle, production uses

$$(K_1, K_2, J[, J_\sigma]) = (256, 256, 64[, 64]),$$

and reference uses

$$(512, 512, 128[, 128]).$$

The J_σ axis appears only for $\mathcal{D}_{\text{logu}}$, where σ is continuous. We evaluate both random-prior tables on the three static regimes $\mathcal{D}_1, \mathcal{D}_2, \mathcal{D}_3$, giving six cells.

Magnitude diagnostic. The max-threshold column can look large because it is an absolute error, and thresholds scale with σ . For random-variance tables we therefore also report the median relative threshold difference, $\text{median}|\Delta \widehat{C}|/\sigma_{\text{regime}}$, and use action disagreement as the gate.

training	test regime	max $ \Delta\hat{C} $	median $ \Delta\hat{C} $	median $\frac{ \Delta\hat{C} }{\sigma_{\text{regime}}}$	action-label disagree	gate
$\mathcal{D}_{\text{disc}}$	$\mathcal{D}_1$ ($\sigma = 1$)	4.13×10^0	2.90×10^{-3}	2.90×10^{-3}	8.16×10^{-5}	PASS
$\mathcal{D}_{\text{disc}}$	$\mathcal{D}_2$ ($\sigma = 10$)	5.02×10^0	2.90×10^{-2}	2.90×10^{-3}	9.02×10^{-5}	PASS
$\mathcal{D}_{\text{disc}}$	$\mathcal{D}_3$ ($\sigma = 100$)	1.22×10^1	2.97×10^{-1}	2.97×10^{-3}	8.55×10^{-5}	PASS
$\mathcal{D}_{\text{logu}}$	$\mathcal{D}_1$ ($\sigma = 1$)	8.03×10^{-1}	9.58×10^{-3}	9.58×10^{-3}	1.48×10^{-4}	PASS
$\mathcal{D}_{\text{logu}}$	$\mathcal{D}_2$ ($\sigma = 10$)	7.90×10^{-1}	3.90×10^{-2}	3.90×10^{-3}	1.53×10^{-4}	PASS
$\mathcal{D}_{\text{logu}}$	$\mathcal{D}_3$ ($\sigma = 100$)	1.40×10^0	6.44×10^{-1}	6.44×10^{-3}	2.12×10^{-4}	PASS

Table 5: Random-ADP convergence diagnostic. Six training-distribution/test-regime cells. Production $(K_1, K_2, J[, J_\sigma]) = (256, 256, 64[, 64])$ vs. reference $(512, 512, 128[, 128])$. Gate: action-label disagreement $< 10^{-3}$.

H Agreement and Payoff

Per-step action agreement and the competitive ratio can differ sharply on the same (model, test regime) cell. A diagnostic example is $\mathcal{D}_3\text{-cv}$ on $\mathcal{D}_1$ (Figure 38): the model agrees with the per-regime oracle on 97.8% of decisions yet attains $R/R^* = 0.016$. The mismatch is mechanical: most timesteps are trivial rejects (the optimal action is “continue” and any sufficiently large threshold also rejects), inflating agreement; the few decision-critical timesteps determine the entire payoff. We retain agreement as a diagnostic accompanying R/R^* , never as a substitute for it.

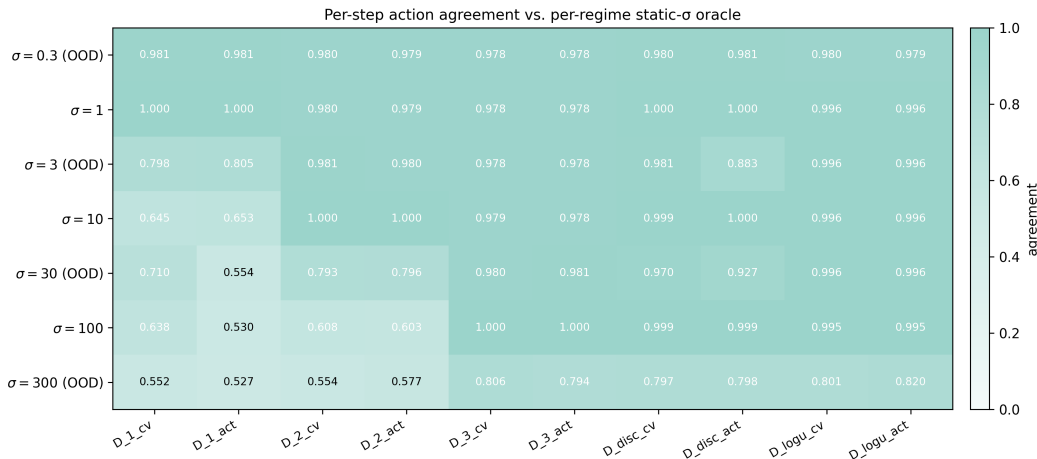


Figure 38: Per-step action agreement between each trained model (columns) and the per-regime static- σ oracle (rows), same axes as Figure 7.

I Alternative Supervision

We replace the ADP-oracle labels of Section 2.5 with their offline-optimum counterparts and retrain the four random-variance models. For each sampled $X_{1:n}$, the offline labels are $y_t^{\text{cv}} := \max_{s>t} X_s$ and $y_t^{\text{act}} := \mathbf{1}[X_t \geq \max_{s>t} X_s]$ for $t = 1, \dots, n-1$, with $y_n^{\text{act}} = 1$ and the continuation-value label at $t = n$ masked out. The labels are computed by a single reverse-cummax pass — $O(n)$ per sequence, no ADP table. All other hyperparameters match the oracle-supervised counterparts ($\mathcal{D}_{\text{disc-cv}}$, $\mathcal{D}_{\text{disc-act}}$, $\mathcal{D}_{\text{logu-cv}}$, $\mathcal{D}_{\text{logu-act}}$; the $1/\sigma_i^2$ continuation-value loss normalizer still applies

because the offline-optimum value scales with σ just like the oracle’s). Evaluation uses the same seven static- σ test caches and the same per-regime Bayes-optimal payoff R^* as Section 6.3.

Mean R/R^* gap (offline minus oracle) across the seven regimes: $\mathcal{D}_{\text{disc-cv}} -0.059$, $\mathcal{D}_{\text{disc-act}} -0.022$, $\mathcal{D}_{\text{logu-cv}} -0.004$, $\mathcal{D}_{\text{logu-act}} +0.016$. On $\mathcal{D}_{\text{logu}}$ both gaps are within 0.02 of zero; on $\mathcal{D}_{\text{disc}}$ they are concentrated almost entirely at the $\sigma = 3$ interpolation regime (cv -0.31 , act -0.13). There, the ADP-oracle-supervised model attains $1.6\times$ – $2.5\times$ higher R/R^* than the matched random-ADP-disc policy (Figure 14), while the offline-optimum labeler does not reproduce that off-prior interpolation behavior.

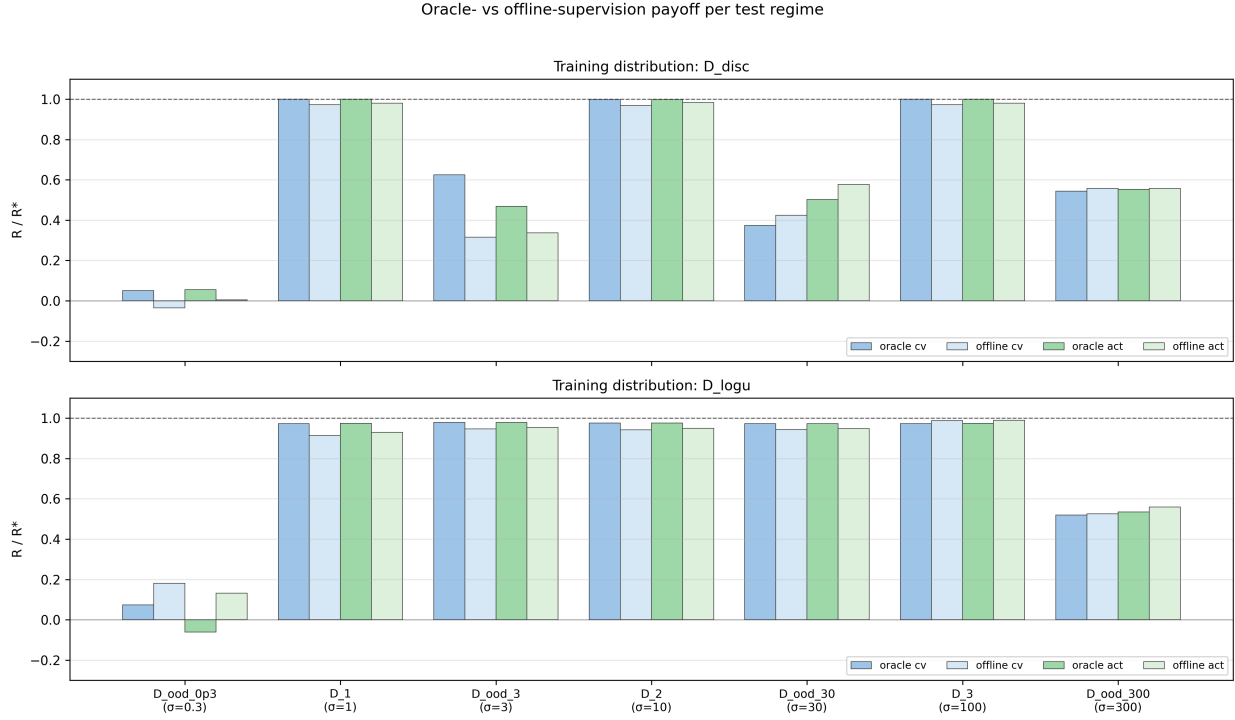


Figure 39: Oracle- vs. offline-supervision R/R^* per test regime, one panel per training distribution. On $\mathcal{D}_{\text{logu}}$ the four bars are visually indistinguishable on every in-prior regime; on $\mathcal{D}_{\text{disc}}$ the offline continuation-value bar dips below its oracle counterpart at $\sigma = 3$.

Conditioned on a prefix, the expected offline continuation label upper-bounds the online continuation value, but any single realised suffix maximum can lie above or below C_t^* . In the continuation-value runs, training on this realised lookahead target tends to produce a more conservative threshold. Figures 40 and 41 show this directly: the offline-supervised threshold often sits above its oracle-supervised counterpart. The effect is small on the log-uniform runs and on most discrete-prior regimes, but it is visible at the discrete prior’s $\sigma = 3$ interpolation point. Thus offline labels are a useful cheap sensitivity check, not a replacement for the ADP labels in the main experiments.

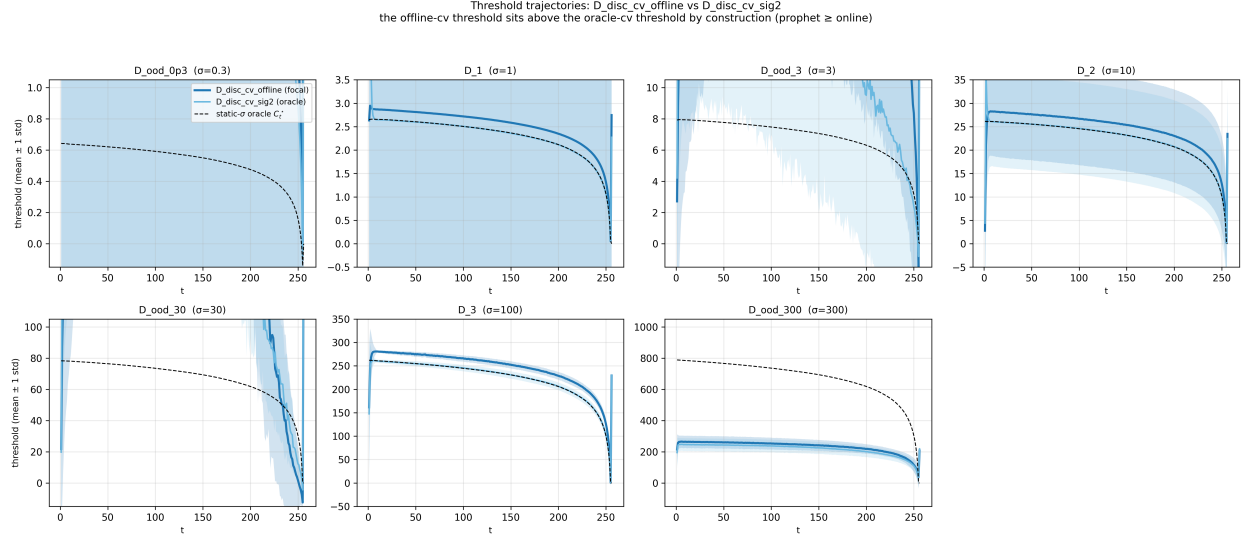


Figure 40: Threshold trajectories for $\mathcal{D}_{\text{disc-cv-offline}}$ (focal, blue, ± 1 std band), the oracle counterpart $\mathcal{D}_{\text{disc-cv}}$ (pale blue), and the per-regime static- σ oracle C_t^* (black dashed) on all seven test regimes. The offline-supervised threshold is generally more conservative, but it is a learned response to realised suffix labels rather than a pointwise oracle bound.

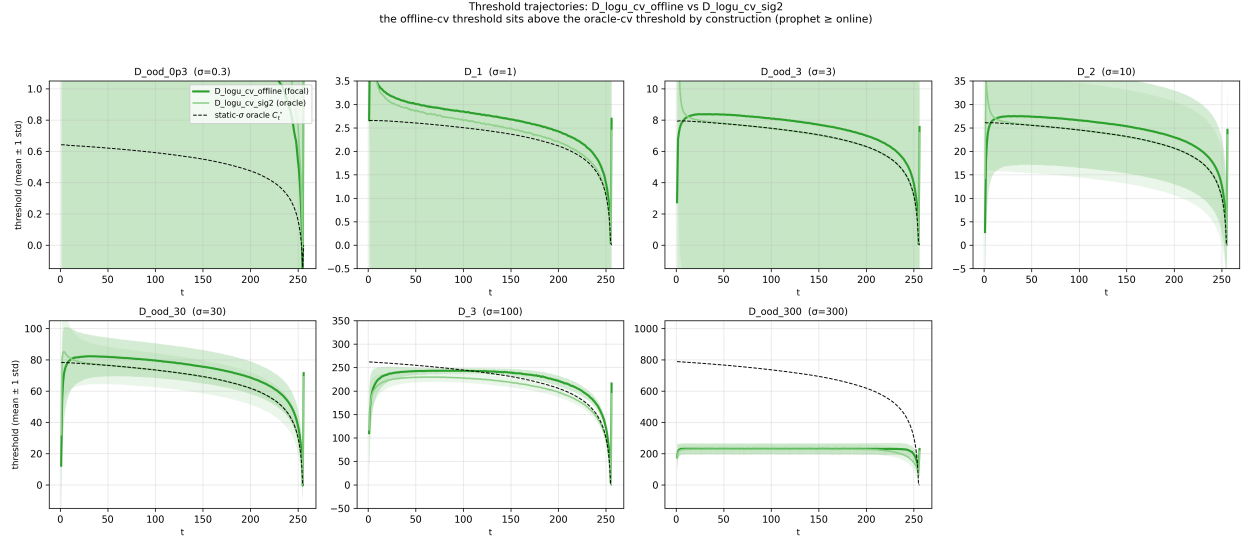


Figure 41: Same layout, focal $\mathcal{D}_{\text{logu-cv-offline}}$ (green). On $\mathcal{D}_{\text{logu}}$ the offline threshold tracks both the oracle's threshold and C_t^* closely on every regime.

J Temporal Shift

This appendix collects the per-shift summary statistics and per-sequence $t_{1/2}$ histograms summarised in Section 6.4.

set	model	shift	mean $t_{1/2}$	p_{10}	p_{50}	p_{90}	never	pre err	post err	ρ_{200}
B.1	$\mathcal{D}_{\text{disc-cv}}$	$\sigma=1 \rightarrow 100$	1.8	1	1	3	0	0.007	17.866	+0.999
B.1	$\mathcal{D}_{\text{logu-cv}}$	$\sigma=1 \rightarrow 100$	3.2	1	3	6	0	0.186	45.075	+0.910
B.1	$\mathcal{D}_{\text{disc-cv}}$	$\sigma=100 \rightarrow 1$	127.0	127	127	127	13	0.953	167.554	-0.010
B.1	$\mathcal{D}_{\text{logu-cv}}$	$\sigma=100 \rightarrow 1$	127.0	127	127	127	16	19.148	117.001	+0.050
B.1	$\mathcal{D}_{\text{disc-cv}}$	$\sigma=10 \rightarrow 100$	2.9	1	2	6	0	0.085	10.529	+0.995
B.1	$\mathcal{D}_{\text{logu-cv}}$	$\sigma=10 \rightarrow 100$	16.1	8	15	25	0	1.531	52.451	+0.798
B.1	$\mathcal{D}_{\text{disc-cv}}$	$\sigma=100 \rightarrow 10$	126.9	127	127	127	23	0.956	147.938	-0.011
B.1	$\mathcal{D}_{\text{logu-cv}}$	$\sigma=100 \rightarrow 10$	126.8	126	127	127	42	19.079	103.452	+0.098
B.2	$\mathcal{D}_{\text{disc-cv}}$	$\sigma=3 \rightarrow 30$	22.5	1	6	69	0	10.045	27.315	+0.787
B.2	$\mathcal{D}_{\text{logu-cv}}$	$\sigma=3 \rightarrow 30$	19.2	2	13	40	0	0.462	15.730	+0.796
B.2	$\mathcal{D}_{\text{disc-cv}}$	$\sigma=30 \rightarrow 3$	118.8	99	128	128	5502	157.042	38.454	-0.162
B.2	$\mathcal{D}_{\text{logu-cv}}$	$\sigma=30 \rightarrow 3$	125.7	121	128	128	6773	4.618	32.137	+0.096
B.2	$\mathcal{D}_{\text{disc-cv}}$	$\sigma=3 \rightarrow 300$	1.6	1	1	3	0	10.040	331.656	+0.751
B.2	$\mathcal{D}_{\text{logu-cv}}$	$\sigma=3 \rightarrow 300$	4.7	1	5	8	0	0.466	321.764	+0.744
B.2	$\mathcal{D}_{\text{disc-cv}}$	$\sigma=300 \rightarrow 3$	126.4	125	127	127	2	503.619	148.685	+0.234
B.2	$\mathcal{D}_{\text{logu-cv}}$	$\sigma=300 \rightarrow 3$	126.3	124	127	127	2	513.776	141.471	+0.226
B.2	$\mathcal{D}_{\text{disc-cv}}$	$\sigma=30 \rightarrow 300$	2.3	1	2	4	0	157.246	328.141	+0.514
B.2	$\mathcal{D}_{\text{logu-cv}}$	$\sigma=30 \rightarrow 300$	39.7	29	39	52	0	4.611	311.692	+0.521
B.2	$\mathcal{D}_{\text{disc-cv}}$	$\sigma=300 \rightarrow 30$	51.0	3	50	103	0	503.605	101.728	+0.489
B.2	$\mathcal{D}_{\text{logu-cv}}$	$\sigma=300 \rightarrow 30$	100.9	89	102	113	0	513.787	107.438	+0.451

Table 6: Per-sequence regime shift (B.1 + B.2), statistics over $N_{\text{test}} = 10^4$ sequences. “Never” = sequences whose ρ_t did not cross 0.5 in the post-shift window. Pre/post err: mean $|\hat{C}_t - C_{\sigma,t}^*|$ over $t \in [50, 128]$ and $[200, 255]$. ρ_{200} : population-mean adaptation at $t = 200$.

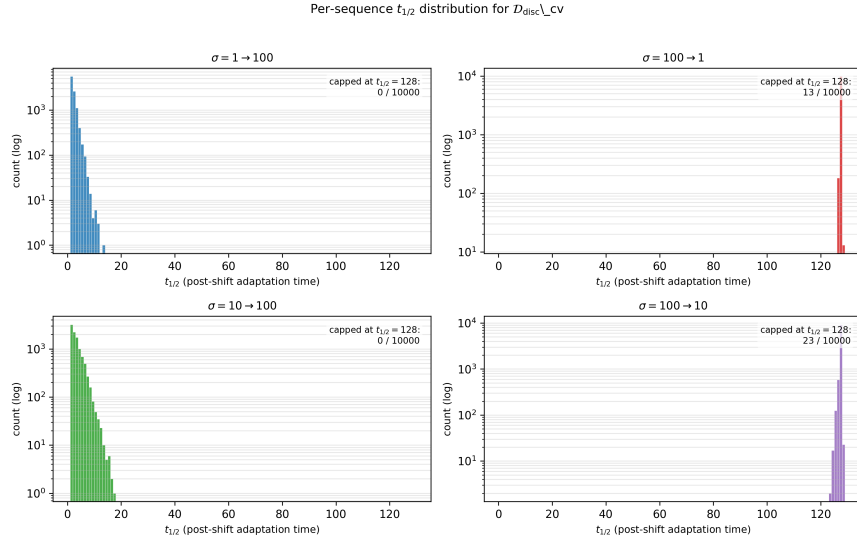


Figure 42: Per-sequence $t_{1/2}$ distributions for $\mathcal{D}_{\text{disc-cv}}$ on the four B.1 pairs.

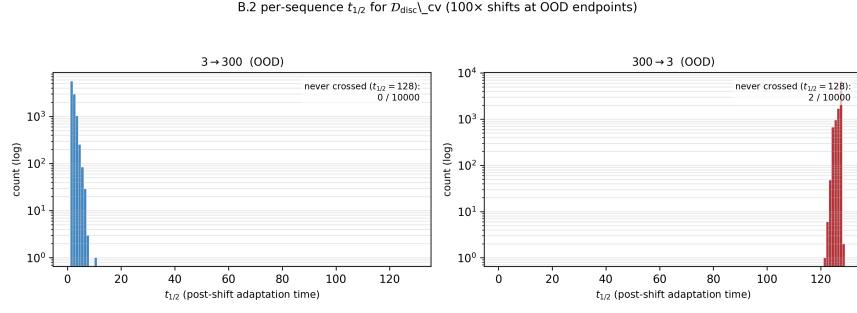


Figure 43: Per-sequence $t_{1/2}$ distributions for $\mathcal{D}_{\text{disc_cv}}$ on the two B.2 100× pairs.